

プログラムの設計方法 第二版（日本語訳）

Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

Contents

プログラムの設計方法 第二版	4
目次	4
前付け	4
本編	5
序文	7
体系的なプログラム設計	9
DrRacket とティーチング言語	11
移行可能なスキル	12
この本とその各部	12
違い点	12
プロローグ：プログラムの仕方	12
算術と算術	13
多くの方法で計算する	14
1つのプログラム、多くの定義	14
あなたはもうプログラマだ	14
I 固定サイズのデータ (Fixed-Size Data)	14
1 算術 (Arithmetic)	15
1.1 数の算術 (The Arithmetic of Numbers)	15
1.2 文字列の算術 (The Arithmetic of Strings)	15
1.3 混ぜ合わせ (Mixing It Up)	16
1.4 画像の算術 (The Arithmetic of Images)	16
1.5 ブール値の算術 (The Arithmetic of Booleans)	16
1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)	17
1.7 述語：自分のデータを知れ (Predicates: Know Thy Data)	17
2 関数とプログラム (Functions and Programs)	17
2.1 関数 (Functions)	17
2.2 計算 (Computing)	18
2.3 関数の合成 (Composing Functions)	18
2.4 グローバル定数 (Global Constants)	18
2.5 プログラム (Programs)	18
3 プログラムの設計方法	19
3.1 関数の設計	20
情報とデータ	20
設計プロセス	23

関数の例から設計へ	24
3.2 指の運動: 関数	24
3.3 ドメイン知識	25
3.4 関数からプログラムへ	25
3.5 テストについて	26
3.6 World プログラムの設計	26
3.7 バーチャルペットの世界	27
4.1 条件式を使ったプログラミング (Programming with Conditionals)	28
4.2 条件付きの計算 (Computing Conditionally)	30
4.3 列挙 (Enumerations)	32
4.4 区間 (Intervals)	34
4.5 項目化 (Itemizations)	36
4.6 項目化を用いた設計 (Designing with Itemizations)	38
4.7 有限状態世界 (Finite State Worlds)	39
5 構造の追加 (Adding Structure)	40
6 項目化と構造 (Itemizations and Structures)	40
7 まとめ (Summary)	40
Intermezzo 1: Beginning Student Language	40
BSL の語彙 (BSL Vocabulary)	41
BSL の文法 (BSL Grammar)	41
BSL の意味 (BSL Meaning)	42
意味とコンピューティング (Meaning and Computing)	42
BSL のエラー (BSL Errors)	42
ブール式 (Boolean Expressions)	42
定数定義 (Constant Definitions)	43
構造型定義 (Structure Type Definitions)	43
BSL のテスト (BSL Tests)	43
BSL のエラーメッセージ (BSL Error Messages)	43
8 リスト (Lists)	44
8.1 リストの作成 (Creating Lists)	44
8.2 ‘() とは何か、cons とは何か (What Is’ (), What Is cons)	48
8.3 リストを使ったプログラミング (Programming with Lists)	50
8.4 リストの計算 (Computing with Lists)	54
9 自己参照データ定義を用いた設計 (Designing with Self-Referential Data Definitions)	56
9.1 指の運動: リスト (Finger Exercises: Lists)	62
9.2 空でないリスト (Non-empty Lists)	63
9.3 自然数 (Natural Numbers)	69
9.4 ロシア人形 (Russian Dolls)	73

9.5 リストと World (Lists and World)	76
9.6 リストと集合についての注意 (A Note on Lists and Sets)	81
10 リストの更なる世界 (More on Lists)	85
10.1 リストを生成する関数 (Functions that Produce Lists)	85
10.2 リスト内の構造体 (Structures in Lists)	88
10.3 リストのリスト、およびテキストファイル (Lists in Lists, and Text Files)	92
10.4 グラフィカルエディタの再訪 (A Graphical Editor, Revisited)	98
11 合成による設計 (Design by Composition)	104
11.1 list 関数 (The list Function)	104
11.2 関数の合成 (Composing Functions)	106
11.3 再帰する補助関数 (Auxiliary Functions that Recur)	108
11.4 別の視点からの多角形描画 (Another Look at Polygon Rendering) . . .	112
12 プロジェクト: リスト (Projects: Lists)	115
12.1 現実世界のデータ: 辞書 (Real-World Data: Dictionaries)	115
12.2 現実世界のデータ: iTunes (Real-World Data: iTunes)	117
12.3 ワードゲーム: 合成 (Word Games: Composition)	119
12.4 ワードゲーム: 問題の核心 (Word Games: The Heart of the Problem) . .	120
12.5 ワードゲーム再訪とワームゲーム (Worm Game)	120
12.6 シンプル・テトリス (Simple Tetris)	121
12.7 フル・スペースウォー (Full Space War)	121
12.8 有限状態機械 (Finite State Machines)	121
13 まとめ (Summary)	122
II 任意サイズのデータ (Arbitrarily Large Data)	123
Intermezzo 2: Quote, Unquote	123
III 抽象化 (Abstraction)	123
導入: 抽象化の必要性	123
14 類似性はどこにでもある (Similarities Everywhere)	124
14.1 関数における類似性 (Similarities in Functions)	124
14.2 異なる類似性 (Different Similarities)	126
14.3 データ定義における類似性 (Similarities in Data Definitions) . . .	129
14.4 関数は値である (Functions Are Values)	132
14.5 関数を使った計算 (Computing with Functions)	133
15 抽象化の設計 (Designing Abstractions)	136
15.1 例からの抽象化 (Abstractions from Examples)	136
15.2 署名における類似性 (Similarities in Signatures)	139
15.3 単一の制御点 (Single Point of Control)	141
15.4 テンプレートからの抽象化 (Abstractions from Templates)	142
17 無名関数 (Nameless Functions)	143

17.1 lambda から生まれる関数 (Functions from lambda)	143
17.2 lambda を使った計算 (Computing with lambda)	143
17.3 lambda を使った抽象化 (Abstracting with lambda)	143
17.4 lambda を使った仕様 (Specifying with lambda)	143
17.5 lambda を使った表現 (Representing with lambda)	143
Intermezzo 3: Scope and Abstraction	143
IV 絡み合ったデータ (Intertwined Data)	143
Intermezzo 4: The Nature of Numbers	143
V 生成的再帰 (Generative Recursion)	143
Intermezzo 5: The Cost of Computation	144
VI 蓄積子 (Accumulators)	144
エピローグ: 先へ進む (Epilogue: Moving On)	144

プログラムの設計方法 第二版

How to Design Programs, Second Edition

著者: Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

© 2014 (更新版 2026) MIT Press

この資料は著作権保護されており、Creative Commons CC BY-NC-ND ライセンスの下で提供されています。

目次

前付け

- 序文 (Preface)
 - 体系的なプログラム設計
 - DrRacket とティーチング言語
 - 移行可能なスキル
 - この本とその各部
 - 違い点
- プロローグ: プログラムの仕方 (Prologue: How to Program)
 - 算術と算術
 - 入力と出力
 - 計算のさまざまな方法
 - 1つのプログラム、多くの定義
 - もう1つの定義
 - あなたはもうプログラマだ
 - 違う!

本編

I 固定サイズのデータ (Fixed-Size Data)

- 1 算術
 - ▶ 1.1 数の算術
 - ▶ 1.2 文字列の算術
 - ▶ 1.3 混ぜ合わせ
 - ▶ 1.4 画像の算術
 - ▶ 1.5 ブール値の算術
 - ▶ 1.6 ブール値と混ぜ合わせ
 - ▶ 1.7 述語: 自分のデータを知れ
- 2 関数とプログラム
 - ▶ 2.1 関数
 - ▶ 2.2 計算
 - ▶ 2.3 関数の合成
 - ▶ 2.4 グローバル定数
 - ▶ 2.5 プログラム
- 3 プログラムの設計方法 (How to Design Programs)
 - ▶ 3.1 関数の設計
 - ▶ 3.2 指の運動: 関数
 - ▶ 3.3 ドメイン知識
 - ▶ 3.4 関数からプログラムへ
 - ▶ 3.5 テストについて
 - ▶ 3.6 World プログラムの設計
 - ▶ 3.7 バーチャルペットの世界
- 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)
 - ▶ 4.1 条件式を使ったプログラミング
 - ▶ 4.2 条件付き計算
 - ▶ 4.3 列挙
 - ▶ 4.4 区間
 - ▶ 4.5 項目化
 - ▶ 4.6 項目化を使った設計
 - ▶ 4.7 有限状態の世界
- 5 構造の追加 (Adding Structure)
 - ▶ 5.1 位置から posn 構造へ
 - ▶ 5.2 posn を使った計算
 - ▶ 5.3 posn を使ったプログラミング
 - ▶ 5.4 構造型定義
 - ▶ 5.5 構造を使った計算
 - ▶ 5.6 構造を使ったプログラミング

- ▶ 5.7 データの宇宙
- ▶ 5.8 構造を使った設計
- ▶ 5.9 World 中の構造
- ▶ 5.10 グラフィカルエディタ
- ▶ 5.11 さらに多くのバーチャルペット
- 6 項目化と構造 (Itemizations and Structures)
 - ▶ 6.1 再び、項目化を使った設計
 - ▶ 6.2 World を混ぜる
 - ▶ 6.3 入力エラー
 - ▶ 6.4 World の検査
 - ▶ 6.5 等価述語
- 7 まとめ

Intermezzo 1: Beginning Student Language

BSL の語彙、文法、意味、エラーなど

II 任意サイズのデータ (Arbitrarily Large Data)

- 8 リスト (Lists)
- 9 自己参照データ定義を使った設計
- 10 リストの続き
- 11 合成による設計 (Design by Composition)
- 12 プロジェクト: リスト
 - ▶ 実世界データ: 辞書、iTunes
 - ▶ 単語ゲーム、ワーム、単純テトリス、完全宇宙戦争、有限状態機械
- 13 まとめ

Intermezzo 2: Quote, Unquote

III 抽象化 (Abstraction)

- 14 どこにでも類似点がある
- 15 抽象化の設計
- 16 抽象化の使用
- 17 無名関数 (Nameless Functions / lambda)
- 18 まとめ

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ (Intertwined Data)

- 19 S 式の詩 (The Poetry of S-expressions)
- 20 反復的洗練 (Iterative Refinement)
- 21 インタプリタの洗練
- 22 プロジェクト: XML の商業

- 23 同時処理
- 24 まとめ

Intermezzo 4: The Nature of Numbers

V 生成的再帰 (Generative Recursion)

- 25 非標準の再帰
- 26 アルゴリズムの設計
- 27 バリエーション
- 28 数学的な例
- 29 バックトラックするアルゴリズム
- 30 まとめ

Intermezzo 5: The Cost of Computation

VI 蓄積子 (Accumulators)

- 31 知識の喪失
- 32 蓄積子スタイル関数の設計
- 33 蓄積のさらなる使用
- 34 まとめ

エピローグ: 先へ進む (Epilogue: Moving On)

注意: 本翻訳は学習・個人利用のためのものです。サンプルコードは原文のまま保持しています。

原典: <https://htdp.org/2026-5-28/Book/index.html>

序文

Preface

序文

プロローグ：プログラムの仕方

I 固定サイズのデータ

Intermezzo 1: Beginning Student Language

II 任意サイズのデータ

Intermezzo 2: Quote, Unquote

III 抽象化

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ

Intermezzo 4: The Nature of Numbers

V 生成的再帰

Intermezzo 5: The Cost of Computation

VI 蓄積子

エピローグ：先へ進む

多くの職業で何らかのプログラミングが必要です。会計士はスプレッドシートをプログラムし、ミュージシャンはシンセサイザーをプログラムし、著者はワードプロセッサをプログラムし、Web デザイナーはスタイルシートをプログラムします。私たちが初版のこれらの言葉を書いたとき（1995-2000）、読者はそれらを未来的だと考えたかもしれません。今や、プログラミングは必須のスキルとなり、雇用見通しを高めるという目的で、数多くの書籍、オンラインコース、K-12 カリキュラムがこのニーズに応えています。

典型的なプログラミングのコースでは、「動くまでいじくり回す」アプローチを教えます。それが動くと、学生は「動いた！」と叫んで次に進みます。悲しいことに、このフレーズはコンピューティングにおける最短の嘘でもあり、多くの人々の人生から何時間も奪ってきました。対照的に、本書は良いプログラミングの習慣に焦点を当て、プロフェッショナルおよび職業プログラマの両方を対象としています。

「良いプログラミング」とは、ソフトウェア作成へのアプローチで、最初から、すべての段階で、すべてのステップで体系的な思考、計画、理解に依拠するものを意味します。この点を強調するために、私たちは「体系的なプログラム設計」と「体系的に設計されたプログラム」について語ります。後者は、望まれる機能の根拠を明確に述べます。良いプログラミングは達成感の美的感覚も満たします。良いプログラムのエレガンスは、時の試練に耐えた詩や過去の時代の白黒写真に匹敵します。要するに、プログラミングと良いプログラミングの違いは、ダイナーのクレヨン画と美術館の油絵のようなものです。

いいえ、この本は誰をもマスターペインターには変えません。しかし、私たちがこの版を書くのに 15 年を費やしたのは、以下を信じているからに他なりません。

誰もがプログラムを設計できる

そして

誰もが創造的設計から得られる満足感を経験できる

実際、私たちはさらに進んで主張します。

プログラム設計— しかしプログラミングではない— は、リベラルアーツ教育において数学や言語スキルと同じ役割を果たすべきである。

設計を学ぶ学生が二度とプログラムに触れなくても、普遍的に有用な問題解決スキルを身につけ、深く創造的な活動を経験し、新しい形式の美を理解することを学ぶでしょう。この序文の残りでは、「体系的設計」とは何を意味するのか、誰がどのように恩恵を受けるのか、そしてそれをどのように教えるのかを詳しく説明します。

体系的なプログラム設計

プログラムは人々（ユーザーと呼ばれる）と他のプログラム（サーバーとクライアントコンポーネントの場合）と相互作用します。したがって、合理的に完全なプログラムは多くの構成要素から成ります。一部は入力扱い、一部は出力を作成し、一部はそれらの間のギャップを橋渡しします。私たちは関数を基本的な構成要素として選ぶ理由は、誰もが前代数で関数に遭遇するからであり、最も単純なプログラムがまさにそのような関数だからです。鍵は、どの関数が必要か、それらをどのように接続するか、そして基本的な材料からそれらをどのように構築するかを発見することです。

この文脈で、「体系的なプログラム設計」とは、2つの概念の混合を指します。設計レシピと反復的洗練です。私たちはMichael JacksonのCOBOL プログラム作成法、Daniel Friedmanとの再帰に関する会話、Robert Harperとの型理論、Daniel Jacksonとのソフトウェア設計から着想を得ました。設計レシピは著者らの創作であり、ここでは後者の使用を可能にします。

問題分析からデータ定義へ 表現しなければならない情報と、それが選択したプログラミング言語でどのように表現されるかを特定せよ。データ定義を定式化し、例で説明せよ。 署名、目的文、ヘッダ 望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。その関数が何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に沿ったスタブを定義せよ。 関数の例 関数の目的を説明する例に取り組み。 関数のテンプレート データ定義を関数のアウトラインに翻訳せ

よ。関数定義 関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。テスト 例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。これにより誤りが発見される。テストはまた、必要が生じたときに他の人が定義を読み理解するのを助ける—そしてそれは深刻なプログラムでは必ず生じる。

図 1: 関数設計レシピの基本ステップ

設計レシピは完全なプログラムと個々の関数の両方に適用されます。本書は完全なプログラムのための2つのレシピだけを扱います。GUIを持つプログラム用とバッチプログラム用です。一方、関数用の設計レシピにはさまざまなバリエーションがあります。数値のような原子的なデータ形式用、さまざまな種類のデータの列挙用、固定された方法で他のデータを複合するデータ用、有限だが任意に大きいデータ用、など。

関数レベルの設計レシピは共通の設計プロセスを共有します。図 1 はその6つの必須ステップを示しています。各ステップのタイトルは期待される成果を指定し、「コマンド」は主要な活動を示唆します。例はほぼすべての段階で中心的な役割を果たします。

各ステップには鋭い質問が付随しており、本書の6つのパートを通じて導入されます。特定のステップ—例えば関数の例の作成やテンプレート—では、質問はデータ定義に訴えかけるかもしれません。答えはほぼ自動的に中間成果物を作成します。

このアプローチの新しさは、初心者レベルのプログラムのための中間成果物の作成にあります。初心者が詰まったとき、専門家や講師は既存の中間成果物を検査できます。検査は設計プロセスの一般的な質問を使う可能性が高く、したがって初心者に自分で修正させることになります。そしてこの自己強化プロセスこそが、プログラミングとプログラム設計の重要な違いです。

反復的洗練は、問題が複雑で多面的であるという問題に対処します。一度にすべてを正しくすることはほぼ不可能です。代わりに、コンピュータ科学者はこの設計問題に取り組むために物理科学から反復的洗練を借用します。本質的に、反復的洗練は、最初にすべての非本質的な詳細を取り除き、残された中核問題の解決策を見つけることを推奨します。洗練ステップは、これらの省略された詳細の1つを追加し、既存の解決策を可能な限り使用して拡張された問題を再解決します。これらの洗練ステップの繰り返し（反復とも呼ばれる）は、最終的に完全な解決策につながります。

この意味で、プログラマはミニ科学者です。科学者は世界の理想化されたバージョンのための近似モデルを作成し、それについての予測を行います。モデルの予測が当たっている限り、すべてはうまくいきます。予測された出来事が実際のもthingと異なる場合、科学者はその不一致を減らすためにモデルを改訂します。同様に、プログラマにタスクが与えられたとき、彼らは最初の設計を作成し、それをコードにし、実際のユーザーで評価し、プログラムの振る舞いが望まれる製品に密接に一致するまで設計を反復的に洗練します。

本書は2つの異なる方法で反復的洗練を導入します。洗練による設計がプログラムの設計が複雑になると有用になるため、本書は問題がある程度の難易度を獲得した第4部でこ

の技法を明示的に導入します。さらに、私たちは第1部から第3部を通じて同じ問題のますます複雑なバリエーションを述べるために反復的洗練を使用します。つまり、中核問題を選び、1つの章でそれを扱い、その後、後続の章で新たに導入された概念に合致する詳細で類似の問題を提示します。

DrRacket とティーチング言語

プログラムを設計することを学ぶには、繰り返しのハンズオン練習が必要です。ピアノを弾かずにピアノ奏者になれないのと同じように、実際のプログラムを作成して正しく動作させることなしにプログラム設計者にはなれません。したがって、本書には最小限のソフトウェアサポートが付属しています。プログラムを書き留めるための言語と、プログラムをワード文書のように編集し、プログラムを実行できるプログラム開発環境です。

多くの人が「コードの書き方を学びたい」と言い、次にどのプログラミング言語を学ぶべきかを尋ねます。いくつかのプログラミング言語が得ている注目を考えると、この質問は驚くべきことではありません。しかし、それは全く不適切でもあります。初心者を対象としないコースでは、既製の言語を設計レシピとともに使用できるかもしれません。現在流行のプログラミング言語でプログラミングを学ぶことは、学生を最終的な失敗に導くことがよくあります。この世界での流行は非常に短命です。「Xで素早くプログラミング」本やコースは、次の流行の言語に移行できる原則を教えることに失敗することが多いです。さらに悪いことに、言語自体が、解決策を表現するレベルでもプログラミングミスに対処するレベルでも、移行可能なスキルの習得から気を逸らします。

対照的に、プログラムを設計することを学ぶことは、主に原則の研究と移行可能なスキルの習得についてです。理想的なプログラミング言語はこれら2つの目標をサポートしなければなりませんが、既製の産業用言語はそうしません。重要な問題は、初心者が言語の多くを知る前に間違いを犯すことです。しかしプログラミング言語は、プログラマがすでに言語全体を知っているかのように常にこれらのエラーを診断します。その結果、診断レポートは初心者をしばしば困惑させます。

私たちの解決策は、私たち自身の仕立てられたティーチング言語から始めることです。「Beginning Student Language」またはBSLと呼ばれます。この言語は本質的に、学生が前代数のコースで習得する「外国語」です。関数定義、関数適用、条件式のための記法を含みます。また、式を入れ子にすることもできます。この言語は非常に小さいので、言語全体の観点からのエラー診断でも、前代数しか持たない読者にとってまだアクセス可能です。

構造的設計原則を習得した学生は、次に「Intermediate Student Language」と他の高度な方言（総称して*SL）に進むことができます。本書はこれらの方言を使用して抽象化と一般再帰の設計原則を教えます。私たちは、このような一連のティーチング言語を使用することが、幅広い専門プログラミング言語（JavaScript、Python、Ruby、Javaなど）でプログラムを作成するための優れた準備を読者に提供すると固く信じています。

注：ティーチング言語は Racket で実装されています。Racket はプログラミング言語を構築するためのプログラミング言語として私たちが構築したものです。Racket はラボから現実世界に逃れ出ました。それはプログラミング言語です。

(続き：DrRacket の使用、ティーチング言語の選択など詳細)

移行可能なスキル

(中略：設計レシピから得られる普遍的な問題解決スキル、創造的活動としてのプログラミングの価値について)

この本とその各部

本書は 6 つの主要なパートと 5 つの Intermezzo に分かれています。各パートは設計レシピの特定の側面に焦点を当て、徐々に複雑さを増していきます。

- Part I: 固定サイズのデータ – 基本的な算術から設計レシピの導入まで。
- Part II: 任意サイズのデータ – リストと自己参照データ定義。
- Part III: 抽象化 – 類似性の発見と高階関数。
- Part IV: 絡み合ったデータ – 複雑なデータ定義の相互作用。
- Part V: 生成的再帰 – 構造的再帰を超えたアルゴリズム。
- Part VI: 蓄積子 – 効率的な再帰のための追加の知識の保持。

各 Intermezzo では、言語の形式的な側面（文法、意味、計算モデル）を扱います。

違い点

(初版との違いの説明：より明確な設計レシピ、プロジェクトの強化など)

謝辞 (Acknowledgments)

… (詳細は原典を参照。多くの貢献者への感謝)

本翻訳は原典の内容を忠実に日本語化したものです。サンプルコードは原文を維持しています。

原典 URL: https://htdp.org/2026-5-28/Book/part_preface.html

プロローグ：プログラムの仕方

Prologue: How to Program

(目次リンクは省略)

あなたが小さな子供だったとき、親はあなたに数を数え、指で簡単な計算をすることを教えました。「 $1 + 1$ は 2 」;「 $1 + 2$ は 3 」; など。そして「 $3 + 2$ は？」と尋ね、あなたは片手の指を数えました。彼らはプログラムし、あなたは計算しました。そしてある意味で、それが本当にプログラミングとコンピューティングのすべてです。

今度は役割を交代する時です。DrRacket を起動しましょう。そうすると DrRacket のウィンドウが表示されます。「Language」メニューから「Choose language」を選び、「How to Design Programs」の「Teaching Languages」から「Beginning Student」(BSL) を選択し、OK をクリックして DrRacket を設定します。

これで準備完了です。あなたがプログラムし、DrRacket ソフトウェアが子供の役割を果たします。

最も単純な計算から始めましょう。DrRacket の上部に次のように入力します：

(+ 1 1)

RUN をクリックすると、下部に 2 が表示されます。

(図：DrRacket の画面)

それほど単純なのがプログラミングです。DrRacket を子供のように質問し、DrRacket があなたのために計算します。一度に複数のリクエストを処理させることもできます：

(+ 2 2)

(* 3 3)

(- 4 2)

(/ 6 2)

RUN をクリックすると、期待される結果 4 9 2 3 が表示されます。

少しペースを落として用語を導入しましょう。

DrRacket の上半分は「定義領域 (definitions area)」と呼ばれます。この領域でプログラムを作成します。これを編集と呼びます。定義領域に単語を追加したり変更したりすると、左上に SAVE ボタンが表示されます。初めて SAVE をクリックすると、DrRacket はファイルを保存するための名前を尋ねます。一度ファイルに関連付けられると、SAVE をクリックすると定義領域の内容が安全にファイルに保存されます。

プログラムは式 (expressions) で構成されます。数学で式に出会ったことがあるでしょう。今のところ、式は単なる数値か、左括弧「(」で始まり対応する右括弧「)」で終わるものです—DrRacket はその括弧の間の領域を強調表示します。

RUN をクリックすると、DrRacket は定義領域の式を評価し、その結果を対話領域 (interactions area) に表示します。その後、DrRacket はプロンプト (>) であなたのコマンドを待ちます。プロンプトが表示されたら、追加の式を入力でき、DrRacket は定義領域と同じようにそれを評価します。

このようなプログラミングとコンピューティングを DrRacket で簡単に探求できます。

算術と算術

プログラミングが数と算術だけだったら、数学と同じくらい退屈でしょう。幸い、プログラミングには数値以外にも多くのものがあります。テキスト、真偽値、画像などです。

BSL で知っておくべき最初のことは、テキストは二重引用符（ “ ）で囲まれた任意のキーボード文字のシーケンスであるということです。これを文字列（string）と呼びます。したがって、

```
"hello world"
```

は完全に正しい文字列であり、DrRacket がこの文字列を評価すると、対話領域で数値と同じようにそのままエコーバックします。

BSL には数の算術に加えて、文字列の算術もあります。以下に 2 つの例を示します：

```
(string-append "hello" "world")  
; "helloworld"
```

```
(string-append "hello " "world")  
; "hello world"
```

+ と同様に、string-append は演算子です。2 番目の文字列を最初の文字列の末尾に追加して新しい文字列を作ります。最初の例が示すように、2 つの文字列の間に何も挿入せずに文字通り結合します。空白、カンマ、何もありません。

(以下、画像の算術、ブール値、条件式、関数の定義などプロローグの続きが展開されます。)

多くの方法で計算する

(条件式の導入例)

1 つのプログラム、多くの定義

(define を使った定数と関数の導入)

あなたはもうプログラマだ

ここまで来たら、あなたはすでに小さなプログラムを書けるプログラマです。以降の章で設計レシピを学び、体系的にプログラムを構築していきます。

注意：サンプルコードは原文のままです。

原典：https://htdp.org/2026-5-28/Book/part_prologue.html

I 固定サイズのデータ (Fixed-Size Data)

本書の第 I 部では、基本的な原子データ（数値、文字列、画像、真偽値）を操作する「算術」と、それらを組み合わせた最も単純なプログラムである「関数」について学び、体系的なプログラム設計の最初の一步を踏み出します。

1 算術 (Arithmetic)

BSL (Beginning Student Language) における算術は、前置表記 (Prefix Notation) で記述されます。基本規則： 1. 開き括弧 (を書く 2. 基本演算の名前 (+ など) を書く 3. 引数をスペースで区切って書く 4. 閉じ括弧) を書く

例：

```
(+ 1 2)
```

ネスト (入れ子) された例：

```
(+ 1 (+ 1 (+ 1 1) 2) 3 4 5)
```

評価は、まず引数となる式をすべて評価し、その結果のデータ片を演算に与えて計算します。

```
(+ 1 2)
```

```
==
```

```
3
```

1.1 数の算術 (The Arithmetic of Numbers)

BSL は自然数、整数、有理数、実数、複素数をサポートし、正確数 (Exact) と不正確数 (Inexact) を区別します。

演算例： +, -, *, /, abs, add1, ceiling, denominator, exact->inexact, expt, floor, gcd, log, max, numerator, quotient, random, remainder, sqr, tan

```
> (sin 0)
```

```
0
```

また、円周率 pi やオイラーの定数 e も事前定義されています。不正確数 (近似値) には #i プレフィックスが付きます (例：#i1.4142135623730951)。

Exercise 1. 定義領域に以下を追加し、デカルト平面上の点 (x, y) から原点 (0, 0) までの距離を計算する式を記述せよ。

```
(define x 3)
```

```
(define y 4)
```

1.2 文字列の算術 (The Arithmetic of Strings)

文字列はダブルクォーテーション " で囲まれた文字列です。

演算例： string-append, string-length, substring, string-ith

```
(string-append "hello" " " "world")
```

```
; "hello world"
```

Exercise 2. prefix と suffix という名前の文字列定数を定義し、それらを結合する式を記述せよ。

1.3 混ぜ合わせ (Mixing It Up)

文字列と数値を相互に変換したり、型を操作する演算です。

演算例: number->string, string->number, string-length

```
(string-length "42")  
==  
2
```

Exercise 3. 文字列 `str` と数値 `i` を定義し、`str` の `i` 番目の位置に `_` を挿入する式を記述せよ。

```
(define str "helloworld")  
(define i 5)
```

Exercise 4. 文字列 `str` と数値 `i` が与えられたとき、`i` 番目の文字を削除する式を記述せよ。

1.4 画像の算術 (The Arithmetic of Images)

2http/image ティーチパックを使用すると、画像をデータとして操作できます。

演算例: circle, rectangle, empty-scene, place-image, overlay, image-width, image-height

Exercise 5. 1 枚の画像のピクセル数を計算する式を作成せよ。

```
(define cat <画像>)
```

Exercise 6. 任意の画像 `cat` が与えられたとき、それが縦長か、横長か、正方形かを判定する式を作成せよ。

Exercise 7. 以下の画像作成を試せ。

```
(circle 10 "solid" "red")  
(rectangle 10 20 "outline" "blue")
```

Exercise 8. 青いバナーの上に赤い文字でテキストを表示する画像を生成する式を記述せよ。

1.5 ブール値の算術 (The Arithmetic of Booleans)

真偽値 (`#true`, `#false`) と、論理演算 (`and`, `or`, `not`)、および比較述語 (`<`, `>`, `=`, `string=?`) です。

```
(or #false #true)  
==  
#true
```

Exercise 9. 画像 `in` に対して、それが縦長であるか、または横長であるかを判定するブール値の式を作成せよ。

1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)

条件に応じて処理を分岐させるための `if` 式です。

```
(if (string=? "red" "green") "stop" "go")  
==  
"go"
```

Exercise 10. 与えられた画像が縦長なら `"tall"`、横長なら `"wide"`、正方形なら `"square"` を返す `if` 式を記述せよ。

1.7 述語: 自分のデータを知れ (Predicates: Know Thy Data)

データ型を検査するための述語です。 `number?`, `string?`, `image?`, `boolean?`, `any?`

Exercise 11. `number?` や `string?` の動作を確認する式を記述せよ。

Exercise 12. 値 `v` が与えられたとき、それが画像ならその面積、文字列ならその長さ、数値ならその値自体を返す条件分岐式を記述せよ。

2 関数とプログラム (Functions and Programs)

プログラムの半分は算術であり、もう半分はそれらを再利用可能にする「関数」です。

2.1 関数 (Functions)

`define` を使って新しい関数を定義します。

```
(define (f x) (+ x 10))
```

Exercise 13. 与えられた文字列の最初の文字を返す関数 `string-first` を定義せよ。

Exercise 14. 与えられた文字列の最後の文字を返す関数 `string-last` を定義せよ。

Exercise 15. 2つのブール値 `sunny` と `friday` を消費し、金曜日で晴れの日であるかを判定する関数 `play` を定義せよ。

Exercise 16. 画像を消費し、そのピクセル面積を計算する関数 `image-area` を定義せよ。

Exercise 17. 画像を消費し、それが縦長であるかを判定する関数 `image-classify` を定義せよ。

Exercise 18. 2つの文字列を消費し、それらを `"_"` で結合する関数 `string-join` を定義せよ。

Exercise 19. 文字列 `str` と位置 `i` を消費し、`i` 番目に `"_"` を挿入する関数 `string-insert` を定義せよ。

Exercise 20. 文字列 `str` と位置 `i` を消費し、`i` 番目の文字を削除する関数 `string-delete` を定義せよ。

2.2 計算 (Computing)

関数の評価は、実引数 (Argument) を仮引数 (Parameter) に置き換える「代入モデル」に従って行われます。

```
(f 2)
==
(+ 2 10)
==
12
```

Exercise 21. (ff 2) がどのように評価されるか代入ステップを記述せよ。

```
(define (ff x) (* 10 x))
```

Exercise 22. (distance 3 4) の評価ステップを記述せよ。

Exercise 23. (string-first "hello") の評価ステップを記述せよ。

Exercise 24. (implies #true #false) の評価ステップを記述せよ。

Exercise 25. 画像分類関数の評価ステップを記述せよ。

Exercise 26. (string-insert "hello" 2) の評価ステップを記述せよ。

2.3 関数の合成 (Composing Functions)

複雑なタスクは、単純な関数を組み合わせることで解決します。

```
(define (profit price)
  (- (revenue price) (cost price)))
```

Exercise 27. 映画館のチケット価格と利益の関係をモデル化するプログラムを構築せよ。

Exercise 28. チケット価格ごとの利益をシミュレーションし、最適なチケット価格を見つけよ。

Exercise 29. コスト構造が変化した場合の最適なチケット価格を再計算せよ。

2.4 グローバル定数 (Global Constants)

変更に強いプログラムを作るために、マジックナンバーをグローバル定数として定義します。

```
(define BASE-PRICE 5)
```

Exercise 30. 映画館の利益計算プログラムにおける定数（基本価格、固定費、変動費など）を定数定義として分離せよ。

2.5 プログラム (Programs)

BSL では 2http/batch-io を使ったバッチ処理プログラムや、2http/universe を使った対話型（イベント駆動）プログラム (big-bang) を設計できます。

```
(big-bang 0
  [on-tick subl]
  [to-draw render]
  [on-key handle-key])
```

Exercise 31. ファイルからテキストを読み込んで処理するバッチプログラムを作成せよ。

Exercise 32. big-bang を使って、画面を動く UF0 やロケットのアニメーションプログラムを作成せよ。

3 プログラムの設計方法

本書の最初の数章では、プログラミングを学ぶには多くの概念をある程度習得する必要があることを示してきました。一方では、プログラミングには言語が必要です。つまり、計算したいことを伝えるための表記法です。プログラムを定式化するための言語は人工的な構成物ですが、プログラミング言語の習得は自然言語の習得といくつかの要素を共有しています。どちらも語彙、文法、そして「句」が何を意味するかの理解を伴います。

他方では、問題文からプログラムに至る方法を学ぶことが重要です。問題文の中で何が関連していて、何を無視できるかを判断する必要があります。プログラムが何を消費し、何を生成し、入力と出力をどのように関連付けるかを明らかにする必要があります。選択した言語とそのティーチパックが、プログラムが処理するデータに対する特定の基本演算を提供しているかどうかを知る（または調べる）必要があります。提供していない場合は、それらの演算を実装する補助関数を開発しなければならないかもしれません。最後に、プログラムができたなら、それが実際に意図した計算を行っているかどうかを確認しなければなりません。そして、これはあらゆる種類のエラーを明らかにする可能性があり、それらを理解して修正できなければなりません。

これらすべてはかなり複雑に聞こえ、なぜただ実験しながら適当にやって、結果がまあまあならそのままにしておけばいいのかと思うかもしれません。このプログラミングへのアプローチはしばしば「ガレージプログラミング」と呼ばれ、多くの場合に成功します。時にはスタートアップ企業の立ち上げの足がかりになることもあります。しかし、スタートアップはその「ガレージ努力」の結果を販売することはできません。なぜなら、元のプログラマとその友人しか使えないからです。

良いプログラムには、それが何をするか、どのような入力を期待するか、何を生成するかを説明する短い書き込みが付属しています。理想的には、それが実際に動作することをある程度保証するものも付きます。最良の場合、プログラムの問題文へのつながりが明白なので、問題文への小さな変更をプログラムへの小さな変更簡単に翻訳できます。ソフトウェアエンジニアはこれを「プログラミング製品」と呼びます。

このすべての追加作業が必要なのは、プログラマが自分自身のためにプログラムを作成するわけではないからです。プログラマは、他のプログラマが読むためにプログラムを書きます。そして時折、人々は仕事をするためにこれらのプログラムを実行します。

「他の」という言葉には、通常、若い頃の自分がプログラムの制作に込めた思考のすべてを思い出すことができない年上のバージョンのプログラマも含まれます。

ほとんどのプログラムは、協力し合う関数の大規模で複雑な集合であり、誰もが1日でこれらすべての関数を書くことはできません。プログラマはプロジェクトに参加し、コードを書き、プロジェクトを去ります。他の人がそのプログラムを引き継いで作業します。もう一つの困難は、プログラマのクライアントが、本当に解決してほしい問題について考えを変えがちだということです。彼らは通常、ほぼ正しく理解していますが、多くの場合、いくつかの詳細を間違えます。さらに悪いことに、プログラムのような複雑な論理的構成物は、ほとんど常に人間のエラーに悩まされます。要するに、プログラマは間違いを犯します。最終的に誰かがこれらのエラーを発見し、プログラマはそれらを修正しなければなりません。彼らは1ヶ月前、1年前、または20年前のプログラムを読み直して変更する必要があります。

Exercise 33. 「year 2000」問題を調べよ。

ここでは、ステップバイステップのプロセスと、問題データを中心にプログラムを組織する方法を統合した設計レシピを提示します。長い間空白の画面を見つめ続けるのが嫌いな読者にとって、この設計レシピは体系的な方法で進捗を達成する方法を提供します。他の人をプログラム設計に教える人にとって、レシピは初心者の困難を診断するための道具です。他の人々にとっては、このレシピは医学、ジャーナリズム、工学など他の分野にも適用できるものかもしれません。本物のプログラマになりたい人にとって、設計レシピは既存のプログラムを理解し作業するための方法も提供します—ただし、すべてのプログラマがこのような設計レシピのような方法を使ってプログラムを考え出すわけではありません。この章の残りは、設計レシピの世界への最初の一步を捧げます。以降の章とパートは、レシピを何らかの形で洗練・拡張していきます。

3.1 関数の設計

情報とデータ

プログラムの目的は、ある情報を消費し、新しい情報を生成する計算プロセスを記述することです。この意味で、プログラムは数学の教師が生徒に与える指示のようなものです。しかし生徒とは異なり、プログラムは数値以上のものを扱います。ナビゲーション情報を計算し、人の住所を調べ、スイッチを入れ、ビデオゲームの状態を検査します。これらすべての情報は現実世界の一部—しばしばプログラムのドメインと呼ばれる—から来ており、プログラムの計算結果はこのドメイン内のさらなる情報を表します。

情報は私たちの説明で中心的な役割を果たします。情報をプログラムのドメインに関する事実として考えてください。家具カタログを扱うプログラムにとって、「5本脚のテーブル

ル」や「2メートル四方の正方形テーブル」は情報の断片です。ゲームプログラムは異なる種類のドメインを扱い、「5」はあるオブジェクトがキャンバスのある部分から別の部分へ移動する際の、クロックティックあたりのピクセル数を指すかもしれません。また、給与計算プログラムは「5つの控除」を扱う可能性が高いでしょう。

プログラムが情報を処理するためには、それをプログラミング言語の何らかの形のデータに変換しなければなりません。そしてデータを処理し、完了したら結果のデータを再び情報に変換します。対話型プログラムはこれらのステップをさらに混ぜ合わせ、必要に応じて世界からより多くの情報を取得し、その間に情報を配信するかもしれません。

私たちはBSLとDrRacketを使用するので、情報からデータへの変換について心配する必要はありません。DrRacketのBSLでは、関数をデータに直接適用して何を生成するかを観察できます。その結果、情報をデータに変換したりその逆を行う関数を書くという深刻なニワトリと卵の問題を避けられます。単純な種類の情報の場合、そのようなプログラム部品の設計は些細です。単純な情報以外のものについては、例えばパーズングを知る必要があります、それは直ちにプログラム設計における多くの専門知識を必要とします。

ソフトウェアエンジニアは、BSLとDrRacketがデータ処理を情報→データのパーズングやデータ→情報への変換から分離する方法をmodel-view-controller (MVC) というスローガンで表現します。実際、十分に設計されたソフトウェアシステムはこの分離を強制することが今や受け入れられた知恵となっています。入門書の多くは依然としてそれらを混同していますが、したがって、BSLとDrRacketで作業することで、プログラムの核心の設計に集中でき、そこに十分な経験を積んだ後、情報-データ変換部分の設計を学ぶことができます。

ここでは、データと情報の分離を示すために、2つのプリインストール済みティーチパックを使用します: `2htdp/batch-io` と `2htdp/universe` です。この章から、バッチプログラムと対話型プログラムのための設計レシピを開発し、完全なプログラムがどのように設計されるかのアイデアを提供します。完全なプログラミング言語のティーチパックは完全なプログラムのためのより多くの文脈を提供しており、設計レシピを適切に適応させる必要があることを心に留めておいてください。

図 15: 情報からデータへ、そして戻る

情報とデータの中心的な役割を考えると、プログラム設計はそれらの間のつながりから始めなければなりません。具体的には、私たちプログラマは、選択したプログラミング言語を使って関連する情報の断片をデータとしてどのように表現し、データをどのように解釈して情報とするかを決定しなければなりません。図 15 はこの考えを抽象的な図で説明しています。

この考えを具体的にするために、いくつかの例を見てみましょう。数値の形で情報を消費・生成するプログラムを設計しているとします。表現の選択は簡単ですが、解釈には42のような数値がドメインで何を表すかを説明する必要があります:

- 42 は画像のドメインでは上端からのピクセル数を表すかもしれない
- 42 はシミュレーションやゲームオブジェクトが移動するクロックティックあたりのピクセル数を表すかもしれない
- 42 は物理学のドメインでは華氏、摂氏、または絶対温度目盛りでの温度を意味するかもしれない
- 42 はプログラムのドメインが家具カタログの場合、あるテーブルのサイズを指定するかもしれない
- 42 は単に文字列内の文字数を数えるかもしれない

鍵は、数値を情報として数値をデータとして（そしてその逆へ）どのように移行するかを知ることです。

この知識はプログラムを読むすべての者にとって非常に重要なので、私たちはしばしばそれをコメントの形で書き留め、データ定義と呼びます。データ定義には2つの目的があります。第一に、意味のある単語を使ってデータのあるコレクション（クラス）に名前を付けます。第二に、そのクラスの要素をどのように作成し、任意のデータ片がそのクラスに属するかどうかをどのように判断するかを読者に伝えます。

例：

```
; Temperature は Number
; 摂氏での温度を表す
```

このデータ定義は、任意の数値を温度として解釈できることを述べています。-400 も Temperature です。読者はデータ定義から、-400 が有効な Temperature であることを理解します。

あるデータ定義が与えられたとき、読者は「このクラスに属するデータ片の例を挙げよ」と尋ねられるべきです。Temperature の場合、-273、0、100 は良い例です。読者はまた「この特定のデータはどのクラスに属するか？」と尋ねられるべきです。42 は Number、String、Image、Boolean のいずれでもありません。なぜなら文字列は Temperature に属さないからです。そして、Temperature のサンプルを作れと言われたら、-400 のようなものを思いつくかもしれません。

可能な最低温度が約-273℃であることを知っている場合、この知識をデータ定義で表現できるかどうかを疑問に思うかもしれません。データ定義は実際にはクラスの英語による記述に過ぎないので、確かにここに示したよりもはるかに正確な方法で温度のクラスを定義できます。本書では、このようなデータ定義に様式化された英語を使用し、次の章で「-274 より大きい」などの制約を課すためのスタイルを導入します。

これまで、4つのデータクラスの名前に出会いました：Number、String、Image、Boolean です。この知識があれば、新しいデータ定義を定式化することは、既存のデータ形式に新しい名前を導入することに他なりません。例えば「temperature」を数値に与えることです。この限られた知識でも、私たちの設計プロセスの概要を説明するのに十分です。

設計プロセス

入力情報をデータとして表現する方法と、出力データを情報として解釈する方法を理解したら、個々の関数の設計は簡単なプロセスに従って進められます。

1. 情報をデータとしてどのように表現したいかを表現せよ。1行のコメントで十分です：

； センチメートルを表すために数値を使います。

2. プログラムの成功する設計に関連すると考えるデータクラスに対して、Temperatureのようなデータ定義を定式化せよ。
3. 署名、目的文、関数ヘッダを書け。

関数署名 は、設計の読者にあなたの関数がいくつの入力を消費し、それらがどのクラスから来ているか、何種類のデータを生成するかを伝えるコメントです。以下は3つの例です：

```
； String -> String
； String -> Number
； Number String Image -> Image
```

目的文 は、関数が何をするかを1文で述べるコメントです。良い目的文は、関数を読まずに何をするかを誰でも理解できるようにします。

ヘッダ（スタブとも呼ばれる）は単純化された関数定義です。署名内の各入力クラスに対して1つの変数名を選び、関数の本体は出力クラスからの任意のデータ片にできます。

```
(define (f a-string) 0)
(define (g n) "a")
(define (h num str img) (empty-scene 100 100))
```

パラメータ名は、パラメータが表すデータの種類を反映すべきです。時には、パラメータの目的を示唆する名前を使いたいと思うかもしれません。

目的文を定式化するとき、パラメータ名を使って何が計算されるかを明確にすると役立つことがよくあります。

4. 関数の例を作成せよ。

各関数について、少なくとも2～3個の具体例を挙げ、入力と期待される出力を書きましょう。これを「指の運動」と呼びます。

```
； given: "hello", expect: "h"
； given: "world", expect: "w"
```

5. テンプレートを書け。

データ定義から関数のアウトラインを導き出します。条件分岐が必要な場合は `cond` の骨組みを、構造体が必要なら選択子を使った骨組みを書きます。

6. 関数定義を完成させよ。

テンプレートを埋め、目的文と例を使って実際の計算ロジックを記述します。

7. テストを行え。

例を `check-expect` に変換し、すべてが通ることを確認します。

図 1: 関数設計レシピの基本ステップ（前述の 6 ステップの要約図）

このプロセスは初心者が進捗を確実にし、エラーを早期に発見するのを助けます。

関数の例から設計へ

ここでは、文字列の最初の文字を抽出する簡単な関数 `string-first` を設計する例を示します。

（コード例は原文のまま）

```
; String -> String
; 文字列 s の最初の文字を抽出する
(define (string-first s) "a")

; 例:
; given: "hello", expect: "h"
; given: "world", expect: "w"
; given: " ", expect: " "

(check-expect (string-first "hello") "h")
(check-expect (string-first "world") "w")
```

（以降の章の例も同様に設計レシピを適用して進めます。）

3.2 指の運動：関数

この節の最初のいくつかの練習問題は、前の「関数」節のものとほぼコピーです。ただし、後者で「define」という単語を使っているところを、ここでは「design」という単語を使っています。この違いが意味するのは、設計レシピに従ってこれらの関数を作成し、解答には関連するすべての部品を含めるべきだということです。

節のタイトルが示唆するように、これらはプロセスを身体化するための練習問題です。ステップが第二の天性になるまで、1つも飛ばさないでください。飛ばすと簡単に避けられるエラーにつながります。プログラミングには複雑なエラーの余地がたくさん残されています。ばかばかしいエラーに時間を無駄にする必要はありません。

Exercise 34. 関数 `string-first` を設計せよ。文字列 `s` から最初の文字を抽出する。

Exercise 35. 関数 `string-last` を設計せよ。文字列 `s` から最後の文字を抽出する。

Exercise 36. 関数 `image-area` を設計せよ。画像 `img` の面積を計算する。

Exercise 37. 関数 `string-rest` を設計せよ。文字列 `s` から最初の文字を除いた残りを返す。

Exercise 38. 関数 `string-remove-last` を設計せよ。文字列 `s` から最後の文字を除いた残りを返す。

(本書ではさらに多くの練習問題が続き、すべて設計レシピの適用を練習するものです。)

3.3 ドメイン知識

関数の本体をコード化するのにどのような知識が必要かを不思議に思うのは自然です。少し反省すると、このステップにはプログラムのドメインに対する適切な把握が必要であることがわかります。実際、そのようなドメイン知識には2つの形態があります：

1. 外部ドメインからの知識—数学、音楽、生物学、土木工学、芸術など。プログラマはコンピューティングのすべての応用ドメインを知ることはできないため、ドメイン専門家と問題を議論できるように、さまざまな応用分野の言語を理解する準備ができていなければなりません。数学は多くの（すべてではない）ドメインの交差点にあります。したがって、プログラマはドメイン専門家と問題を処理する際に新しい言語を習得しなければならないことがよくあります。
2. ティーチング言語とそのライブラリに関する知識。プログラマは、与えられた問題に対して言語が何を提供しているかを知る必要があります。

設計プロセスの中で、ドメイン知識は主にステップ5（関数定義を完成させる）で必要になりますが、良い例を作成するためにも役立ちます。

3.4 関数からプログラムへ

すべてのプログラムが単一の関数定義から成るわけではありません。いくつかの関数が必要とするものもあります。多くのものは定数定義も使用します。いずれにせよ、すべての関数を体系的に設計することは常に重要ですが、グローバル定数や補助関数は設計プロセスを少し変えます。

グローバル定数を定義した場合、関数は結果を計算するためにそれらを使用するかもしれません。それらの存在を自分に思い出させるために、テンプレートにこれらの定数を追加するとよいでしょう。結局のところ、それらは関数定義に寄与し得るものの目録に属します。

複数関数のプログラムは、対話型プログラムが自動的にキーイベントやマウスイベントを処理する関数、状態を画像として描画する関数などを必要とするために生じます。各補助関数も設計レシピに従って設計すべきです。

プログラム全体にはトップレベルの目的文も書くべきです。

3.5 テストについて

テストはすぐに労働集約的な作業になります。小さなプログラムを対話領域で実行するのは簡単ですが、多くの機械的な作業と複雑な検査が必要です。プログラマがシステムを成長させるにつれ、多くのテストを実施したいと思うようになります。やがてこの作業は圧倒的になり、プログラマはそれを怠るようになります。同時に、テストは基本的な欠陥を発見・防止するための最初の道具です。ずさんなテストはすぐにバグのある関数—隠れた問題を抱えた関数—につながり、バグのある関数はプロジェクトをしばしば複数の方法で遅らせます。

したがって、手動でテストを行うのではなく、テストを機械化することが重要です。多くのプログラミング言語と同様に、BSL にはテスト機能が含まれており、DrRacket はこの機能を認識しています。

check-expect を使って例を自動テストに変換します：

```
(check-expect (string-first "hello") "h")
```

DrRacket は RUN 時にすべてのテストを実行し、結果を表示します。失敗したテストは赤で表示され、デバッグに役立ちます。

良いテストには以下が含まれます： - 典型的なケース - 境界値（空文字列、長さ 1 の文字列など） - 特殊ケース

テストはまた、プログラムのドキュメントとしても機能します。

3.6 World プログラムの設計

前の章では 2htdp/universe ティーチパックをアドホックな方法で紹介しましたが、この節では設計レシピがどのようにして world プログラムを体系的に作成するのを助けるかを示します。まず、データ定義と関数署名に基づく 2htdp/universe ティーチパックの簡単な要約から始めます。次に、world プログラムのための設計レシピを詳述します。

ティーチパックは、プログラマが世界の状態を表すデータ定義と、すべての可能な世界状態に対して画像を作成する方法を知っている関数 render を開発することを期待します。プログラムの必要性に応じて、プログラマは次にクロックティック、キー入力、マウスイベントに応答する関数を設計しなければなりません。最後に、対話型プログラムは big-bang を使って状態、描画関数、イベントハンドラを結び付ける必要があります。

world プログラム設計レシピの概要：

1. 世界の状態を表すデータ定義を書く。
2. render 関数の署名と目的文を書く：WorldState -> Image
3. 必要に応じて on-tick、on-key、on-mouse などのハンドラの署名を書く。
4. 各関数について設計レシピを適用する。
5. big-bang でプログラムを組み立てる。

例として、シンプルなカウンターやアニメーションを設計できます。

3.7 バーチャルペットの世界

この練習問題の節は、バーチャルペットゲームの最初の2つの要素を紹介します。キャンバスを横切って歩き続ける猫の表示から始まります。もちろん、歩き続けると猫は不幸になり、その不幸が現れます。すべてのペットと同様に、撫でて少し助けるか、餌を与えてたくさん助けることができます。

まず、私たちのお気に入りの猫の画像から始めましょう：

```
(define cat1 <画像>)
```

猫の画像をコピーして DrRacket に貼り付け、define で画像に名前を付けましょう。

Exercise 45. 「バーチャルキャット」の world プログラムを設計せよ。猫を左から右に継続的に動かす。cat-prog と呼び、猫の開始位置を消費すると仮定せよ。さらに、猫をクロックティックあたり3ピクセル動かせ。猫が右端で消えたら左端に再出現させる。modulo 関数について読むとよい。

Exercise 46. 少し異なる画像で猫のアニメーションを改善せよ：

```
(define cat2 <画像>)
```

Exercise 45 の描画関数を調整し、x座標が奇数かどうかで2つの猫画像のどちらかを使うようにせよ。HelpDesk で odd? を調べ、cond 式を使って猫画像を選択せよ。

Exercise 47. 「幸福ゲージ」を維持・表示する world プログラムを設計せよ。gauge-prog と呼び、幸福度は0から100（両端を含む）の数値であると合意せよ。

各クロックティックで幸福レベルは0.1減少する。ただし0を下回らない。下矢印キーが押されるたびに幸福度は1/5減少、上矢印が押されると1/3ジャンプする。

幸福レベルを表示するために、黒い枠のある塗りつぶしの赤い長方形を持つシーンを使え。幸福レベル0では赤いバーは消え、最大幸福レベルHではバーがシーン全体にわたるようにせよ。

Note. 十分な知識を得たら、gauge-prog を Exercise 45 の解と組み合わせる方法を説明する。その後、猫を助けることができるようになる。無視し続けると猫はますます不幸になる。撫でるとより幸せに、餌をやるとずっとずっと幸せになる。だからこそ、これらの最初の3章が教えてくれるよりもずっと多くの world プログラムの設計について知りたいと思う理由がわかるだろう。

注意：この章のすべての Racket コード、関数定義、check-expect、画像リテラル、big-bang 式などは原文のまま保持しています。翻訳は説明文、演習の記述、注記、設計プロセスの解説のみです。

原典: How to Design Programs, Second Edition - Chapter 3 “How to Design Programs” URL: https://htdp.org/2026-5-28/Book/part_one.html ## 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)

現時点では、情報をデータとして表現するための選択肢は、数値、文字列、画像、真偽値の4つしかありません。多くの問題に対してはこれで十分ですが、これら4つの基本データの集合だけでは対応しきれない複雑な問題も多く存在します。優れたソフトウェア設計者には、情報を表現するためのさらなる仕組みが必要です。

少なくとも、プログラムを設計する際には、これらの組み込みデータの集合に対して「制約」を課す方法を学ばねばなりません。制約を課す一つの方法は、データ集合からいくつかの具体的なデータだけをリストアップし、この問題においてはそれらしか使用しないと合意することです。このように要素をリストアップする方法を列挙 (Enumeration) と呼びますが、これは要素数が有限の場合にのみ有効です。一方で、「無限に」多くの要素を含む集合を扱うために、特定の条件を満たす要素の範囲を指定する区間 (Interval) を導入します。

[!NOTE] ここでいう「無限」とは、「要素数が多すぎて、一つひとつリストアップするのが完全に不可能である」という現実的な意味も含まれます。

列挙や区間を定義することは、異なる種類の要素を区別することを意味します。プログラムコード内でこれらを区別するには、条件分岐関数 (Conditional Functions)、すなわち引数の値に応じて異なる計算方法を選択する関数が必要です。「プロローグ：プログラムの仕方」や「ブール値と混ぜ合わせ」では、こうした条件分岐関数の書き方をいくつかの例で示しましたが、いずれも体系的な設計手法 (設計レシピ) は用いていませんでした。本章では、列挙、区間、そしてこれらを組み合わせた新しいデータ表現方法である項目化 (Itemization) の一般的な設計手法を解説します。

まず、cond 式の使い方を詳細に見直し、その後に列挙、区間、項目化の3つのデータ定義方法について説明します。列挙はそのクラスに属するすべてのデータを一つずつリストアップするものであり、区間はデータの範囲を指定するものです。最後の項目化は、定義の一部で範囲を指定し、別の部分で特定のデータを指定するという、両者を混ぜ合わせたものです。最後に、これらのデータ型に対する一般的な設計レシピ (設計戦略) を整理します。

4.1 条件式を使ったプログラミング (Programming with Conditionals)

「プロローグ：プログラムの仕方」で学んだ条件式を思い出してください。cond 式は本書で最も複雑な式の形であるため、その一般的な形式を詳しく見てみましょう。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2])
```

```
...
[ConditionExpressionN ResultExpressionN])
```

cond 式はキーワードである (cond で始まり、対応する) で終わります。キーワードに続いて、必要なだけの cond 行 (これを cond 節 と呼びます) を記述します。各 cond 節は、角括弧 [と] で囲まれた 2 つの式で構成されます (※丸括弧 () を使っても文法上問題ありませんが、視認性を高めるために角括弧を使うのが一般的です)。

条件式を使用した関数定義の例を見てみましょう：

```
(define (next traffic-light-state)
  (cond
    [(string=? "red" traffic-light-state) "green"]
    [(string=? "green" traffic-light-state) "yellow"]
    [(string=? "yellow" traffic-light-state) "red"]]))
```

この例は、cond 式を使用することの便利さを示しています。多くの問題において、関数は複数の異なる状況を区別しなければなりません。cond 式を使用すれば、発生しうる可能性ごとに 1 つの cond 節を割り当てることができ、問題文で示された異なる状況とプログラムコードとの対応関係を読者に対して明確に示すことができます。

if 式 (「ブール値と混ぜ合わせ」を参照) との使い分けについての注意：if 式は「ある 1 つの状況と、それ以外のすべての状況」を区別するために使われます。そのため、3 つ以上の状況を区別する必要がある文脈には適していません。if 式は「A か B か」の二者択一を簡潔に表現する場合にのみ使用すべきです。したがって、データ定義から導き出される複数の明確に区別された状況をコードで表現する場合には、常に cond 式を使用します。

cond 式の条件が複雑になり、「その他のすべての場合」と表現したい場合があります。このような場合、cond 式の最後の節に限って else キーワードを使用することが許可されています。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2]
  ...
  [else DefaultResultExpression])
```

もし最後以外の cond 節で else を使用すると、DrRacket の BSL はエラーを返します：

```
> (cond
  [(> x 0) 10]
  [else 20]
  [(< x 10) 30])
cond:found an else clause that isn't the last clause in its cond expression
```

文法的に正しくないプログラムコードは意味を持たないため、BSL は実行前にこれを拒否します。

ゲームプログラムの一部として、ゲーム終了時のスコアに応じて賞（アワード）を計算する関数 `reward` を設計するとします。関数のヘッダとデータ定義は以下の通りです：

```
; A PositiveNumber is a Number greater than/equal to 0.  
; PositiveNumber -> String  
; computes the reward level from the given score s
```

この関数を実装する2つのバリエーションを比較してみましょう：

左側の実装（3つの明確な条件を使用）：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [(< 20 s) "gold"])))
```

右側の実装（`else` 節を使用）：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [else "gold"])))
```

左側の実装では、3番目の条件として `(< 20 s)` を正しく記述するために、以下の計算が成り立つことをプログラマが頭の中で保証する必要があります：`* s` は非負の数値であること。`* (<= 0 s 10)` が `#false` であること。`* (and (< 10 s) (<= s 20))` もまた `#false` であること。

この例では計算は簡単に見えますが、複雑な条件分岐では境界値の計算ミスによってプログラムにバグ（不具合）が混入しやすくなります。したがって、最後のケースが他の条件の完全な「補集合（それ以外）」であることが明らかな場合は、右側のように `else` 節を使用する方が安全で優れています。

4.2 条件付きの計算 (Computing Conditionally)

DrRacket が `cond` 式をどのように評価するかをより正確に見ていきましょう。先ほどの `reward` 関数の定義を考えます：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [else "gold"])))
```

関数を呼び出す際、`cond` 式単体を見ただけではどの節が実行されるかを予測することはできません。これが関数の目的そのものです。関数は多様な入力（例えば 3, 12, 21 など）を受け取り、それぞれの入力に対して異なる振る舞いをします。入力データのバリエーションを区別して処理することが `cond` 式の役割です。

具体的な評価ステップを見てみましょう。まず、(reward 3) を呼び出した場合です：

```
(reward 3)
==
(cond
  [(<= 0 3 10) "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])
==
(cond
  [#true "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])
==
"bronze"
```

ここでは、最初の条件 ($\leq 0 \ 3 \ 10$) が #true に評価されたため、対応する結果式である "bronze" が cond 式全体の値となります。

次に、(reward 21) を呼び出した場合を見てみます：

```
(reward 21)
==
(cond
  [(<= 0 21 10) "bronze"]
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [#false "bronze"]
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
```

最初の条件が #false と評価されたため、その cond 節全体が評価対象から取り除かれ、次の節へと計算が進みます：

```
(cond
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and #true (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and #true #false) "silver"]
  [else "gold"])
```

```

==
(cond
  [#false "silver"]
  [else "gold"])
==
(cond
  [else "gold"])
==
"gold"

```

このように、1 番目と 2 番目の条件が共に #false であったため、最後の else 節へと処理が移行し、結果として "gold" が得られます。

Exercise 48. DrRacket の定義領域に reward 関数の定義と (reward 18) を入力し、ステップ実行 (Stepper) を使用して評価のプロセスを確認せよ。

Exercise 49. cond 式は式の種類であるため、他の式の中にネスト (入れ子に) して記述することができます。

```
(- 200 (cond [(> y 200) 0] [else y]))
```

y の値が 100 の場合と 210 の場合について、Stepper を使ってこの式がどのように評価されるかを確認せよ。

また、図 20 に示す UFO/ロケットのアニメーションシーン描画関数 create-rocket-scene.v5 をリファクタリングし、cond 式をネストさせることで、place-image の記述が一度だけで済むように実装を書き換えよ。

```

(define WIDTH 100)
(define HEIGHT 60)
(define MTSCN (empty-scene WIDTH HEIGHT))
(define ROCKET <画像>)
(define ROCKET-CENTER-TO-TOP (- HEIGHT (/ (image-height ROCKET) 2)))

; 元の実装
(define (create-rocket-scene.v5 h)
  (cond
    [(<= h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 h MTSCN)]
    [(> h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 ROCKET-CENTER-TO-TOP
      MTSCN)]))

```

4.3 列挙 (Enumerations)

すべての文字列が任意に使えるわけではありません。例えば、マウスイベントをプログラムに通知する文字列は以下の 6 つに制限されています：

```

; A MouseEvent is one of these Strings:
; - "button-down"
; - "button-up"
; - "drag"
; - "move"

```



```
; - "enter"
; - "leave"
```

このように、取りうるすべての具体的なデータを網羅して定義するデータ型を列挙 (Enumeration) と呼びます。日常のプログラム設計において、列挙型は非常によく現れます。例えば、信号機の状態を以下のように定義できます：

```
; A TrafficLight is one of the following Strings:
; - "red"
; - "green"
; - "yellow"
; interpretation: the three strings represent the three possible states of a
traffic light
```

列挙データ型を消費する関数の設計は非常にシンプルです。入力データがケースバイケースで列挙されている場合、関数側でもそれぞれのケースを `cond` 節で区別して処理します。信号機の状態を受け取って、次の信号機の状態を返す関数 `traffic-light-next` は以下のように設計できます：

```
; TrafficLight -> TrafficLight
; yields the next state given current state s
(check-expect (traffic-light-next "red") "green")

(define (traffic-light-next s)
  (cond
    [(string=? "red" s) "green"]
    [(string=? "green" s) "yellow"]
    [(string=? "yellow" s) "red"])))
```

`TrafficLight` の定義が3つのデータ要素で構成されているため、`traffic-light-next` 関数もまた自然に3つの `cond` 節で分岐を行います。

Exercise 50. 上記の `traffic-light-next` 関数の定義を `DrRacket` にコピー&ペーストして `RUN` すると、テストが不十分なため、いくつかの `cond` 節がハイライト表示 (テスト未通過) されます。すべての節がカバーされるようにテストケースを追加せよ。

Exercise 51. 信号機のアニメーションをシミュレートする `big-bang` プログラムを設計せよ。信号機の状態を適切な色の円で描画し、クロックティックごとに状態 (色) が遷移するようにせよ。最も適切な初期状態について検討せよ。

列挙型の本質は、データ集合を「有限個の具体的なデータ片」として定義することです。もう一つの重要な例を紹介します：

```
; A 1String is a String of length 1, including:
; - "\" (the backslash),
; - " " (the space bar),
; - "  " (tab),
; - "" (return), and
```

```
; - "␣" (backspace).
; interpretation: represents keys on the keyboard
```

BSL では、キーボードの入力イベント (KeyEvent) を以下のように表現します：

```
; A KeyEvent is one of:
; - lString
; - "left"
; - "right"
; - "up"
; - "down"
; - ...
```

この定義に従うと、キーイベントを処理する関数 (キーイベントハンドラ) の設計テンプレートは自然と以下ようになります：

```
; WorldState KeyEvent -> WorldState
(define (handle-key-events w ke)
  (cond
    [(= (string-length ke) 1) ...]
    [(string=? "left" ke) ...]
    [(string=? "right" ke) ...]
    [(string=? "up" ke) ...]
    [(string=? "down" ke) ...]
    [else ...]))
```

Exercise 52. キーボードの左右の矢印キーが押されたときに、赤いドットを左右に移動させるキーイベントハンドラ `keh` を設計せよ。不要なキーイベントは `else` 節で無視すること。図 21 に示す以下のコードを参考にせよ。

```
; A Position is a Number.
; interpretation: distance between the left margin and the ball
; Position KeyEvent -> Position
; computes the next location of the ball
```

```
(check-expect (keh 13 "left") 8)
(check-expect (keh 13 "right") 18)
(check-expect (keh 13 "a") 13)
```

```
(define (keh p k)
  (cond
    [(string=? "left" k) (- p 5)]
    [(string=? "right" k) (+ p 5)]
    [else p]))
```

4.4 区間 (Intervals)

UFO が降下する様子を描画するプログラムを考えます (図 22 を参照)。

```
; A WorldState is a Number.
; interpretation: number of pixels between the top and the UFO
(define WIDTH 300)
```

```

(define HEIGHT 100)
(define CLOSE (/ HEIGHT 3))
(define MTSCN (empty-scene WIDTH HEIGHT))
(define UFO (overlay (circle 10 "solid" "green") (rectangle 30 4 "solid"
"green")))

; WorldState -> WorldState
(define (main y0)
  (big-bang y0
    [on-tick nxt]
    [to-draw render]))

; WorldState -> WorldState
(define (nxt y) (+ y 3))

; WorldState -> Image
(define (render y)
  (place-image UFO (/ WIDTH 2) y MTSCN))

```

このUFOシミュレーションにおいて、UFOの高さ（位置）に応じて以下のようなステータス画面を表示させたいとします：* UFOの高さが画面上部の3分の1（CLOSE）より上にあるときは "descending"（降下中）。* それを超えて下部（HEIGHT）までの間にあるときは "closing in"（接近中）。* 画面の底（HEIGHT）に達したときは "landed"（着陸）。

この場合、状態を表す数値は無限に存在するため、単純に個々の値を列挙することは不可能です。代わりに、境界値を用いて数値の範囲を切り分ける区間（Intervals）を使用します。

区間は、境界値（左端と右端）によって定義されます。境界値自体が範囲に含まれる場合を「閉じた（closed）」、含まれない場合を「開いた（open）」と呼びます。数学の表記に倣い、閉じた境界にはブラケット [や]、開いた境界にはペアレントシス (や) を用いて表現します：* [3, 5]：閉区間（3以上5以下）* (3, 5]：左開・右閉区間（3より大きく5以下）* [3, 5)：左閉・右開区間（3以上5未満）* (3, 5)：开区間（3より大きく5未満）

これを用いると、世界の状態 WorldState は以下の3つの区間に分類できます：1. 0 から CLOSE の間（降下中）2. CLOSE から HEIGHT の間（接近中）3. HEIGHT 以上（着陸）

これをプログラムで表現する場合、境界値がどの区間に属するかという「重複」を避ける必要があります。BSLの cond 条件式は順番に評価されるため、この性質を利用して以下のように記述できます：

```

; WorldState -> Image
(define (render/status y)
  (cond
    [(<= 0 y CLOSE)
     (place-image (text "descending" 11 "green") 10 10 (render y))]
    [(and (< CLOSE y) (<= y HEIGHT))

```

```

    (place-image (text "closing in" 11 "orange") 10 10 (render y)))
  [(> y HEIGHT)
   (place-image (text "landed" 11 "red") 10 10 (render y))])

```

UF0 のアニメーションプログラムをこのステータス表示付きのバージョンにするには、main 関数で [to-draw render/status] を呼び出すように変更します。

UF0 の位置情報や表示位置を一箇所で管理するために、UF0 の座標やテキスト表示位置の「コピー」を避ける実装を設計するのが賢明です。図 24 のように、cond 式自体を place-image の引数として入れ子にすることができます：

```

(define (render/status y)
  (place-image
   (cond
    [(<= 0 y CLOSE) (text "descending" 11 "green")]
    [(and (< CLOSE y) (<= y HEIGHT)) (text "closing in" 11 "orange")]
    [else (text "landed" 11 "red")])
   20 20
   (render y)))

```

4.5 項目化 (Itemizations)

列挙型は具体的な有限のデータ片をリストアップし、区間は無限に存在する数値の範囲を指定します。これら両方を混在させて定義するデータ型を項目化 (Itemizations) と呼びます。

例えば、string->number 関数の出力結果を考えてみましょう。この関数は、文字列が数値に変換可能な場合は数値を返し、不可能な場合は #false を返します。この結果を表すデータ型 NorF は以下のように定義できます：

```

; An NorF is one of:
; - #false
; - a Number

```

これは、具体的なデータ片である #false と、無限のデータクラスである Number を組み合わせた項目化の典型例です。この NorF を消費する関数の設計は、やはりそれぞれのケース (項目) に対応する cond 節を持つ形になります：

```

; NorF -> Number
; adds 3 to the given number; 3 otherwise
(check-expect (add3 #false) 3)
(check-expect (add3 0.12) 3.12)

(define (add3 x)
  (cond
   [(false? x) 3]
   [else (+ x 3)]))

```

もう少し実用的な例として、ロケットの打ち上げカウントダウンをシミュレートするプログラムを設計します。

```
; An LRCD (for launching rocket countdown) is one of:
; - "resting"
; - a Number between -3 and -1
; - a NonnegativeNumber
; interpretation:
;   "resting" represents a grounded rocket.
;   a number in [-3, -1] denotes the countdown phase.
;   a nonnegative number denotes the height of the rocket in flight.
```

このデータ型に対するシミュレーションプログラムを以下のように構成します： * 定数定義：

```
(define HEIGHT 300)
(define WIDTH 100)
(define YDELTA 3)
(define BACKG (empty-scene WIDTH HEIGHT))
(define ROCKET (rectangle 5 30 "solid" "red"))
(define CENTER (/ (image-height ROCKET) 2))
```

• 描画関数 **show**:

```
; LRCD -> Image
(define (show x)
  (cond
    [(string? x)
     (place-image ROCKET 10 (- HEIGHT CENTER) BACKG)]
    [(<= -3 x -1)
     (place-image (text (number->string x) 20 "red")
                   10 (* 3/4 WIDTH)
                   (place-image ROCKET 10 (- HEIGHT CENTER) BACKG))]
    [(>= x 0)
     (place-image ROCKET 10 (- x CENTER) BACKG))])
```

• キーイベントハンドラ **launch**:

```
; LRCD KeyEvent -> LRCD
(define (launch x ke)
  (cond
    [(string? x) (if (string=? " " ke) -3 x)]
    [(<= -3 x -1) x]
    [(>= x 0) x]))
```

• 時間経過ハンドラ **fly** (カウントダウンおよび飛行のシミュレーション) :

```
; LRCD -> LRCD
(define (fly x)
  (cond
    [(string? x) x]
    [(<= -3 x -1) (if (= x -1) HEIGHT (+ x 1))]
    [(>= x 0) (- x YDELTA)]))
```

• メインプログラム実行:

```
(define (main2 s)
  (big-bang s
    [to-draw show]
    [on-key launch]
    [on-tick fly 1])) ; クロックティックは1秒ごと
```

Exercise 53～57. 以上のカウントダウン付きロケットプログラムを実際に DrRacket 上で構築・実行し、終了処理 (stop-when) の追加などの拡張を試みよ。

4.6 項目化を用いた設計 (Designing with Itemizations)

これまでに学んだ内容を基に、「From Functions to Programs」で紹介した設計レシピを拡張し、列挙、区間、項目化を扱う関数を体系的に設計するためのステップを定義します。

以下の消費税計算を例に、ステップを確認します： > 安価な商品 (1,000 ドル未満) には税金がかかりません。 > 高級品 (10,000 ドル超) には 8% の税率が適用されます。 > それらの中間の商品には 5% の税金がかかります。

1. データ定義 (Data Definition) 問題文で区別されている状況をカバーする、重複のない明確なデータ定義を記述します。

```
; A Price falls into one of three intervals:
; - 0 through 1000 (exclusive)
; - 1000 through 10000 (exclusive)
; - 10000 and above.
```

2. 署名、目的文、ヘッダ (Signature, Purpose, Header)

```
; Price -> Number
; computes the amount of tax charged for p
(define (sales-tax p) 0)
```

3. 具体的な例 (Examples) それぞれの区間、および境界値からテストデータを抽出します (境界値の扱いについて専門家や仕様書に確認することが重要です)。

```
(check-expect (sales-tax 537) 0)
(check-expect (sales-tax 1000) (* 0.05 1000))
(check-expect (sales-tax 12017) (* 0.08 12017))
```

4. テンプレート (Template) データ定義の項目 (サブクラス) の数と同じ数の cond 節を持つ条件分岐のアウトラインを作成します。

```
(define (sales-tax p)
  (cond
    [(and (<= 0 p) (< p 1000)) ...]
    [(and (<= 1000 p) (< p 10000)) ...]
    [(>= p 10000) ...]))
```

5. 定義の完成 (Definition) 各節の空欄 ... を、パラメータを用いた具体的な計算式で埋めます。

```
(define (sales-tax p)
  (cond
    [(and (<= 0 p) (< p 1000)) 0]
    [(and (<= 1000 p) (< p 10000)) (* 0.05 p)]
    [(>= p 10000) (* 0.08 p)]))
```

6. テスト (Testing) テストを実行し、すべての `cond` 節がカバーされ、期待通りの値が返ることを確認します。

4.7 有限状態世界 (Finite State Worlds)

信号機のように、「有限個の状態」と「状態間の遷移ルール」によって定義されるシステムを有限状態機械 (Finite State Machine: FSM) または有限状態オートマトン (Finite State Automaton: FSA) と呼びます。

信号機のシミュレーションプログラムは、以下の要素で構成できます： * 状態の定義: "red", "green", "yellow" * 時間経過による遷移関数 `tl-next`: racket ; TrafficLight -> TrafficLight (define (tl-next cs) (cond [(string=? "red" cs) "green"] [(string=? "green" cs) "yellow"] [(string=? "yellow" cs) "red"]))) * 描画関数 `tl-render`: 状態に応じて適切な色のライトを表示する画像を生成する。

Exercise 59~61. 信号機の有限状態シミュレーションプログラムを完成させよ。

自動ドアクローザーのシミュレーション

自動ドアは以下の状態と遷移ルールを持ちます： * 状態 (`DoorState`): * LOCKED (施錠中) * CLOSED (閉扉中・未施錠) * OPEN (開扉中) * イベントルール: * LOCKED 状態で "u" キーを押すと CLOSED になる (解錠)。 * CLOSED 状態で "l" キーを押すと LOCKED になる (施錠)。 * CLOSED 状態でスペースキー " " を押すと OPEN になる (開扉)。 * OPEN 状態からは、クロックティック (時間経過) により自動的に CLOSED に戻る。

このシミュレーションプログラムは以下のように設計できます：

```
(define LOCKED "locked")
(define CLOSED "closed")
(define OPEN "open")

; DoorState -> DoorState
; closes an open door over the period of one tick
(define (door-closer state-of-door)
  (cond
    [(string=? LOCKED state-of-door) LOCKED]
    [(string=? CLOSED state-of-door) CLOSED]
    [(string=? OPEN state-of-door) CLOSED]))

; DoorState KeyEvent -> DoorState
; turns key event k into an action on state s
(define (door-action s k)
```

```

(cond
  [(and (string=? LOCKED s) (string=? "u" k)) CLOSED]
  [(and (string=? CLOSED s) (string=? "l" k)) LOCKED]
  [(and (string=? CLOSED s) (string=? " " k)) OPEN]
  [else s]))

; DoorState -> Image
; translates the state s into a large text image
(define (door-render s)
  (text s 40 "red"))

; DoorState -> DoorState
(define (door-simulation initial-state)
  (big-bang initial-state
    [on-tick door-closer 3] ; 3秒ごとにドアが閉まる
    [on-key door-action]
    [to-draw door-render]))

```

Exercise 62. ドアシミュレータを DrRacket で実行し、動作を確認せよ。

5 構造の追加 (Adding Structure)

(今後の翻訳作業のためのプレースホルダー)

6 項目化と構造 (Itemizations and Structures)

(今後の翻訳作業のためのプレースホルダー)

7 まとめ (Summary)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 1: Beginning Student Language

第 I 部「固定サイズのデータ」は、BSL を自然言語であるかのように扱いました。言語の「基本的な単語」を紹介し、「単語」を「文」に組み立てる方法を示唆し、これらの「文」の直感的な理解のために代数学の知識に訴えかけました。このような導入はある程度機能しますが、真に効果的なコミュニケーションには、ある程度の形式的な学習が必要です。

多くの点で、第 I 部のアナロジーは正しいものです。プログラミング言語には語彙と文法があり、プログラマはこれらの要素を構文 (syntax) と呼びます。BSL における「文」は、式 (expression) または定義 (definition) です。BSL の文法は、これらの句をどのように形成するかを規定します。しかし、文法的に正しい文であっても、すべてが意味を持つわけではありません—英語でもプログラミング言語でも同じです。例えば、「the cat is round」という英語の文は意味を持ちますが、「the brick is a car」は文法的には完全に正しくても意味がありません。文が意味を持つかどうかを判断するには、言語の意味 (meaning) を知る必要があります。プログラマはこれを意味論 (semantics) と呼びます。

この Intermezzo では、BSL を中学校で馴染みのある算数・代数の言語の拡張であるかのように提示します。結局のところ、計算はこの単純な数学の形式から始まるのですから、この数学とコンピューティングのつながりを理解すべきです。最初の3つの節では、BSL のかなりの部分の構文と意味論を提示します。この新しいBSL の理解に基づき、4つ目の節ではエラーの議論を再開します。残りの節はこの理解を完全な言語に拡張し、最後の節ではテストを表現するためのツールを拡張します。

BSL の語彙 (BSL Vocabulary)

図 39 は、BSL の基本的な語彙を紹介し定義しています。語彙は以下のものから構成されます：

- リテラル定数 (数値や真偽値など)
- BSL によって意味が与えられている名前 (例: `cond` や `+`)
- プログラムが `define` や関数のパラメータを通じて意味を与えることができる名前

名前 (name) または 変数 (variable) とは、空白や以下の文字を含まない文字の並びです： `" , ' () [] { } | ; #``

- プリミティブ (primitive) とは、BSL が意味を割り当てる名前です。例: `+` や `sqrt`。
- 変数 (variable) とは、事前に意味が割り当てられていない名前です。

値 (value) は以下のいずれかです：

- 数 (number): `1`, `-1`, `3/5`, `1.22`, `#i1.22`, `0+1i` など (正確数・不正確数、実数・複素数などを含む)
- ブール値 (Boolean): `#true` または `#false`
- 文字列 (String): ダブルクォートで囲まれた文字の並び (例: `"hello"`)
- 画像 (Image): `circle`, `rectangle` などで生成される画像

(完全な定義は図 39 を参照。直感的に理解してください。各集合には追加の要素が含まれる場合があります。)

BSL の文法 (BSL Grammar)

図 40 は、BSL 文法の大部分を示しています。他の言語と比べて非常に単純です。表現力については見た目に惑わされないでください。

文法の読み方： - 各行の `=` は「～のいずれかである」を意味します。 - `|` は「または」を意味します。 - `...` は先行するものの繰り返しを意味します。

主要な構文カテゴリ (簡略)：

```
program = def-expr ...
def      = (define name expr)
          | (define (name name ...) expr)
expr     = name
```

```
| value
| (name expr ...)
| (cond [expr expr] ... [else expr])
| (and expr ...)
| (or expr ...)
```

define、cond、else はキーワードです。これらは名前ではありますが、特別な役割を持つためキーワードと呼ばれます。

BSL の意味 (BSL Meaning)

DrRacket で式を評価すると、算数と代数の法則を使って値を求めます。

これまでの BSL の変種では、値は式の部分集合です。値には Boolean、String、Image、数値などが含まれます。

意味の法則の例 (算術から拡張) :

```
(+ 1 2)
==
3
(+ 1 (+ 1 1) 2)
==
(+ 1 2 2)
==
5
```

条件式の意味:

```
(cond
  [(zero? x) 0]
  [else (+ x 1)])
```

最初の条件が真なら対応する式を評価、そうでなければ次の条件へ。else は常に真として扱われます。

意味とコンピューティング (Meaning and Computing)

BSL の意味は、DrRacket がどのように式をステップごとに簡約していくかを正確に定義します。このプロセスは、算術の計算と本質的に同じです。

BSL のエラー (BSL Errors)

DrRacket が括弧で囲まれた句が BSL に属しないと判断すると、構文エラー (syntax error) を報告します。

文法的に正しくても意味を持たないプログラムは実行時にエラーになります (例: 0 での除算、存在しない名前の使用)。

ブール式 (Boolean Expressions)

BSL には and と or があります。

```
(and (string=? "a" "a") (> 10 9))
==
#true

(or (string=? "a" "b") (= 1 2))
==
#false
```

短絡評価が行われます（必要に応じて後続の式を評価しない）。

定数定義 (Constant Definitions)

```
(define x 3)
(define y (+ x 1))
(define name "Matthias")
```

定数は式の中で名前として使用できます。

構造型定義 (Structure Type Definitions)

```
(define-struct posn [x y])
```

これにより `make-posn`、`posn-x`、`posn-y`、`posn?` が自動的に定義されます。

BSL のテスト (BSL Tests)

```
(check-expect (f 1) 2)
(check-expect (f 2) 4)

(check-within (g 1) 3.14 0.01)
(check-error (h 0) "division by zero")
```

テストはプログラムの一部として書くことができます。

BSL のエラーメッセージ (BSL Error Messages)

DrRacket は親切なエラーメッセージを出します。主要なカテゴリ：

- ・ 構文エラー（括弧の不一致、キーワードの誤用）
- ・ 実行時エラー（未定義の名前、型エラー、0 除算など）
- ・ テスト失敗

各エラーメッセージは問題の場所と可能な原因を示唆します。

（詳細なエラーメッセージ一覧は本書の該当節を参照してください。）

注意：この *Intermezzo* の目的は、BSL を「ただ動くもの」としてではなく、構文・意味論を持つ正式な言語として理解することです。これにより後の章でより複雑なプログラムを設計する基盤が得られます。

8 リスト (Lists)

これまで自己参照的な定義に出会ったことはおそらくないでしょう。国語 (英語) の教師は間違いなくこのような定義を避けますし、多くの数学の授業でもこうした定義は曖昧に扱われます。しかし、プログラマは曖昧さを許容できません。プログラマの仕事には正確さが必要です。一般に、定義には自分自身への参照が複数含まれることがあります。この章ではリストを皮切りに、自己参照をちょうど1つだけ必要とする有用な例を紹介します。

リストを導入することで、私たちが学ぶことのできるアプリケーションの幅も広がります。この章では例を通じて直感を丁寧に築き上げるとともに、自己参照データを体系的に処理する関数を作成する方法を説明する次の章での設計レシピの改訂へと動機づけます。

8.1 リストの作成 (Creating Lists)

私たちは皆、日常的にリストを作成しています。食料品の買い物に行く前には、購入したい品目のリストを書き留めます。毎朝 ToDo リストを書く人もいます。12 月には、多くの子供たちがクリスマスの欲しいもののリストを用意します。パーティーを計画するときは、招待者のリストを作ります。情報をリストの形で整理することは、私たちの生活のいたるところにあります。

情報がリストの形で与えられるなら、そのようなリストを BSL (Beginning Student Language) のデータとして表現する方法を明確に学ぶ必要があります。実際、リストは非常に重要であるため、BSL にはデカルト座標の点 (posn) と同様に、リストの作成 and 操作のための組み込みサポートが備わっています。点とは対照的に、リスト of データ定義は常に自分で書く必要があります。しかし、まずは順を追って進めましょう。リストの作成から始めます。

リストを形成するとき、私たちは常に空のリスト (empty list) から始めます。BSL では、空のリストを以下のように表現します。

```
'()
```

これは「エンプティ (empty)」と発音され、「空リスト (empty list)」の略です。#true や 5 と同様に、'() は単なる定数です。リストに何かを追加するときは、別のリストを構築します。BSL では、cons 操作がこの目的を果たします。例えば、

```
(cons "Mercury" '())
```

は、'() リストと文字列 "Mercury" からリストを構築します。図 44 は、このリストを構造体と同じように視覚的に表現したものです。cons の箱には、first と rest という 2 つのフィールドがあります。この特定の例では、first フィールドに "Mercury" が入り、rest フィールドに '() が入っています。

Figure 44: リストの構築

- list: (cons "Mercury" '())
 - diagram: [first: "Mercury" | rest: '()]
- list: (cons "Venus" (cons "Mercury" '()))
 - diagram: [first: "Venus" | rest: ●] ---> [first: "Mercury" | rest: '()]
- list: (cons "Earth" (cons "Venus" (cons "Mercury" '())))
 - diagram: [first: "Earth" | rest: ●] ---> [first: "Venus" | rest: ●] ---> [first: "Mercury" | rest: '()]

1つの要素を持つリストができたら、再び cons を使って2つの要素を持つリストを構築できます。以下はその一例です。

```
(cons "Venus" (cons "Mercury" '()))
```

また、別の例を示します。

```
(cons "Earth" (cons "Mercury" '()))
```

図44の中央の行は、2つの要素を含むリストをどのようにイメージできるかを示しています。これも2つのフィールドを持つ箱ですが、今回の rest フィールドには別の箱が入っています。実際、同じ図の上の行にある箱が入っています。

最後に、3つの要素を持つリストを構築します。

```
(cons "Earth" (cons "Venus" (cons "Mercury" '())))
```

図44の最後の行は、3つの要素を持つリストを表しています。その rest フィールドには、さらに箱を含む箱が入っています。このように、リストを作成する際には、箱の中に箱を入れ、その中にさらに箱を入れるということを繰り返します。一見すると奇妙に思えるかもしれませんが、これは誕生日に贈られる入れ子のおもちゃ（マトリョーシカ）や、重ねられたコップのようなものです。唯一の違いは、BSLのプログラムは、職人が物理的な箱を重ねるよりもはるかに深くリストを入れ子にできるという点です。

優れた画家であっても、深く入れ子になった構造を描くのは難しいため、コンピュータ科学者は代わりに「箱と矢印 (box-and-arrow)」の図を使用します。図45は、図44の最後の行を再配置する方法を示しています。各 cons 構造体は別々の箱になります。もし rest フィールドが複雑すぎて箱の中に描けない場合は、代わりに黒丸 (●) を描き、そこからそれが含む箱へと矢印付きの線を引きます。箱の配置方法によって、2種類の図が得られます。

図45の上の行に示されている最初の配置は、箱が作成された順に並べたものです。下の行に示されている2番目の配置は、それらが cons された順に並べたものです。したがって、2番目の図を見れば、リストの長さにかかわらず、そのリストに first を適用したときに何が生成されるかがすぐに分かります。このため、プログラマは2番目の配置を好みます。

Figure 45: リストの描画（作成された順、または cons で結合された順に箱を再配置したダイアグラム）

Exercise 129. 以下のデータを表す BSL リストを作成せよ。1. 天体のリスト（少なくとも太陽系のすべての惑星を含むもの）。2. 食事の品目のリスト（例えば、ステーキ、フライドポテト、インゲン豆、パン、水、ブリー・チーズ、アイスクリーム）。3. 色のリスト。

これらのリストについて、図 44 や図 45 と同様の箱表現をスケッチせよ。どちらのスケッチが気に入ったか？

数値のリストを作成することもできます。以下は 10 個の数字（0 から 9 までの桁）を含むリストです。

```
(cons 0
  (cons 1
    (cons 2
      (cons 3
        (cons 4
          (cons 5
            (cons 6
              (cons 7
                (cons 8
                  (cons 9 '()))))))))))))
```

このリストを構築するには、10 回のリスト構築（cons）と 1 つの '()' が必要です。例えば、3 つの任意の数値からなるリスト、

```
(cons pi
  (cons e
    (cons -2.3 '()))))
```

を構築するには、3 つの cons が必要です。

一般に、リストには 1 種類だけの値を含める必要はなく、任意の値を混在させて含めることができます。

```
(cons "Robbie Round"
  (cons 3
    (cons #true '()))))
```

（※注意：もしこのリストが固定個の要素からなるレコードを表すつもりなら、代わりに構造体型を使用すべきです。）

このリストの最初の要素は文字列、2 番目は数値、最後はブール値です。このリストは、従業員の氏名、社歴（年数）、会社を通じて健康保険に加入しているかどうかという 3 つのデータからなる人事レコードの表現と考えることができます。あるいは、ゲームの仮想プレイヤーを表しているとも考えることもできます。データ定義がなければ、そのデータが何についてのものであるかを知ることができません。

以下は、cons を含む最初のデータ定義です。

```
;; A 3LON is a list of three numbers:  
;; (cons Number (cons Number (cons Number '())))  
;; interpretation: a point in 3-dimensional space
```

もちろん、このデータ定義は、他者が構造体のインスタンスにコンストラクタを使用するのと同じように cons を使用しており、ある意味で cons は特別なコンストラクタにすぎません。このデータ定義が示せていないのは、任意の長さのリスト（要素が何もないリスト、1つのリスト、2つのリスト、10のリスト、あるいはおそらく 1,438,901 個の要素を持つリスト）をどのように形成するかということです。

そこで、もう一度試してみましょう。

```
;; A List-of-names is one of:  
;; -- '()  
;; -- (cons String List-of-names)  
;; interpretation: a list of invitees, by last name
```

深呼吸をして、もう一度読んでください。このデータ定義は、これまでに遭遇した中で最も珍しい定義の1つです。なぜなら、自分自身を参照する定義を見たことがないからです。これが意味をなすかどうかは明らかではありません。結局のところ、国語の先生に『机』とは『机』のことである」と説明しても、「デタラメだ!」と言われるだけでしょう。自己参照的な定義は単語の意味を説明しないからです。

しかし、コンピュータ科学やプログラミングにおいては、自己参照的な定義が中心的な役割を果たしており、注意を払えば実際に意味をなします。ここで「意味をなす」とは、データ定義を本来の目的、すなわち定義されているクラスに属するデータの例を生成したり、与えられたデータが定義されたクラスに属しているかどうかを確認したりするために使用できることを意味します。この観点から見ると、List-of-names の定義は完全に意味をなしています。

最低限、項目の最初の節 ('()) を使って、1つの例として '() を生成できます。'() が List-of-names の要素であることが分かれば、2番目の例を作るのも簡単です。

```
(cons "Findler" '())
```

ここでは、文字列 (String) と List-of-names の唯一の既知のリスト ('()) を使用して、項目化の2番目の節に従ってデータを生成しています。同じルールで、この種のリストをさらに多く生成できます。

```
(cons "Flatt" '())  
(cons "Felleisen" '())  
(cons "Krishnamurthi" '())
```

これらのリストにはそれぞれ1つの名前（文字列として表現されたもの）しか含まれていませんが、データ定義の2行目を使用して、より多くの名前を含むリストを作成することもできます。


```
(cons "Felleisen" (cons "Findler" '()))
```

このデータは List-of-names に属します。なぜなら、"Felleisen" は文字列であり、(cons "Findler" '()) はすでに確認された List-of-names の要素だからです。

Exercise 130. 5つの文字列を含む List-of-names の要素を作成せよ。図 44 の図と同様のリストの箱表現をスケッチせよ。

また、なぜ以下が List-of-names の要素であり、

```
(cons "1" (cons "2" '()))
```

なぜ以下がそうではないのかを説明せよ。

```
(cons 2 '())
```

Exercise 131. Boolean 値のリストを表現するためのデータ定義を提供せよ。このクラスは、任意の長さの Boolean のリストすべてを含みます。

8.2 ‘() とは何か、cons とは何か (What Is ‘(), What Is cons)

少し立ち止まって、'() と cons を詳しく見てみましょう。前述したように、'() は単なる定数です。5 や "this is a string" などの定数と比較すると、これは関数名や変数のように見えます。しかし、#true や #false と比較すれば、これが単に BSL における空のリストの表現であることが簡単にわかるはずです。

評価ルールの観点から見ると、'() は数値やブール値、文字列などとは異なる、新しい種類のアトミックな（原子的な）値です。また、posn のような複合値（構造体インスタンス）ではありません。実際、'() は非常にユニークであり、それ自体で独自の値を形成します。そのため、この値のクラスには、'() だけを認識し、他には何も認識しない述語が用意されています。

```
;; Any -> Boolean  
;; 与えられた値が '() であるかを判定する  
(define (empty? x) ...)
```

すべての述語と同様に、empty? は BSL のあらゆる値に適用できます。これが #true を生成するのは、'() に適用されたときだけです。

```
> (empty? '())  
#true  
> (empty? 5)  
#false  
> (empty? "hello world")  
#false  
> (empty? (cons 1 '()))  
#false  
> (empty? (make-posn 0 0))  
#false
```


次に、cons について考えます。これまで見てきたことはすべて、cons が構造体型定義によって導入されるようなコンストラクタにすぎないことを示唆しています。より正確には、cons は2つのフィールドを持つ構造体のコンストラクタであるように見えます。最初のフィールドは任意の値、2番目のフィールドはリストに類する値が入ります。以下の定義は、このアイデアを BSL で表現したものです。

```
(define-struct pair [left right])
;; A ConsPair is a structure:
;;   (make-pair Any Any).

;; Any Any -> ConsPair
(define (our-cons a-value a-list)
  (make-pair a-value a-list))
```

唯一のキャッチ（注意点）は、our-cons が2番目の引数としてすべての可能な BSL 値を受け入れるのに対し、cons はそうではないという点です。以下の実験がこれを検証しています。

```
> (cons 1 2)
cons: second argument must be a list, but received 1 and 2
```

言い換えれば、cons は実際には「チェックされた関数」（第Ⅰ部「項目化と構造体」で説明したもの）であり、以下のような洗練されたデータ定義を示唆しています。

```
;; A ConsOrEmpty is one of:
;; -- '()
;; -- (make-pair Any ConsOrEmpty)
;; interpretation: ConsOrEmpty is the class of all lists

;; Any Any -> ConsOrEmpty
(define (our-cons a-value a-list)
  (cond
    [(empty? a-list) (make-pair a-value a-list)]
    [(pair? a-list) (make-pair a-value a-list)]
    [else (error "cons: second argument ...")]))
```

もし cons がチェックされたコンストラクタ関数であるなら、結果の構造体からどのようにして要素を取り出すのか疑問に思われるかもしれません。結局のところ、第Ⅰ部「構造体の追加」では、構造体を使ったプログラミングにはセレクトが必要であると説明しました。cons 構造体には2つのフィールドがあるため、2つのセレクトがあります。それは first と rest です。これらも、pair 構造体を使って簡単に定義できます。

```
;; ConsOrEmpty -> Any
;; 与えられたペアの left 部分を抽出する
(define (our-first a-list)
  (if (empty? a-list)
      (error 'our-first "...")
      (pair-left a-list)))
```

立ち止まって、`our-rest` を定義してみてください。

もしあなたのプログラムが `pair` の構造体型定義に直接アクセスできると、`right` フィールドに `'()` や別の `pair` を含まないような `pair` のインスタンスを簡単に作成できてしまいます。このような不適切なインスタンスが意図的または偶発的に作成されると、関数やプログラムが奇妙な方法で破損する原因となります。そのため、BSL ではこれらの問題を回避するために、`cons` の実際の構造体型定義を隠蔽しています。第 III 部「局所定義」では、プログラムでこのような定義を隠蔽する方法の 1 つを示しますが、現時点ではその必要はありません。

Figure 46: リストのプリミティブ

- `'()` : 特別な値。主に空リストを表現するために使用される。
- `empty?` : `'()` だけを認識する述語。
- `cons` : 2つのフィールドを持つインスタンスを作成する、チェックされたコンストラクタ。
- `first` : 最後に追加された項目を抽出するセクタ。
- `rest` : 拡張された（残りの）リストを抽出するセクタ。
- `cons?` : `cons` のインスタンスを認識する述語。

図 46 はこのセクションをまとめたものです。重要な洞察は、`'()` がユニークな値であり、`cons` はリストの値を生成するチェックされたコンストラクタであるということです。さらに、`first`、`rest`、`cons?` は、通常の述語やセクタに対する単なる別の名前にすぎません。したがって、この章が教えているのは、データを表現する新しい方法ではなく、「データ定義を定式化する新しい方法」なのです。

8.3 リストを使ったプログラミング (Programming with Lists)

例えば、あなたが友人たちのリストを管理していて、そのリストがあまりにも大きくなったため、ある特定の名前がそのリストに含まれているかどうかを判定するプログラムが必要になったとしましょう。（※注意：ここでは「友人 (friend)」という言葉は、現実世界ではなくソーシャルネットワークの意味で使用しています。） このアイデアを具体化するために、以下の練習問題を考えてみましょう。

設計課題 (Sample Problem) あなたは新しい携帯電話の連絡先リストの開発に取り組んでいます。携帯電話の所有者は、さまざまな機会にこのリストを更新し、参照します。差し当たり、あなたはこの連絡先リストを受け取り、その中に "Flatt" という名前が含まれているかどうかを判定する関数を設計するタスクを割り当てられました。

この課題の解決策が得られたら、リスト内の任意の名前にマッチする一般的な関数へと一般化します。

先ほど定義した List-of-names のデータ定義は、関数が検索する名前のリストを表現するのに適しています。次に、ヘッダー（シグネチャ、目的、スタブ）を作成します。

```
;; List-of-names -> Boolean
;; リストに "Flatt" が含まれているかを判定する
(define (contains-flatt? alon)
  #false)
```

関数が受け取る名前のリストを表す引数名として、a-list-of-names は良い名前ですが、少し長くて煩雑なため、短く alon と略します。

一般的な設計レシピに従い、次に関数の目的を説明する例を作成します。まず、最も単純な入力に対する出力を決定します。それは '() です。このリストには文字列が一つも含まれていないため、確実に "Flatt" も含まれていません。

```
(check-expect (contains-flatt? '()) #false)
```

次に、要素が1つだけのリストを考えます。以下は2つの例です。

```
(check-expect (contains-flatt? (cons "Findler" '())) #false)
(check-expect (contains-flatt? (cons "Flatt" '())) #true)
```

最初のケースでは、リストの唯一の項目が "Flatt" ではないため、答えは #false です。2番目のケースでは、唯一の項目が "Flatt" であるため、答えは #true です。最後に、より一般的な例を示します。

```
(check-expect (contains-flatt? (cons "A" (cons "Flatt" (cons "C" '())))) #true)
```

やはり、リストに "Flatt" が含まれているため、答えは #true になるはずです。立ち止まって、答えが #false にならないければならない一般的な例を作成してください。

一息ついて、プログラムを実行してみましょう。ヘッダーは関数の「ダミー（スタブ）」定義であり、いくつかの例があり、それらはテストに変換されています。最も良いことに、そのうちのいくつかは実際に成功します。誤った理由で成功しているのですが、成功することには変わりありません。ここまで理解できたら、先を読み進めましょう。

第4のステップは、データ定義に一致する関数テンプレートを設計することです。文字列リストのデータ定義には2つの節があるため、関数の本体は2つの節を持つ cond 式でなければなりません。2つの条件によって、関数が受け取ったのがどちらの種類のリストであるかを判定します。

```
(define (contains-flatt? alon)
  (cond
    [(empty? alon) ...]
    [(cons? alon) ...]))
```

2番目の節では、(cons? alon) の代わりに else を使用することもできます。

cond 式の各節を順に調べることで、テンプレートにさらにヒントを追加できます。具体的には、入力のクラスが複合データ（構造体など）で構成されている場合、設計レシピが各節にセレクト式を注記することを推奨しているのを思い出してください。今回のケースでは、'()' には複合データがないため、構成要素はありません。そうでない場合（cons の場合）は、リストは1つの文字列と別の文字列リストから構築されているため、テンプレートに (first alon) と (rest alon) を追加することで、この事実を思い出せるようにします。

```
(define (contains-flatt? alon)
  (cond
    [(empty? alon) ...]
    [(cons? alon)
     (... (first alon) ... (rest alon) ...)]))
```

これで、プログラミングの本来のタスクである、設計レシピの第5ステップに移行する準備が整いました。テンプレートから始めて、それぞれの cond 節を個別に処理します。

もし (empty? alon) が真なら、入力空のリストです。この場合、関数は結果として #false を生成する必要があります。

2番目のケースでは、(cons? alon) が真です。テンプレートの注記は、最初の文字列とリストの残りの部分があることを思い出させてくれます。そこで、このカテゴリに該当する例を考えてみましょう。

```
(cons "A" (cons ... '()))
```

人は名前を探すとき、最初の項目と "Flatt" を比較する必要があります。関数も同様です。この例では、最初の項目は "A" であり "Flatt" ではないため、比較結果は #false になります。代わりに他の例、例えば、

```
(cons "Flatt" (cons ... '()))
```

を考えた場合、関数は入力の最初の項目が "Flatt" であると判断し、結果として #true を返します。これは、cond 式の2行目に、リストの最初の名前と "Flatt" を比較する式を含めるべきであることを意味します。

```
(define (contains-flatt? alon)
  (cond
    [(empty? alon) #false]
    [(cons? alon)
     (... (string=? (first alon) "Flatt") ... (rest alon) ...)]))
```

さらに、比較結果が #true になった場合、関数は #true を生成する必要があります。

比較結果が #false の場合、残りの文字列リストである (rest alon) が残されます。明らかに、このケースでは関数は最終的な答えをまだ知ることができません。なぜなら、答えは残りのリストに何が含まれているか (... が何を表すか) に依存するからです。言い

換えれば、最初の項目が "Flatt" でない場合、リストの残りの部分に "Flatt" が含まれているかどうかを確認する何らかの方法が必要です。

幸いなことに、私たちには `contains-flatt?` があり、それがまさにその目的に適合します。目的ステートメントによれば、これはリストに "Flatt" が含まれているかを判定します。このステートメントは、文字列リスト `l` に対して `(contains-flatt? l)` を呼び出すことで、そのリスト `l` に "Flatt" が含まれているかがわかります。同様に、`(contains-flatt? (rest alon))` は、"Flatt" が `(rest alon)` のメンバーであるかどうかを判定します。これはまさに私たちが知る必要があることです。

要するに、最後の行には `(contains-flatt? (rest alon))` が含まれるべきです。

```
(define (contains-flatt? alon)
  (cond
    [(empty? alon) #false]
    [(cons? alon)
     (... (string=? (first alon) "Flatt")
          ... (contains-flatt? (rest alon)) ...))])
```

コツは、これら2つの式（「最初の要素が Flatt か」と「残りのリストに Flatt が含まれるか」）の値を適切に組み合わせることです。前述したように、最初の式が `#true` を生成した場合、リストの残りの部分を検索する必要はありません。しかし、それが `#false` であっても、2番目の式が `#true` を生成する可能性があり、その場合は "Flatt" がリストの残りの部分に存在することを意味します。これらすべては、最後の行の最初の式、または (or) 2番目の式が `#true` を返した場合に、`(contains-flatt? alon)` の結果が `#true` になることを示しています。

Figure 47: リストの検索

```
;; List-of-names -> Boolean
;; alon に "Flatt" が含まれているかを判定する
(check-expect (contains-flatt? (cons "X" (cons "Y" (cons "Z" '())))) #false)
(check-expect (contains-flatt? (cons "A" (cons "Flatt" (cons "C" '()))))
#true)

(define (contains-flatt? alon)
  (cond
    [(empty? alon) #false]
    [(cons? alon)
     (or (string=? (first alon) "Flatt")
         (contains-flatt? (rest alon)))]])
```

図 47 は完全な定義を示しています。全体として、この本の最初の章にある定義とそれほど違って見えません。シグネチャ、目的ステートメント、2つの例、および定義で構成されています。この関数定義が、これまでに見てきたものと唯一異なる点は、自己参照、すなわ

ち `define` の本体で `contains-flatt?` を参照している点です。しかし、データ定義自体も自己参照的であるため、関数における自己参照もそれほど驚くべきことではありません。

Exercise 132. DrRacket を使用して、以下の例で `contains-flatt?` を実行せよ。

```
(cons "Fagan"
      (cons "Findler"
            (cons "Fisler"
                  (cons "Flanagan"
                        (cons "Flatt"
                              (cons "Felleisen"
                                    (cons "Friedman" '()))))))))
```

どのような答えを予想するか？

Exercise 133. 以下は、`contains-flatt?` における 2 番目の `cond` 節のもう一つの定式化です。

```
...
[(string=? (first alon) "Flatt") #true]
[else (contains-flatt? (rest alon))]
...
```

この式が、図 47 のバージョンの `or` 式と同じ答えを生成する理由を説明せよ。あなたにとってどちらのバージョンがわかりやすいか？説明せよ。

Exercise 134. 与えられた文字列が、与えられた文字列のリストに出現するかどうかを判定する `contains?` 関数を開発せよ。（※注意: BSL には、2 つの値を受け取り、最初の値が 2 番目の値（リスト）に出現するかどうかを確認する `member?` 関数が実際に組み込まれています。）

```
> (member? "Flatt" (cons "b" (cons "Flatt" '())))
#true
```

`contains?` 関数を定義するために `member?` を使用してはなりません。

8.4 リストの計算 (Computing with Lists)

私たちはまだ BSL を使用しているため、代数の規則（インターメッツォ 1 を参照）から、DrRacket がなくても以下のような式の値を決定する方法がわかります。

```
(contains-flatt? (cons "Flatt" (cons "C" '())))
```

プログラマはこの計算がどのように機能するかを直感的に理解しておく必要があるため、この簡単な例の計算ステップを順に追ってみましょう。

図 48 は最初のステップを示しています。これは通常の代入規則 (substitution rule) を使用して適用の値を決定します。結果は条件式 (`cond` 式) になります。代数の先生が言うように、この関数は段階的な方法で定義されているからです。

Figure 48: リストを使った計算、ステップ 1

```
(contains-flatt? (cons "Flatt" (cons "C" '())))  
==  
(cond  
  [(empty? (cons "Flatt" (cons "C" '()))) #false]  
  [(cons? (cons "Flatt" (cons "C" '())))  
   (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt")  
       (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))]])
```

Figure 49: リストを使った計算、ステップ 2

```
==  
(cond  
  [#false #false]  
  [(cons? (cons "Flatt" (cons "C" '())))  
   (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt")  
       (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))]])  
==  
(cond  
  [(cons? (cons "Flatt" (cons "C" '())))  
   (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt")  
       (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))]])  
==  
(cond  
  [#true  
   (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt")  
       (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))]])  
==  
(or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt")  
    (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))
```

計算は図 49 へと続きます。cond 式の正しい節を見つけるには、条件の値を一つずつ判定する必要があります。cons されたリストは空ではないため、最初の条件の結果は #false になり、最初の cond 節は削除されます。最後に、cons されたリストに対する cons? は成立するため、2 番目の節の条件は #true に評価されます。

Figure 50: リストを使った計算、ステップ 3

```
==  
(or (string=? "Flatt" "Flatt")  
    (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))  
==  
(or #true (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))  
==  
#true
```


ここからは、最終的な結果を得るためのわずか3つの代数ステップです。図 50 にその3つのステップを示します。最初のステップは、`first` の法則に基づいて (`first (cons "Flatt" ...)`) を `"Flatt"` に評価します。2番目のステップは、`"Flatt"` が文字列であり `"Flatt"` と等しいことを見出します (`#true`)。3番目のステップは、`x` が何であれ、(`or #true x`) は `#true` であるという規則を適用します。

Figure 51: データ定義とテンプレートにおける自己参照の矢印 (データ定義 `List-of-names` の自己参照と、テンプレート `contains-flatt?` の自己参照を示すダイアグラム)

Exercise 135. DrRacket のステッパー (stepper) を使用して、以下の計算を確認せよ。

```
(contains-flatt? (cons "Flatt" (cons "C" '())))
```

また、ステッパーを使用して以下の値を判定せよ。

```
(contains-flatt? (cons "A" (cons "Flatt" (cons "C" '()))))
```

`"Flatt"` が `"B"` に置き換えられた場合、何が起こるか?

Exercise 136. DrRacket のステッパーで以下を検証せよ。

```
(our-first (our-cons "a" '())) == "a"  
(our-rest (our-cons "a" '())) == '()
```

これらの関数の定義については、セクション 8.2 「`'()` とは何か、`cons` とは何か」を参照してください。

9 自己参照データ定義を用いた設計 (Designing with Self-Referential Data Definitions)

一見すると、自己参照データ定義は、混合データに対するものよりもはるかに複雑に見えるかもしれませんが、しかし、`contains-flatt?` の例が示しているように、設計レシピの6つのステップは引き続き有効に機能します。それにもかかわらず、このセクションでは、自己参照データ定義に対して設計レシピをさらに効果的に適用できるように一般化します。新しい部分は、自己参照データ定義が必要とされる時期の発見、テンプレートの導出、および関数本体の定義プロセスに関するものです。

1. 問題分析: 問題文が「任意のサイズの情報」に関するものである場合、それを表現するために自己参照データ定義が必要です。現時点では、そのようなクラスとして `List-of-names` のみを見てきました。図 51 の左側は、同様の方法で `List-of-strings` を定義する方法を示しています。他のアトミックなデータのリストも同様に機能します。(※補足: 数値も任意に大きくなるように見えます。不正確な数 (浮動小数点数など) の場合、これは錯覚です。しかし、正確な整数 (多倍長整数など) の場合には、実際に任意に大きくなります。したがって、整数を扱うこともこの章の一部です。)

自己参照データ定義が有効であるためには、2つの条件を満たす必要があります。第一に、少なくとも2つの節 (clause) を含んでいる必要があります。第二に、少なくとも1つの節は、定義されているデータクラス自体を参照し返してはなりません。データ定義における自己参照から、定義されている用語へ戻る矢印を明示的に識別することが良い習慣です。このような注記の例については、図 51 を参照してください。

データ例の作成を通じて、自己参照データ定義の妥当性を確認する必要があります。まず、データ定義を参照しない節から始めます。次に、その最初の例を使用して、自身を参照する節を使って次の例を作成します。図 51 のデータ定義については、結果として以下のような3つのリストが得られます。

- '(): 第1節による。
- (cons "a" '()): 第2節と、前述の例による。
- (cons "b" (cons "a" '())): 第2節と、前述の例による。

もしデータ定義から例を生成することが不可能であるなら、その定義は無効です。例を生成することはできても、ますます大きな例を生成する方法が見出せない場合、その定義は意図した解釈に沿っていない可能性があります。

2. ヘッダー: シグネチャ、目的ステートメント、およびダミー定義 (スタブ) に関しては何も変わりません。目的ステートメントを定式化するときは、関数が「何を (what)」計算するのかに焦点を当て、「どのように (how)」それを処理するのか (特に、与えられたデータのインスタンスをどのように辿るか) には焦点を当てないでください。

この設計レシピを具体化するための例を示します。

```
;; List-of-strings -> Number
;; alos に含まれる文字列の個数を数える
(define (how-many alos)
  0)
```

この目的ステートメントは、関数が与えられた入力内の文字列を単に数えるだけであることを明確に述べています。このアイデアを BSL の関数としてどのように定式化するかを前もって思い悩む必要はありません。

3. 例: 機能的な例を作成する際には、データ定義の自己参照節を複数回使用した入力について必ず検討してください。これは、後で関数定義全体をカバーするテストを定式化するための最良の方法です。

私たちの進行中の例については、目的ステートメント自体がデータ例からほぼ自動的に機能例を生成します。

- (how-many '()) に対して期待される結果は 0。空リストには何も含まれていないためです。
- (how-many (cons "a" '())) に対して期待される結果は 1。1つの文字列 of リストだからです。

- (how-many (cons "b" (cons "a" '()))) に対して期待される結果は 2。2 つの文字列 of リストについてです。
4. テンプレート：本質的に、自己参照データ定義は混合データのデータ定義と似ています。したがって、テンプレートの開発は、第 I 部「項目化と構造体」のレシピに従って進めることができます。具体的には、データ定義の節と同じ数の節を持つ cond 式を定式化し、各識別条件をデータ定義の対応する節に一致させ、複合データを処理するすべての cond 行に適切なセレクト式を書き留めます。

図 52 はこの考え方を問答形式で表現したものです。

図 52: 自己参照データに対する関数の設計テンプレート質問

- 質問：データ定義はデータの異なるサブクラスを区別しているか？
 - 回答：テンプレートには、データ定義が区別するサブクラスと同じ数の cond 節が必要です。
- 質問：サブクラスは互いにどのように異なっているか？
 - 回答：その違いを利用して、節ごとに条件（テスト）を定式化します。
- 質問：構造化された値を扱う節はあるか？
 - 回答：もしあるなら、その節に適切なセレクト式を追加します。
- 質問：データ定義は自己参照を使用しているか？
 - 回答：テンプレートがデータ定義の自己参照を表すように、「自然な再帰」を定式化します。
- 質問：データ定義が他のデータ定義を参照している場合、この相互参照はどこにあるか？
 - 回答：もう一つのデータ定義のためにテンプレートを特殊化し、そのテンプレートを参照します（第 I 部「項目化の設計、再び」のステップ 4・5 を参照）。

最後の行を無視して、最初の 3 つの質問を List-of-strings を受け取る任意の関数に適用すると、次のような形状になります。

```
(define (fun-for-los alos)
  (cond
    [(empty? alos) ...]
    [else
     (... (first alos) ... (rest alos) ...)]))
```

しかし、テンプレートの目的は、データ定義を関数のレイアウトとして表現することであることを思い出してください。つまり、テンプレートは、入力 of データ定義が英語と BSL の混在で表現している内容をコードとして表現します。したがって、データ定義のすべての重要な部分はテンプレートに対応するものを見つけなければならない、このガイドラインはデータ定義が自己参照的である（定義の内部から定義される用語への矢印が含まれる）場合にも適用されるべきです。特に、データ定義の第 i 節で言及されている構造体の第 k フィールドで自己参照している場合、テンプレートは第 i

番目の `cond` 節の第 k フィールドのセレクタ式において自己参照するべきです。そのようなセレクタ式ごとに、関数のパラメータに戻る矢印を追加します。最終的に、テンプレートにはデータ定義と同じ数の矢印が存在しなければなりません。

図 51 は、この考え方を `List-of-strings` を受け取る関数のテンプレートとデータ定義を並めて示しています。両方とも、第 2 節（それぞれ `rest` フィールドと `rest` セレクタ）から始まり、それぞれの定義のトップを指し示す 1 つの矢印を含んでいます。

BSL やほとんどのプログラミング言語はテキスト指向であるため、矢印の代替手段、すなわち適切なセレクタ式に対する関数の自己適用を使用する必要があります。

```
(define (fun-for-los alos)
  (cond
    [(empty? alos) ...]
    [else
     (... (first alos) ... (fun-for-los (rest alos)) ...)]))
```

関数の自己利用を再帰 (recursion) と呼び、この本の最初の 4 つの部では自然な再帰 (natural recursion) と呼びます。

5. 定義: 関数本体については、再帰的な関数呼び出しを含まない `cond` 行、すなわち基本ケース (base cases) から始めます。対応する答えは通常、定式化が容易であるか、すでに例として与えられています。

次に、自己参照のケースを処理します。まず、テンプレート行の各式が何を計算するかを自分自身に思い出させることから始めます。自然な再帰については、関数が目的ステートメントで指定された通りにすでに動作していると仮定します。この最後のステップは「信頼の飛躍 (leap of faith)」ですが、見ての通り、常に機能します。

残る作業は、さまざまな値を組み合わせることです。

図 53: テンプレートを関数定義に変換する手順

- 質問: 非再帰的な `cond` 節に対する答えは何か?
 - ▶ 回答: 例がここで必要な値を示しているはずです。そうでなければ、適切な例とテストを定式化してください。
- 質問: 再帰節のセレクタ式は何を計算するか?
 - ▶ 回答: データ定義がこれらの式がどのような種類のデータを抽出するかを教えてください、データ定義の解釈がこのデータが何を表すかを教えてください。
- 質問: 自然な再帰は何を計算するか?
 - ▶ 回答: 関数の目的ステートメントを使用して、再帰の値が何を意味するかを決定します (どのようにこの答えを計算するかではありません)。目的ステートメントが答えを教えてくれない場合は、目的ステートメントを改善してください。
- 質問: 関数は望ましい答えを得るために、これらの値をどのように組み合わせることができるか?

- ・ 回答: 値を組み合わせる BSL の関数を見つけます。それが機能しない場合は、補助関数を「願望 (wish)」として設計します。多くの関数にとって、この最後のステップは簡単です。目的、例、およびテンプレートが組み合わせあって、利用可能な値を適切な結果に組み合わせる (combine) 関数や式を教えてください。この関数や式を、既存の用語を少し拡張して「結合子 (combinator)」と呼びます。

図 54: テンプレートから関数への変換、テーブル法

- ・ 質問: ここで行き詰まった場合はどうすればよいか?
 - ・ 回答: 第 3 のステップの例をテーブルに整理します。左端の列に与えられた入力を配置し、右端の列に望ましい出力を配置します。中間の列に、セレクト式と自然な再帰の値を入力します。結合子を示唆するパターンが現れるまで、例の行を追加してください。
- ・ 質問: テンプレートが他のテンプレートを参照している場合、補助関数は何を計算するか?
 - ・ 回答: その補助関数の目的ステートメントと例を参照して何を計算するかを判断し、そのヘルパー関数の設計が完了していなくても、その結果を利用できると仮定します。

図 53 はこのステップに関する最初の 4 つの質問と回答を定式化したものです。このゲームを使って how-many の定義を完成させましょう。fun-for-los テンプレートを how-many に名前変更すると、ここまでのものが得られます。

```
;; List-of-strings -> Number
;; alos に含まれる文字列の個数を判定する
(define (how-many alos)
  (cond
    [(empty? alos) ...]
    [else
     (... (first alos) ... (how-many (rest alos)) ...)]))
```

機能的な例がすでに示唆しているように、基本ケースに対する答えは 0 です。2 番目の節の 2 つの式は、最初の項目と (rest alos) 内の文字列の個数を計算します。alos 全体に何個の文字列があるかを計算するには、関数は後者の式の値に 1 を加算するだけです。

```
(define (how-many alos)
  (cond
    [(empty? alos) 0]
    [else
     (+ (how-many (rest alos)) 1)]))
```

値を望ましい答えに組み合わせる正しい方法を見つけることは、常にこれほど簡単であるとは限りません。初心者のプログラマは、このステップでよく行き詰まります。図 54 が示唆するように、機能的な例を、テンプレート内の式の値も明記したテーブルに整理することをお勧めします。図 55 は、how-many の例におけるこのテーブルの様子を

示しています。左端の列にはサンプル入力をリストし、右端の列にはこれらの入力に対する望ましい答えを含めます。その間の3つの列は、テンプレート式の値を示しています:(first alos)、(rest alos)、そして自然な再帰である (how-many (rest alos)) です。このテーブルを十分に長く見つめれば、結果の列が常に自然な再帰の列の値よりも 1 だけ大きいことに気づくでしょう。したがって、あなたは以下のように推測するかもしれません。

```
(+ (how-many (rest alos)) 1)
```

これが望ましい結果を計算する式です。DrRacketはこの種の推測を素早くチェックできるため、それをプラグインして「RUN」をクリックします。例がテストに合格した場合は、その式がすべてのリストで機能することを確認するために思考を巡らせます。そうでない場合は、別のアイデアが得られるまでテーブルにさらに多くの例の行を追加します。

図 55: 引数、中間値、および結果の集計 (Tabulating arguments, intermediate values, and results)

alos	(first alos)	(rest alos)	(how-many (rest alos))	(how-many alos)
'()			0	1
(cons "a" '())	"a"	'()	0	1
(cons "b" (cons "a" '()))	"b"	(cons "a" '())	1	2
(cons "x" (cons "b" (cons "a" '())))	"x"	(cons "b" (cons "a" '()))	2	3

このテーブルは、テンプレート内のいくつかのセレクト式が実際の定義においては無関係である可能性もあることを指摘しています。ここでは、最終的な答えを計算するために (first alos) は必要ありません。これは、テンプレートの画一的な両方の式を使用する contains-flatt? とは対照的です。

この本の残りの部分を進めるにあたり、多くの場合、組み合わせステップは BSL のプリミティブ (例えば +、and、cons) で表現できることを覚えておってください。しかし、場合によっては、補助関数を設計 (願望) しなければならないこともあります。また、入れ子になった条件が必要な場合もあります。

6. テスト: 最後に、すべての例をテストに変換し、これらのテストが合格し、それらを実行することが関数のすべての部分をカバーしていることを確認してください。

how-many の例をテストに変換したものは以下の通りです。

```
(check-expect (how-many '()) 0)
(check-expect (how-many (cons "a" '())) 1)
(check-expect (how-many (cons "b" (cons "a" '()))) 2)
```

例を直接テストとして定式化するのが最善であり、BSLはこれを可能にします。そうすることは、前ステップのテーブルベースの推測アプローチに頼る必要がある場合にも役立ちます。

図 56 は、このセクションの設計レシピを要約したものです。この図は、この章で用いるような自己参照リストの定義に合わせて調整されています。

図 56: 自己参照データに対する関数の設計 (Designing a function for self-referential data) | ステップ (steps) | 成果 (outcome) | 活動 (activity) | | :
- | :- | :- | | 問題分析 (problem analysis) | データ定義 (data definition)
| 情報のためのデータ表現を構築する。具体的な情報について例を作成し、データを情報として解釈する。自己参照を識別する。 | | ヘッダー (header) | シグネチャ;
目的; ダミー定義 | 定義された名前を使ってシグネチャを書き下ろす。簡潔な目的
ステートメントを定式化する。指定された値の範囲から定数値を生成するダミー関数
を作成する。 | | 例 (examples) | 例とテスト | いくつかの例を作成する。デー
タ定義の各節につき、少なくとも 1 つは作成する。 | | テンプレート (template) |
関数テンプレート | データ定義をテンプレートに翻訳する: データ節ごとに 1 つの
cond 節; 条件が構造体を識別する場所にはセクタ; 自己参照ごとに 1 つの自然な再
帰。 | | 定義 (definition) | 完全な定義 | cond 節内の式の値を組み合わせるこ
とで、期待される答えを算出する関数を見つける。 | | テスト (test) | 検証され
たテスト | 例を check-expect テストに変換し、実行する。 |

9.1 指の運動: リスト (Finger Exercises: Lists)

Exercise 137. `contains-flatt?` のテンプレートと `how-many` のテンプレートと比較しなさい。関数名を無視すれば、それらは全く同じです。この類似性を説明しなさい。

Exercise 138. 金額のシーケンスを表現するためのデータ定義を以下に示します。

```
;; A List-of-amounts is one of:  
;; - '()  
;; - (cons PositiveNumber List-of-amounts)
```

このデータ定義を理解するために、いくつかの例を作成しなさい。また、自己参照を指す矢印を追加しなさい。

`List-of-amounts` を受け取り、金額の合計を計算する `sum` 関数を設計しなさい。DrRacket のステップ実行 (Stepper) を使用して、`List-of-amounts` に含まれる短いリスト `l` に対して `(sum l)` がどのように機能するかを確認しなさい。

Exercise 139. 次に、このデータ定義を見てみましょう。

```
;; A List-of-numbers is one of:  
;; - '()  
;; - (cons Number List-of-numbers)
```

このデータクラスの一部の要素は、Exercise 138 の `sum` への入力として適切ですが、適切でないものもあります。

`List-of-numbers` を受け取り、すべての数値が正の数であるかどうかを判定する `pos?` 関数を設計しなさい。言い換えれば、`(pos? l)` が `#true` を返す場合、`l` は `List-of-amounts`

の要素です。DrRacket のステップ実行を使用して、`(cons 5 '())` および `(cons -1 '())` に対して `pos?` がどのように機能するかを確認しなさい。

また、`checked-sum` を設計しなさい。この関数は `List-of-numbers` を受け取ります。入力が `List-of-amounts` にも属していれば、それらの合計を算出します。そうでない場合はエラーを発生させます。ヒント：`check-error` を使用することを思い出してください。

`List-of-numbers` の要素に対して `sum` は何を計算するのでしょうか？

Exercise 140. Boolean 値のリストを受け取り、それらすべてが `#true` であるかどうかを判定する `all-true` 関数を設計しなさい。言い換えれば、リスト内に1つでも `#false` があれば、関数は `#false` を返します。

次に、Boolean 値 of リストを受け取り、リスト内の少なくとも1つの項目が `#true` であるかどうかを判定する `one-true` 関数を設計しなさい。

コーディングにあたっては、テーブルベースのアプローチを採用しなさい。これは基本ケース (base case) を考える際に役立ちます。DrRacket のステップ実行を使用して、これらの関数がリスト `(cons #true '())`、`(cons #false '())`、および `(cons #true (cons #false '()))` をどのように処理するかを確認しなさい。

Exercise 141. 文字列のリストを受け取り、それらすべてを結合して1つの長い文字列にする `cat` 関数の設計を求められた場合、以下のような不完全な定義に必ず辿り着くことになります。

```
;; List-of-string -> String
;; l に含まれるすべての文字列を1つの長い文字列に結合する

(check-expect (cat '()) "")
(check-expect (cat (cons "a" (cons "b" '())))) "ab")
(check-expect (cat (cons "ab" (cons "cd" (cons "ef" '()))))) "abcdef")

(define (cat l)
  (cond
    [(empty? l) ""]
    [else (... (first l) ... (cat (rest l)) ...)]))
```

この定義を完成させなさい。残された作業は、`(first l)` と `(cat (rest l))` の値を組み合わせて望ましい答えを得るための結合子を見つけることです。ヒント:文字列に関する第1部 (「アトミックデータ」) のガイドを参照してください。

9.2 空でないリスト (Non-empty Lists)

これで、`cons` を使用してリストのデータ定義を作成する方法を十分に理解できます。前のセクションの最後にある練習問題のいくつか (あるいはすべて) を解いたならば、さまざまな種類の数値のリスト、Boolean 値のリスト、画像のリストなどを扱うことができるようになっています。このセクションでは、リストとは何か、そしてそれをどのように処理するかについてさらに探求を続けます。

まずは、温度のリストの平均値を計算するという、一見簡単そうな問題から始めましょう。簡単にするために、以下のデータ定義を提供します。

```
(define ABSOLUTE0 -272)
;; A CTemperature is a Number greater than ABSOLUTE0.

;; A List-of-temperatures is one of:
;; - '()
;; - (cons CTemperature List-of-temperatures)
```

私たちの目的においては、温度をプレーンな数値と考えて差し支えありませんが、2番目のデータ定義は、実際にはすべての数値が温度であるわけではないことを思い出させます。この点を頭に留めておいてください。

ヘッダーに関する記述は素直なものです。

```
;; List-of-temperatures -> Number
;; 平均温度を計算する
(define (average alot)
  0)
```

この問題の例を作成するのも簡単ですので、テストを1つだけ定式化します。

```
(check-expect (average (cons 1 (cons 2 (cons 3 '())))) 2)
```

期待される結果はもちろん、温度の合計を温度の個数で割ったものです。

少し考えると、average のテンプレートはこれまで見てきたものと同様になるはずだとわかります。

```
(define (average alot)
  (cond
    [(empty? alot) ...]
    [(cons? alot)
     (... (first alot) ... (average (rest alot)) ...)]))
```

2つの cond 節はデータ定義の2つの節を反映しており、条件式は空リストと空でないリストを区別します。そして、データ定義の自己参照のために、自然な再帰が必要となります。

しかし、このテンプレートを関数定義に変換するのはあまりにも困難です。最初の cond 節には、空の温度コレクションの平均を表す数値が必要ですが、そのような数値は存在しません。同様に、2番目の節は、1つの温度と、残りの温度の平均値を組み合わせる新しい平均値を計算する関数を要求します。そのような計算方法も不可能ではありませんが、極めて不自然です。

温度の平均を計算するとき、私たちはそれらの合計を個数で割ります。この簡単な例を定式化したときにもそう言いました。この文は、average が「合計する」「数える」「割る」という3つのタスクからなる関数であることを示唆しています。「固定サイズデータ」のガイ

ドラインは、1つのタスクにつき1つの関数を書くように教えており、そうすれば `average` の設計は明白になります。

```
;; List-of-temperatures -> Number
;; 平均温度を計算する
(define (average alot)
  (/ (sum alot) (how-many alot)))
```

```
;; List-of-temperatures -> Number
;; 与えられたリスト内の温度を合計する
(define (sum alot)
  0)
```

```
;; List-of-temperatures -> Number
;; 与えられたリスト内の温度を数える
(define (how-many alot)
  0)
```

もちろん、後ろの2つの関数定義は「願望 (wishes)」であり、これらについて完全な定義を設計する必要があります。これを行うのは簡単です。なぜなら、前述の `how-many` は `List-of-strings` にも `List-of-temperatures` にも機能し（なぜでしょう？）、`sum` の設計もこれまでと同じお決まりの手順に従うからです。

```
;; List-of-temperatures -> Number
;; 与えられたリスト内の温度を合計する
(define (sum alot)
  (cond
    [(empty? alot) 0]
    [else
     (+ (first alot) (sum (rest alot)))]))
```

ちょっと待ってください！`average` の例を参考にして `sum` の例を作成し、そのテストが適切に動作することを確認しなさい。その後、`average` のテストを実行しなさい。

この `average` の定義を読めば、学校で平均について学ぶ内容に直接対応しているため、明らかに正しいことがわかります。それでも、プログラムは私たちのためだけでなく、他人のためにも実行されます。特に、他の開発者はシグネチャを読んで関数を利用し、有益な回答を期待できるべきです。しかし、私たちの `average` の定義は、空の温度リストに対しては機能しません。

Exercise 143. `average` を空リストに適用した際、`DrRacket` でどのような挙動になるか確認しなさい。その後、`()` に適用されたときに有益なエラーメッセージを生成する関数 `checked-average` を設計しなさい。

数学的には、Exercise 143 は `average` が「部分関数 (partial function)」であることを示しています。なぜなら、`()` に対してエラーを発生させるからです。

代替の解決策は、average が空リストに対しては機能しないことをシグネチャを通じて将来の読者に知らせることです。そのためには、'()' を除外するようなリストのデータ表現が必要になります。例えば次のようになります。

```
;; An NEList-of-temperatures is one of:  
;; - ???  
;; - (cons CTemperature NEList-of-temperatures)
```

問題は、'()' リストを除外しつつ、他のすべての温度リストを構築できるようにするために、“???” を何に置き換えるべきかということです。1つのヒントは、空リストが最も短いリストであるのに対し、1つの温度のリストが次に短いリストであるということです。このことから、第1節は「1つの温度のすべての可能なリスト」を記述すべきだということが示唆されます。

```
;; An NEList-of-temperatures is one of:  
;; - (cons CTemperature '())  
;; - (cons CTemperature NEList-of-temperatures)  
;; interpretation: Celsius temperatures の空でないリスト
```

この定義は以前のリスト定義とは異なりますが、「自己参照」と「自己参照を用いない節」という決定的な要素を共有しています。設計レシピを厳格に遵守するためには、定義が意味をなすことを確認するために NEList-of-temperatures の例をいくつか作成する必要があります。いつものように基本節から始めるため、例は次のようになります。

```
(cons c '())
```

ここで c は CTemperature を表し、例えば (cons ABSOLUTE0 '()) のようになります。また、List-of-temperatures のすべての空でない要素が、この新しいデータクラスの要素でもあることは明らかです。もし (cons 2 (cons 3 '())) がそうであれば、(cons 1 (cons 2 (cons 3 '()))) も適合します。そして、(cons 2 (cons 3 '())) が NEList-of-temperatures に属するのは、すでに確認したように (cons 3 '()) が NEList-of-temperatures の要素だからです。NEList-of-temperatures のサイズには制限がないことを自分で確認下さい。

では、誰もが「空でないリスト専用である」とわかるように average を設計する問題に戻りましょう。NEList-of-temperatures の定義があれば、シグネチャでやりたいことを表現する手段が得られます。（※この代替的な開発は、この場合、average の定義域を狭めて「全域関数 (total function)」を作成できることを説明しています。）

```
;; NEList-of-temperatures -> Number  
;; 平均温度を計算する  
  
(check-expect (average (cons 1 (cons 2 (cons 3 '())))) 2)  
  
(define (average ne-l)  
  (/ (sum ne-l) (how-many ne-l)))
```

当然、残りの部分（目的ステートメント、例とテスト、関数定義）は同じままです。結局のところ、平均の計算というアイデア自体が、空でない数値のコレクションを前提としており、それが私たちの議論の要点でした。

Exercise 144. `sum` と `how-many` は、`List-of-temperatures` からの入力を想定して設計されているにもかかわらず、`NEList-of-temperatures` に対しても機能するでしょうか？機能しないと思う場合は反例を示しなさい。機能すると思う場合はその理由を説明しなさい。

それにもかかわらず、この定義は、`sum` と `how-many` が `NEList-of-temperatures` のインスタンスを受け取るようになったため、これらをどのように設計するかという疑問も生じさせます。設計レシピの最初の 3 ステップの明らかな結果は以下の通りです。

```
;; NEList-of-temperatures -> Number
;; 与えられた温度の合計を計算する
(check-expect (sum (cons 1 (cons 2 (cons 3 '())))) 6)
(define (sum ne-l)
  0)
```

この例は `average` の例を適応させたものです。ダミー定義は数値を生成しますが、与えられたテストに対しては誤った数値です。

4 番目のステップは、`NEList-of-temperatures` に対する `sum` の設計において最も興味深い部分です。これまでの設計例はすべて、データ定義の形状に基づいて、空リストと空でない（つまり `cons` された）リストを区別するテンプレートを要求していました。しかし、`NEList-of-temperatures` にはこれが当てはまりません。ここでは、両方の節が `cons` されたリストを追加します。ただし、これら 2 つの節は、リストの `rest` フィールドにおいて異なります。特に、第 1 節は常に `rest` フィールドに `'()` を使用し、第 2 節は代わりに `cons` を使用します。したがって、第 1 のデータと第 2 のデータを区別するための適切な条件は、`rest` フィールドを取り出して `empty?` を使用することです。

```
;; NEList-of-temperatures -> Number
(define (sum ne-l)
  (cond
    [(empty? (rest ne-l)) ...]
    [else ...]))
```

ここでは、`else` が `(cons? (rest ne-l))` に置き換わっています。

次に、両方の節を検査し、それらが `ne-l` を構造体であるかのように扱うかどうかを判断する必要があります。もちろんその通りであり、`ne-l` に対して無条件に `rest` を使用していることがそれを示しています。言い換えれば、2 つの節に適切なセレクト式を追加します。

```
(define (sum ne-l)
  (cond
    [(empty? (rest ne-l)) (... (first ne-l) ...)]
    [else (... (first ne-l) ... (rest ne-l) ...)]))
```

読み進める前に、なぜ最初の節にはセクタ式 (rest ne-l) が含まれていないのか説明しなさい。

テンプレート設計 of 最後の質問は、データ定義における自己参照に関するものです。ご存知のように、NEList-of-temperatures には自己参照が1つ含まれているため、sum のテンプレートは1つの再帰的な使用を要求します。

```
(define (sum ne-l)
  (cond
    [(empty? (rest ne-l)) (... (first ne-l) ...)]
    [else (... (first ne-l) ... (sum (rest ne-l)) ...)]))
```

具体的には、データ定義の対応する箇所が自己参照的であるため、2番目の節で (rest ne-l) に対して sum が呼び出されます。

5番目の設計ステップでは、私たちがすでにどれだけのものを手に入れているかを理解しましょう。最初の cond 節は、再帰的な関数呼び出しを持つ2番目の節よりも大幅にシンプルに見えるため、そちらから始めるべきです。この特定のケースでは、条件は sum が正確に1つの温度 (first ne-l) を持つリストに適用されることを示しています。明らかに、この1つの温度が、与えられたリスト内のすべての温度の合計です。

```
(define (sum ne-l)
  (cond
    [(empty? (rest ne-l)) (first ne-l)]
    [else (... (first ne-l) ... (sum (rest ne-l)) ...)]))
```

2番目の節は、リストが1つの温度と少なくとももう1つの温度から構成されていることを示しています。(first ne-l) は最初の位置を抽出し、(rest ne-l) は残りの位置を抽出します。さらに、テンプレートは (sum (rest ne-l)) の結果を使用することを示唆しています。しかし、sum はあなたが定義している最中の関数であり、それが (rest ne-l) をどのように使用するかを知る由もありません。あなたが知っているのは目的ステートメントの内容だけです。すなわち、sum は与えられたリスト (ここでは (rest ne-l)) 内のすべての温度を合計するということです。このステートメントが真であるならば、(sum (rest ne-l)) は ne-l の1つを除くすべての数値を合計します。合計を得るために、関数は最初の温度を加算するだけです。

```
(define (sum ne-l)
  (cond
    [(empty? (rest ne-l)) (first ne-l)]
    [else (+ (first ne-l) (sum (rest ne-l)))]))
```

ここでこの関数のテストを実行すると、信頼の飛躍が正当化されることがわかります。実際、本書の範囲を超える理由により、この飛躍は常に正当化されます。これが、信頼の飛躍が設計レシピの不可欠な一部である理由です。

Exercise 145. NEList-of-temperatures を受け取り、温度が降順でソートされている場合に #true を返す述語 sorted>? を設計しなさい。すなわち、2番目が最初より小さく、3

番目が2番目より小さく、というようになっている場合です。そうでない場合は `#false` を返します。

ヒント : この問題も、結合子を推測するのにテーブルベースの方法がうまく機能する問題です。図 58 にいくつかの例の部分的なテーブルを示します。テーブルの残りの部分を埋めなさい。その後、それらのパーツから結果を計算する式を作成しなさい。

図 58: `sorted>?` のためのテーブル

l	(first l)	(rest l)	(sorted>? (rest l))
(sorted>? l)	:-	:-	:-
(cons 1 (cons 2 '()))	1	(cons 2 '())	#true
(cons 3 (cons 2 '()))	3	(cons 2 '())	#true
(cons 0 (cons 3 (cons 2 '())))	0	(cons 3 (cons 2 '()))	#false

Exercise 146. `NEList-of-temperatures` に対する `how-many` を設計しなさい。これを行うことで `average` が完成するため、`average` がすべてのテストに合格することも確認しなさい。

Exercise 147. Boolean 値の空でないリストの表現である `NEList-of-Booleans` のデータ定義を作成しなさい。その後、Exercise 140 の `all-true` および `one-true` 関数を再設計しなさい。

Exercise 148. このセクションの関数定義 (`sum`, `how-many`, `all-true`, `one-true`) を、前のセクションの対応する関数定義と比較しなさい。空リストを許容するデータ定義を扱う方が、空でないリストの定義を扱うよりも優れているのでしょうか?なぜそう言えるのでしょうか?あるいはそう言えないのでしょうか?

9.3 自然数 (Natural Numbers)

BSL プログラミング言語は、リストを受け取る多くの関数を提供しており、リストを生成する関数もいくつか提供しています。その中には `make-list` があります。これは、数値 `n` と他の任意の値 `v` を受け取り、`v` を `n` 個含むリストを生成します。以下にいくつかの例を示します。

```
> (make-list 2 "hello")
(cons "hello" (cons "hello" '()))
> (make-list 3 #true)
(cons #true (cons #true (cons #true '())))
> (make-list 0 17)
'()
```

要するに、この関数はアトミックなデータを受け取るにもかかわらず、任意に大きなデータを生成します。疑問に思ふべきは、これがどのようにして可能なのかということです。

私たちの答えは、`make-list` の入力が単なる数値ではなく、特別な種類の数値であるということです。幼稚園で、これらの数値はオブジェクトを数えるために使用されるため、「数える数」と呼びました。コンピュータサイエンスでは、これらの数値は自然数 (natural numbers) と呼ばれます。通常の数値とは異なり、自然数にはデータ定義が存在します。

```
;; An N is one of:
;; - 0
;; - (add1 N)
;; interpretation: 物の個数を数える数を表す
```

最初の節は 0 が自然数であることを述べています。これは、数えるべきオブジェクトがないことを示すために使用されます。2 番目の節は、n が自然数であれば n+1 も自然数であることを示しています。なぜなら、add1 は与えられた任意の数値に 1 を加算する関数だからです。この 2 番目の節を (+ n 1) と書くこともできますが、add1 の使用は、この加算が特別であることを示すためのものです。

この add1 の使用が特別である理由は、それが通常の変数というよりも、構造体型の定義におけるコンストラクタのように機能することです。そのため、BSL には add1 に対応する「セクタ」である sub1 関数も用意されています。0 と等しくない任意の自然数 m が与えられたとき、sub1 を使用して m の構築に使われた数値を見つけることができます。言い換えれば、add1 は cons のようなものであり、sub1 は first や rest のようなものです。

この時点で、0 と、0 ではない自然数を区別する述語は何だろうかと思うかもしれません。リストと同様に、これには 2 つの述語があります。与えられた数値が 0 であるかを判定する zero? と、数値が 0 より大きいかを判定する positive? です。

これで、自然数に対する関数（例えば make-list など）を自分で設計する準備が整いました。データ定義はすでに利用可能ですので、ヘッダーの要素を追加しましょう。

```
;; N String -> List-of-strings
;; n 個の s のコピーからなるリストを作成する

(check-expect (copier 0 "hello") '())
(check-expect (copier 2 "hello") (cons "hello" (cons "hello" '())))

(define (copier n s)
  '())
```

テンプレートの開発が次のステップです。テンプレートに関する質問は、copier の本体が、0 に対する節と正の数に対する節の 2 つの節を持つ cond 式であることを示唆しています。さらに、0 はアトムとみなされ、正の数は構造化された値とみなされるため、テンプレートは 2 番目の節にセクタ式を必要とします。最後に重要な点として、N のデータ定義は 2 番目の節において自己参照的です。したがって、テンプレートは 2 番目の節の対応するセクタ式に対して再帰的な適用を必要とします。

```
(define (copier n s)
  (cond
    [(zero? n) ...]
    [(positive? n)
     (... (copier (sub1 n) s) ...)]))
```


図 59 は、テンプレートから得られた `copier` 関数の完全な定義を示しています。このステップを注意深く再現してみましょう。いつものように、再帰呼び出しのない `cond` 節から始めます。ここでは、条件は（重要な）入力が 0 であることを示しており、それは関数が 0 個の項目、すなわち何もないリストを生成しなければならないことを意味します。もちろん、2 番目の例に取り組むことで、このケースはすでに明らかになっています。次に、もう一方の `cond` 節に目を向け、その式が何を計算するかを思い出します。

1. `(sub1 n)` は、`n` (0 より大きいことがわかっています) の構築に使われた自然数を抽出します。
2. `(copier (sub1 n) s)` は、`copier` の目的ステートメントに従って、`(sub1 n)` 個の文字列 `s` のリストを生成します。

しかし、関数には `n` が与えられているため、`n` 個の文字列 `s` のリストを生成しなければなりません。文字列が 1 つ少ないリストが与えられた場合、関数は単に `(copier (sub1 n) s)` の結果の先頭に `s` を `cons` するだけでよいことは容易に理解できます。And それこそが 2 番目の節が指定していることです。

図 59: コピーのリストの作成 (Creating a list of copies)

```
;; N String -> List-of-strings
;; n 個の s のコピーからなるリストを作成する

(check-expect (copier 0 "hello") '())
(check-expect (copier 2 "hello")
               (cons "hello" (cons "hello" '())))

(define (copier n s)
  (cond
    [(zero? n) '()]
    [(positive? n) (cons s (copier (sub1 n) s))]))
```

この時点で、テストを実行して、この関数が少なくとも 2 つの作成した例で動作することを確認する必要があります。さらに、いくつかの追加の入力で関数を使用してみるのもよいでしょう。

Exercise 149. `copier` は、自然数と Boolean 値や画像に適用したときにも適切に動作するでしょうか？それとも、別の関数を設計しなければならないでしょうか？その答えについては、第 III 部「抽象化」を読みなさい。

`copier` の代替の定義では、`else` を使用することができます。

```
(define (copier.v2 n s)
  (cond
    [(zero? n) '()]
    [else (cons s (copier.v2 (sub1 n) s))]))
```

copier と copier.v2 を 0.1 と "x" に適用したとき、これらはどのように動作するでしょうか？説明しなさい。説明を確認するために DrRacket のステップ実行を使用しなさい。

Exercise 150. add-to-pi 関数を設計しなさい。これは自然数 n を受け取り、プリミティブな $+$ 演算を使用せずに、それを π に加算します。以下に開始コードを示します。

```
;; N -> Number
;; + を使用せずに (+ n pi) を計算する

(check-within (add-to-pi 3) (+ 3 pi) 0.001)

(define (add-to-pi n)
  pi)
```

完全な定義を作成したら、この関数を add に一般化しなさい。add は、 $+$ を使用せずに自然数 n を任意の数値 x に加算します。なぜスケルトン（開始コード）では check-within を使用しているのでしょうか？

Exercise 151. multiply 関数を設計しなさい。これは自然数 n を受け取り、 $*$ を使用せずに数値 x と掛け合わせます。

DrRacket のステップ実行を使用して、お好みの x に対して (multiply 3 x) を評価しなさい。multiply は、あなたが小学校で学んだこととどのように関連しているのでしょうか？

Exercise 152. col と row という2つの関数を設計しなさい。

関数 col は自然数 n と画像 img を受け取ります。img を縦方向に n 個並べた画像（カラム）を生成します。

関数 row は自然数 n と画像 img を受け取ります。img を横方向に n 個並べた画像（行）を生成します。

Exercise 153. この練習問題の目的は、1968年スタイルのヨーロッパの学生運動の結果を視覚化することです。大まかなアイデアは以下の通りです。少数の学生が集まって絵の具を詰めた風船を作り、講堂に入り、出席者に向けて風船をランダムに投げます。あなたのプログラムは、風船が講堂の座席をどのように彩るかを表示します。

Exercise 152 の2つの関数を使用して、サイズが 10×10 の正方形が COLUMNS 行 ROWS 列並んだ長方形を作成しなさい。それを同じサイズの empty-scene に配置します。この画像があなたの講堂です。

add-balloons を設計しなさい。この関数は、講堂の寸法に収まる座標を持つ Posn のリストを受け取ります。これは、Posn によって指定された場所に赤色のドットが追加された講堂の画像を生成します。

図 60 は、COLUMNS が 10、ROWS が 20 で、いくつかの Posn リストが与えられた場合の私たちの解決策の出力を示しています。左端はきれいな講堂、2 番目は2つの風船が当たった

後、最後は非常に起こりそうにない10回の命中の分布です。10番目はどこにあるでしょうか？

9.4 ロシア人形 (Russian Dolls)

Wikipediaはロシア人形(マトリョーシカ)を「サイズが小さくなっていく人形で、一方が他方の内部に入れ子状に配置された一組の人形」と定義し、次の写真で説明しています。

[Wikipediaのロシア人形の写真]

この写真では、鑑賞者がすべての人形を見ることができるよう、人形が分解されています。(※この問題は抽象的、あるいは不条理にさえ思えるかもしれません。ロシア人形を表現したい理由や、そのような表現で何をするのかが明確ではないからです。今はただ、この例に付き合ってみてください。)

ここで、そのようなロシア人形をBSLのデータで表現する問題を考えてみましょう。少しの想像力があれば、アーティストが任意の数の人形からなるロシア人形の入れ子を作成できることは容易に理解できます。結局のところ、与えられたロシア人形の周囲にいつでも別のレイヤーを巻き付けることが可能です。一方で、中には何も無い頑丈な人形が深くに入っていることもご知でしょう。

ロシア人形の各レイヤーについて、私たちは多くの異なる事柄に関心を持つことができます。そのサイズ(ただし、これは入れ子の深さに関連しています)、その色、表面に描かれている絵、などです。ここでは、人形の色だけを取り上げ、それを文字列で表現します。それを前提とすると、ロシア人形の各レイヤーには、その色と、その内側にある人形という2つのプロパティがあることがわかります。2つのプロパティを持つ情報を表現するために、私たちは常に構造体型を定義します。

```
(define-struct layer [color doll])
```

And そして、データ定義を追加します。

```
;; An RD (short for Russian doll) is one of:  
;; - String  
;; - (make-layer String RD)
```

当然、このデータ定義の最初の節は、最も内側の人形、正確に言うとその色を表しています。2番目の節は、与えられたロシア人形の周囲にレイヤーを追加するためのものです。これをlayerのインスタンスで表現します。これには明らかに人形の色と、もう1つのフィールド、すなわちこの人形のすぐ内側に入れ子になっている人形が含まれています。

この人形を見てみましょう。

[3つのレイヤーからなる人形の画像]

これは3つの人形で構成されています。赤が最も内側にあり、緑が真ん中にあり、黄が現在の最も外側のラッパーです。この人形をRDの要素として表現するには、どちらの端から始めても構いません。ここでは内側から外側へと進めます。赤い人形をRDとして表現

するのは簡単です。中には何もなく、赤色なので、文字列 "red" で十分です。2 番目のレイヤーには、以下を使用します。

```
(make-layer "green" "red")
```

これは、緑色の（中空の）人形が赤い人形を含んでいることを示しています。最後に、最も外側を得るために、この最後の人形の周囲に別のレイヤーを巻き付けます。

```
(make-layer "yellow" (make-layer "green" "red"))
```

このプロセスにより、色付きのロシア人形の任意のセットからデータ表現へと移行する方法がよくわかるはずです。しかし、プログラマは逆のこともできなければならない、すなわち、データから具体的な情報へと移行できなければならないということを頭に留めておいてください。この精神で、次の RD の要素について略図を描きなさい。

```
(make-layer "pink" (make-layer "black" "white"))
```

これを BSL で試してみるのもよいでしょう。

データ定義が手に入り、実際の人形を表現する方法と RD の要素を人形として解釈する方法を理解したところで、RD を受け取る関数を設計する準備が整いました。具体的には、ロシア人形のセットに何個の人形が含まれているかを数える関数を設計しましょう。この文は優れた目的ステートメントであり、シグネチャも決定します。

```
;; RD -> Number  
;; an-rd に含まれる人形の個数を数える
```

データ例については、(make-layer "yellow" (make-layer "green" "red")) から始めましょう。上記の画像から、赤、緑、黄の3つの人形があるため、期待される答えは 3 であることがわかります。この1つの例に取り組むだけでも、入力が最も内側の人形を表している場合、答えは 1 になることがわかります。

ステップ4ではテンプレートの開発が要求されます。このステップの標準的な質問を使用すると、次のテンプレートが生成されます。

```
;; RD -> Number  
(define (depth an-rd)  
  (cond  
    [(string? an-rd) ...]  
    [(layer? an-rd)  
     (... (layer-color an-rd) ...  
           ... (depth (layer-doll an-rd)) ...)]))
```

cond 節の数は、RD の定義における節の数によって決定されます。各節は、それがどのようなデータに関するものであるかを具体的に示しており、そこから string? および layer? 述語を使用するよう指示されます。文字列は複合データではありませんが、layer のインスタンスは2つの値を保持します。関数がこれらの値を必要とする場合は、セレクト式 (layer-color an-rd) と (layer-doll an-rd) を使用します。最後に、データ定義の2番目の節には、layer 構造体の doll フィールドから定義自体への自己参照が含まれ

ています。したがって、2 番目のセレクト式に対して再帰的な関数呼び出しが必要になります。

例とテンプレートが、関数定義をほぼ決定づけます。再帰のない `cond` 節については、答えは明らかに 1 です。再帰のある節については、テンプレートの式は以下の結果を計算します。

- `(layer-color an-rd)` は、現在のレイヤーの色を表す文字列を抽出します。
- `(layer-doll an-rd)` は、現在のレイヤーの内部に含まれる人形を抽出します。
- `(depth (layer-doll an-rd))` は、`depth` の目的ステートメントに従って、`(layer-doll an-rd)` に何個の人形が含まれているかを判定します。

この最後の数値は望ましい答えに極めて近いですが、完全に一致しているわけではありません。なぜなら、`an-rd` と `(layer-doll an-rd)` の差は 1 つのレイヤー、すなわち追加の 1 つの人形だからです。言い換えれば、関数は実際の答えを得るために、再帰的な結果に 1 を加算しなければなりません。

```
;; RD -> Number
;; an-rd に含まれる人形の個数を数える

(check-expect (depth "red") 1)
(check-expect (depth (make-layer "yellow" (make-layer "green" "red")))) 3)

(define (depth an-rd)
  (cond
    [(string? an-rd) 1]
    [else (+ (depth (layer-doll an-rd)) 1)]))
```

この関数定義が、2 番目の節で `(layer-color an-rd)` を使用していないことに注目してください。繰り返しますが、テンプレートはデータ定義について私たちが知っているすべての情報を整理するためのスキーマですが、実際の定義においてそれらすべてのパーツを必要とするわけではないことがわかります。

最後に、例をテストに変換しましょう。

```
(check-expect (depth "red") 1)
(check-expect (depth (make-layer "yellow" (make-layer "green" "red")))) 3)
```

これらを DrRacket で実行すると、それらの評価が `depth` の定義のすべての部分に触れることがわかります。

Exercise 154. `colors` 関数を設計しなさい。これはロシア人形を受け取り、すべての色をカンマとスペースで区切った文字列を生成します。したがって、私たちの例では次のように生成されます。

```
"yellow, green, red"
```

Exercise 155. `inner` 関数を設計しなさい。これは `rd` を受け取り、最も内側の人形の (色) を生成します。DrRacket のステップ実行を使用して、お好みの `rd` に対して (`inner rd`) を評価しなさい。

9.5 リストと World (Lists and World)

リスト、および自己参照データ定義全般を使用すると、有限データを使用する場合よりも、はるかに多くの興味深い World プログラムを設計して実行することができます。「項目化と構造体」のスペースインベーダープログラムで、プレイヤーが戦車から好きなだけ多くのショットを発射できるバージョンを作成することを想像してみてください。この問題の単純化されたバージョンから始めましょう。(※しばらく World プログラムを設計していない場合は、「World プログラムの設計」を読み返してください。)

Sample Problem ショットの発射をシミュレートする World プログラムを設計しなさい。「プレイヤー」がスペースキーを押すたびに、プログラムはキャンバスの底にショットを追加します。これらのショットは、クロックの1チックごとに1ピクセルの割合で垂直に上昇します。

World プログラムの設計は、情報を定数と、絶えず変化する世界の状態の要素へと分離することから始まります。前者のために物理定数と画像定数を導入し、後者のために世界の状態のデータ表現を開発する必要があります。サンプル問題は具体的な仕様については比較的曖昧ですが、ショットが垂直線に沿って描画される長方形の風景を明確に想定しています。明らかにショットの位置はクロックのチックごとに変化しますが、風景のサイズとショットの列の `x` 座標は同じままです。

```
(define HEIGHT 80) ; distance in terms of pixels
(define WIDTH 100)
(define XSHOTS (/ WIDTH 2))

;; graphical constants
(define BACKGROUND (empty-scene WIDTH HEIGHT))
(define SHOT (triangle 3 "solid" "red"))
```

問題文のどこにもこれらの具体的な選択を要求するものではありませんが、これらが簡単に変更できる (つまり、1つの定義を編集するだけで変更できる) 限り、私たちの目標は達成されています。

「世界」の変化する側面について、問題文は2つ言及しています。第一に、スペースキーを押すとショットが追加されます。第二に、すべてのショットはクロックのチックごとに1ピクセルずつ真上に移動します。プレイヤーが何発のショットを「発射」するかを予測することはできないため、これらを表現するためにリストを使用します。

```
;; A Shot is a Number.
;; interpretation: ショットの y 座標を表す
```

```
;; A List-of-shots is one of:
;; - '()
;; - (cons Shot List-of-shots)
;; interpretation: 発射されたショットのコレクション
```

すべてのショットが同じ x 座標を持ち、この座標が終始同じままであることはすでにわかっています。さらに、すべてのショットは同じ見たいをしています。したがって、それらの y 座標だけが、互いに異なる唯一のプロパティです。したがって、各ショットを数値として表現するだけで十分です。

ショットの表現を HEIGHT 未満の数値の区間に制限することもできます。なぜなら、すべてのショットはキャンバスの底から発射され、その後上へと移動するため、それらの y 座標は連続的に減少することがわかっているからです。

この世界を表現するために、以下のようなデータ定義を使用することもできます。

```
;; A ShotWorld is List-of-numbers.
;; interpretation: このようなリストの各数値はショットの  $y$  座標を表す
```

結局のところ、上記の2つの定義はすべての数値のリストを記述しています。私たちはすでに数値のリストの定義を持っており、ShotWorld という名前は、このデータクラスが何に関するものであるかを皆に伝えています。

定数を定義し、世界の状態を表すデータ表現を開発した後の主要なタスクは、どのイベントハンドラを採用するかを選択し、それらのシグネチャを特定の問題に適応させることです。この進行中の例ではクロックのチックとスペースキーについて言及されており、これは3つの関数のウィッシュリスト (wish list) に翻訳されます。

- 世界の状態を画像に変換する関数：

```
;; ShotWorld -> Image
;; w に含まれる各 y に対して、(XSHOTS, y) の位置にショットの画像を追加して BACKGROUND に描画する
(define (to-image w)
  BACKGROUND)
```

これは問題が視覚的な描画を要求しているためです。

- チックイベントを処理するための関数：

```
;; ShotWorld -> ShotWorld
;; w に含まれる各ショットを1ピクセル上に移動する
(define (tock w)
  w)
```

- キーイベントを処理するための関数：

```
;; ShotWorld KeyEvent -> ShotWorld
;; プレイヤーがスペースキーを押した場合に世界にショットを追加する
(define (keyh w ke)
  w)
```

初期のウィッシュリストに加えて、実際に世界を設定してハンドラをインストールする main 関数を定義する必要があることも忘れないでください。図 61 は、設計されるのではなく標準的なスキーマの修正として定義される、この 1 つの関数を含んでいます。

まず to-image の設計から始めましょう。シグネチャ、目的ステートメント、およびヘッダーが揃っているため、次に例が必要です。データ定義には 2 つの節があるため、少なくとも 2 つの例が必要です: '()' と、例えば (cons 9 '()) のような cons されたリストです。'()' に対する期待される結果は明らかに BACKGROUND です。しかし、y 座標がある場合、関数は XSHOTS と指定された座標にショットの画像を配置しなければなりません。

```
(check-expect (to-image '()) BACKGROUND)
(check-expect (to-image (cons 9 '()))
               (place-image SHOT XSHOTS 9 BACKGROUND))
```

読み進める前に、2 つの Shot のリストに to-image を適用する例に取り組んでみてください。そうすることで、関数がどのように動作するかが理解しやすくなります。

第 4 のステップは、データ定義をテンプレートに翻訳することです。

```
(define (to-image w)
  (cond
    [(empty? w) ...]
    [else
     (... (first w) ... (to-image (rest w)) ...)]))
```

リストのデータ定義に対するテンプレートは今や非常に馴染み深いものであるため、多くの説明は不要でしょう。疑念がある場合は、図 52 の質問を読み返し、自分でテンプレートを設計してください。

ここから関数を定義するのは簡単です。鍵となるのは、例とテンプレートを組み合わせ、図 53 の質問に答えることです。それらに従って、空のショットリストという基本ケースから始め、例から、期待される答えが BACKGROUND であることがわかります。次に、2 番目の cond 節のテンプレート式が何を計算するかを定式化します。

- (first w) はリストから最初の座標を抽出します。
- (rest w) は残りの座標です。
- (to-image (rest w)) は、to-image の目的ステートメントに従って、(rest w) 内の各ショットを背景画像に追加します。

言い換えれば、(to-image (rest w)) はリストの残りの部分を画像として描画するため、ほとんどすべての作業を実行します。不足しているのは最初のショット (first w) です。これら 2 つの式に目的ステートメントを適用すると、2 番目の cond 節の望ましい式が得られます。

```
(place-image SHOT XSHOTS (first w) (to-image (rest w)))
```


追加されるアイコンはショットの標準画像です。2つの座標は目的ステートメントで詳しく説明されています。そして、`place-image` の最後の引数は、リストの残りの部分から構築された画像です。

図 61 は、`to-image` の完全な関数定義と、プログラムの残りの部分を示しています。`tock` の設計は `to-image` の設計とまったく同じであり、自分で取り組んでみてください。ただし、`keyh` ハンドラのシグネチャは1つの興味深い疑問を投げかけます。これは、ハンドラが非自明なデータ定義を持つ2つの入力を受け取ることを指定しています。一方では、`ShotWorld` は自己参照的なデータ定義です。他方では、`KeyEvent` の定義は大きな列挙です。ここでは、2つの引数のどちらがテンプレートの開発を駆動すべきかを「推測」してもらい、このようなケースについては後で詳しく学習することにします。

図 61: リストに基づく World プログラム (A list-based world program)

```
;; ShotWorld -> ShotWorld
(define (main w0)
  (big-bang w0
    [on-tick tock]
    [on-key keyh]
    [to-draw to-image]))

;; ShotWorld -> ShotWorld
;; 各ショットを1ピクセル上に移動する
(define (tock w)
  (cond
    [(empty? w) '()]
    [else (cons (sub1 (first w)) (tock (rest w))))]))

;; ShotWorld KeyEvent -> ShotWorld
;; スペースキーが押された場合に世界にショットを追加する
(define (keyh w ke)
  (if (key=? ke " ")
      (cons HEIGHT w)
      w))

;; ShotWorld -> Image
;; w に含まれる各ショット y を (XSHOTS, y) に配置して BACKGROUND に追加する
(define (to-image w)
  (cond
    [(empty? w) BACKGROUND]
    [else (place-image SHOT XSHOTS (first w)
      (to-image (rest w)))]))
```

World プログラムに関する限り、`keyh` のようなキーハンドラはそれが受け取るキーイベントに関するものです。したがって、私たちはそれを主要な引数と考え、そのデータ定義を使用してテンプレートを導出します。具体的には、「列挙」の `KeyEvent` のデータ定義に従って、関数には以下のような多数の節を持つ `cond` 式が必要であると指示されます。

```
(define (keyh w ke)
  (cond
    [(key=? ke "left") ...]
    [(key=? ke "right") ...]
    ...
    [(key=? ke " ") ...]
    ...
    [(key=? ke "a") ...]
    ...
    [(key=? ke "z") ...]))
```

もちろん、すべての可能な BSL 値を受け取る関数と同様に、キーハンドラは通常、すべての可能なケースを検査する必要はありません。私たちの進行中の問題については、キーハンドラがスペースキーにのみ反応し、他のすべてのキーは無視されることを具体的に知っています。そのため、" " の節を除き、すべての cond 節を else 節に折りたたむのが自然です。

Exercise 156. 図 61 のプログラムにテストを追加し、それらのテストに合格することを確認しなさい。main が何を行うか説明しなさい。その後、main を介してプログラムを実行しなさい。

Exercise 157. 定数に関する恣意的な決定が簡単に変更できるかどうかを確認するために実験しなさい。例えば、1つの定数定義を変更するだけで、望ましい結果が得られるかどうかを確認しなさい。

- change the height of the canvas to 220 pixels;
- change the width of the canvas to 30 pixels;
- change the x location of the line of shots to “somewhere to the left of the middle”;
- change the background to a green rectangle; and
- change the rendering of shots to a red elongated rectangle.

また、他のものを変更せずにショットのサイズを2倍にできるか、あるいは色を黒に変更できるかを確認しなさい。

Exercise 158. main を実行し、スペースキーを押し（ショットを発射し）、かなりの時間待つと、ショットはキャンバスから消えます。しかし、世界のキャンバスを閉じると、結果はまだこの見えないショットを含んでいる世界になります。

ショットをチックごとに1ピクセル移動するだけでなく、キャンバスの上方に位置する座標を持つショットを排除する、代替の tock 関数を設計しなさい。ヒント：再帰的な cond 節のために、補助関数の設計を検討するとよいでしょう。

Exercise 159. Exercise 153 の解決策を World プログラムに変換しなさい。そのメイン関数である riot は、学生が投げたい風船の数を受け取ります。その視覚化は、1秒間

に1個の割合で次々と落下する風船を表示します。関数は、風船が当たった Posn のリストを生成します。

ヒント：

(1) 以下は1つの可能なデータ表現です。

```
(define-struct pair [balloon# lob])  
;; A Pair is a structure (make-pair N List-of-posns)  
  
;; A List-of-posns is one of:  
;; - '()  
;; - (cons Posn List-of-posns)  
;; interpretation: (make-pair n lob) は、n 個の風船が  
;; まだ投げられておらず、lob に追加される必要があることを意味する
```

(2) big-bang 式は実際には単なる式です。それを別の式の中に入れ子にすることは合法です。

(3) random がランダムな数を作成することを思い出さない。

9.6 リストと集合についての注意 (A Note on Lists and Sets)

本書は、BSLの値のコレクションとしての集合 (sets) に関する、あなたの直感的な理解に依存しています。「データの宇宙」では、データ定義が BSL の値の集合の名前を導入すると明確に述べています。本書が集合に関して一貫して問いかける質問が1つあり、それは「ある要素が与えられた集合に含まれているかどうか」です。例えば、4 は Number に含まれますが、"four" は含まれません。本書では、ある値が名前付きの集合のメンバーであるかどうかを確認するためにデータ定義を使用する方法や、いくつかのデータ定義を使用して集合のサンプル要素を生成する方法も示していますが、これら2つの手順はデータ定義に関するものであり、集合そのものに関するものではありません。

同時に、リストは値のコレクションを表します。そのため、リストと集合の違いは何であるのか、あるいはこれは不要な区別であるのかと疑問に思うかもしれません。そうであるなら、このセクションはあなたのためのものです。

現時点での集合とリストの主な違いは、前者はコードの設計手順を議論するために使用する概念であり、後者は選定したプログラミング言語である BSL の多くのデータ形式の1つであるということです。この2つのアイデアは、私たちの会話においてかなり異なるレベルに位置しています。しかし、データ定義が BSL の内部で実際の情報のデータ表現を導入すること、および集合が情報のコレクションであることを踏まえると、集合が BSL の内部でデータとしてどのように表現されるのかを自問するかもしれません。

ほとんどの本格的な言語は、リストと集合の両方のデータ表現を直接サポートしています。

リストは BSL において特別な地位を持っていますが、集合にはそれがありません。しかし同時に、集合はどこかリストに似ています。主な違いは、プログラムが通常どちらのデー

タ形式で使用する関数の種類にあります。BSL は、リストに関するいくつかの基本的な定数と関数（例えば `empty`、`empty?`、`cons`、`cons?`、`first`、`rest`）や、自分で定義できるいくつかの関数（例えば `member?`、`length`、`remove`、`reverse` など）を提供しています。以下は、自分で定義することはできますが、BSL には付属していない関数の例です。

```
;; List-of-string String -> N
;; los 内に s が出現する回数を判定する
(define (count los s)
  0)
```

ちょっと待ってください！この関数の設計を完成させなさい。

簡単で、おそらく素朴な方法で進め、集合は基本的にはリストであるとしましょう。And、さらに簡単にするために、このセクションでは数値のリストに焦点を当てます。ある数値が集合に含まれるかどうかが重要であると認めるならば、リストを使用して集合を表現する方法には2つの異なるアプローチがあることが、ほぼ即座に明らかになります。

図 62: 集合の2つのデータ表現 (Two data representations for sets) | Son.L (左側の表現) | Son.R (右側の表現) | | :- | :- | | ;; A Son.L is one of: | ;; A Son.R is one of: | | ;; - empty | ;; - empty | | ;; - (cons Number Son.L) | ;; - (cons Number Son.R) | | | ;; Constraint: If s is a Son.R, | | ;; Son is used when it applies | ;; no number occurs twice in s | | ;; to Son.L and Son.R | |

図 62 は、これら2つのデータ定義を示しています。どちらも基本的には、集合が数値のリストとして表現されることを述べています。違いは、右側の定義には、リスト上で同じ数値が2回以上出現してはならないという制約 (Constraint) が伴っている点です。結局のところ、集合に関して私たちが問いかける主要な質問は「ある数値が集合に含まれているかどうか」であり、それが集合の中に1回、2回、あるいは3回含まれているかどうかは違いを生じさせないからです。

どちらの定義を選択するかにかかわらず、2つの重要概念をすでに定義できます。

```
;; Son
(define es '())

;; Number Son -> Boolean
;; x は s に含まれるか
(define (in? x s)
  (member? x s))
```

最初のものは空集合 (`empty set`)であり、どちらのケースでも空リストによって表現されます。2番目のものはメンバーシップテスト (所属判定) です。

より大きな集合を構築する1つの方法は、`cons` と上記の定義を使用することです。たとえば、1、2、3 を含む集合の表現を構築したいとします。そのような表現の1つは次の通りです。

```
(cons 1 (cons 2 (cons 3 '())))
```

And それは両方のデータ表現で機能します。But、

```
(cons 2 (cons 1 (cons 3 '())))
```

は本当に同じ集合の表現ではないのでしょうか？あるいは、

```
(cons 1 (cons 2 (cons 1 (cons 3 '()))))
```

はどうでしょうか？

主要な関心が「数値が集合に含まれているかどうか」である限り、その答えは肯定的でなければなりません。それでも、cons の順序は重要になり得ませんが、右側のデータ定義における制約は、最後のリストを Son.R とすることを排除します。なぜなら、それが 1 を 2 回含んでいるからです。

図 63: 集合の 2 つのデータ表現のための関数 (Functions for the two data representations of sets) | Son.L のための関数とテスト | Son.R のための関数とテスト | | :
- | :- | | ;; Son.L | ;; Son.R | | (define s1.L (cons 1 (cons 1 '()))) | (define
s1.R (cons 1 '())) | | | | ;; Number Son.L -> Son.L | ;; Number Son.R -> Son.R
| | ;; s から x を削除する | ;; s から x を削除する | | (check-expect (set-.L 1 s1.L)
es) | (check-expect (set-.R 1 s1.R) es) | | | | (define (set-.L x s) | (define
(set-.R x s) | | (remove-all x s)) | (remove x s)) |

2 つのデータ定義の違いは、関数を設計する際に現れます。集合から数値を削除する関数が必要だとします。以下は、両方の表現に適用されるウィッシュリストの項目です。

```
;; Number Son -> Son  
;; s から x を差し引く (subtracts)  
(define (set- x s)  
  s)
```

目的ステートメントで “subtract” (差し引く) という言葉が使用されているのは、論理学者や数学者が集合を扱う際にこの言葉を使用するからです。

図 63 はその結果を示しています。2 つの列は 2 つの点で異なります。

1. 左側のテストは 1 を 2 回含むリストを使用していますが、右側のテストは同じ集合を 1 つの cons で表現しています。
2. これらの違いにより、左側の set- は remove-all を使用しなければなりませんが、右側のものは remove で済みます。

ちょっと待ってください！コードを DrRacket の定義領域にコピーし、テストが合格することを確認しなさい。その後、読み進めながらコードを使って実験しなさい。

図 63 の好ましくない側面は、テストが期待される結果としてプレーンなリストである es を使用していることです。この問題は一見すると些細なことのように思えるかもしれませんが。しかし、次の例を考えてみましょう。

```
(set- 1 set123)
```

ここで `set123` は、1、2、3 を含む集合を、次の2つの方法のいずれかで表現しています。

```
(define set123-version1
  (cons 1 (cons 2 (cons 3 '()))))
```

```
(define set123-version2
  (cons 1 (cons 3 (cons 2 '()))))
```

Regardless of どちらの表現を選択するかにかかわらず、`(set- 1 set123)` は次の2つのリストのいずれかに評価されます。

```
(define set23-version1
  (cons 2 (cons 3 '())))
```

```
(define set23-version2
  (cons 3 (cons 2 '())))
```

But 私たちはこれら2つのうちどちらを `set-` が生成するかを予測することはできません。

2つの代替案という単純なケースについては、次のように `check-member-of` テスト機能を使用することが可能です。

```
(check-member-of (set-.v1 1 set123.v1)
                  set23-version1
                  set23-version2)
```

もし期待される集合が3つの要素を含んでいる場合、左側のデータ定義が許容する重複のある表現を除外したとしても、6つの可能なバリエーションが存在します。

この問題を解決するには、2つのアイデアを組み合わせる必要があります。第一に、`set-` が本質的に「与えられた要素が結果に出現しないことを保証する」ことに関するものであることを思い出してください。これは、例をテストに変換する私たちの方法では伝わらないアイデアです。第二に、BSL の `check-satisfied` テスト機能を使用すれば、このアイデアを正確に述べることができます。

「インターメッツォ1: Beginning Student Language」で `check-satisfied` について簡単に言及されていますが、簡単に言うと、この機能はある式が特定のプロパティを満たしているかどうかを判定します。プロパティとは、値から `Boolean` への関数です。私たちの具体的なケースでは、1 が集合のメンバーではないことを正確に述べたいと考えます。

```
;; Son -> Boolean
;; 1 が s のメンバーでなければ #true、そうでなければ #false
(define (not-member-1? s)
  (not (in? 1 s)))
```

`not-member-1?` を使用すると、テストケースを次のように定式化できます。

```
(check-satisfied (set- 1 set123) not-member-1?)
```

and このバリエーションは関数が何を達成すべきかを明確に述べています。さらに良いことに、この定式化は入力または出力の集合がどのように表現されているかにまったく依存しません。

要約すると、リストと集合はどちらも値のコレクションに関するものであるという点で関連していますが、大きく異なります。

特性 (property)	リスト (lists)	集合 (sets)
メンバーシップ (membership)	多くのうちの1つ	決定的 (critical)
順序 (ordering)	決定的 (critical)	無関係 (irrelevant)
出現回数 (# of occurrences)	意味を持つ (sensible)	無関係 (irrelevant)
サイズ (size)	有限だが任意	有限または無限

The 最後の行は、明白なことではありますが、新しいアイデアを提示しています。本書で言及されている集合の多く（例えば Number、String、さらには List-of-strings）は無限に大きいです。対照的に、リストは任意の数の項目を含むことができますが、常に有限です。

要約すると、このセクションでは、集合とリストの決定的な違いと、有限リストを用いて有限集合を2つの異なる方法で表現する方法について説明しました。BSL は無限集合を表現するのに十分な表現力を持っていません。練習問題 299 では、無限集合にも対処できる、まったく異なる集合の表現を導入します。ただし、実際のプログラミング言語がどのように集合を表現するかという問題は、本書の範囲を超えています。

Exercise 160. Son.L と Son.R のデータ定義に対して、それぞれ集合 *s* に数値 *x* を追加して集合を作成する関数 *set+.L* と *set+.R* を設計しなさい。

10 リストの更なる世界 (More on Lists)

リストは、現在ほとんどすべての言語に備わっている、非常に汎用性の高いデータ形式です。プログラマーはリストを使って、大規模なアプリケーション、人工知能、分散システムなどを構築してきました。この章では、この世界におけるいくつかのアイデアを説明します。リストを生成する関数、リストの中に構造体を必要とするデータ表現、そしてテキストファイルをリストとして表現することなどが含まれます。

10.1 リストを生成する関数 (Functions that Produce Lists)

以下は、時給制の従業員の賃金を計算する関数です。

```
; Number -> Number
; 労働時間 h に対する賃金を計算する
```

```
(define (wage h)
  (* 12 h))
```

この関数は労働時間数を受け取り、賃金を算出します。しかし、給与計算ソフトウェアを必要としている会社は、この関数単体には興味がありません。会社が欲しいのは、すべての従業員の賃金を計算する関数です。

この新しい関数を `wage*` と呼びましょう。その役割は、すべての従業員の労働時間を処理し、それぞれに支払うべき賃金を決定することです。簡単にするため、入力はい数値のリスト（各数値は1人の従業員が働いた時間数を表す）であり、期待される結果も週給のリスト（これも数値のリストで表される）であると仮定しましょう。

入力と出力に対するデータ定義はすでに持っているのです、すぐに設計の第2ステップに進むことができます。

```
; List-of-numbers -> List-of-numbers
; 週ごとの労働時間リストから週給のリストを計算する
(define (wage* whrs)
  '())
```

次に、入力の例とそれに対応する出力の例が必要です。そこで、週の労働時間を表す短い数値リストをいくつか作ってみましょう。

入力 (given)	期待される出力 (expected)
'()	'()
(cons 28 '())	(cons 336 '())
(cons 4 (cons 2 '()))	(cons 48 (cons 24 '()))

出力を計算するために、与えられた入力リストの各数値に対して週の賃金を決定します。最初の例では、入力リストに数値がないので、出力は `'()` です。2番目と3番目の期待される出力がなぜそうなるのか、理解するようにしてください。

`wage*` は「リスト」の章の他のいくつかの関数と同じ種類のデータを消費し、テンプレートはデータ定義の形にのみ依存するため、このテンプレートを再利用できます。

```
(define (wage* whrs)
  (cond
    [(empty? whrs) ...]
    [else (... (first whrs) ...
                ... (wage* (rest whrs)) ...)]))
```

テンプレートの作成を練習したい場合は、図 52 の質問を使用してください。

これで最も創造的な設計ステップに進む準備ができました。設計レシピに従って、テンプレートの各 `cond` 行を個別に考えます。再帰しないケースでは、`(empty? whrs)` が真であり、入力は `'()` であることを意味します。上記の例から、望ましい答えは `'()` であると指定されているので、これで完了です。

2 番目のケースでは、設計の質問が、テンプレートの各式が何を計算するかを述べるよう指示しています。 - (first whrs) は whrs の最初の数値を返します。これは最初の労働時間数です。 - (rest whrs) は与えられたリストの残りの部分です。 - (wage* (rest whrs)) は、残りの部分が私たちが今定義している関数自体によって処理されることを意味します。いつものように、この式のシグネチャと目的ステートメントを使って結果がどうなるかを考えます。シグネチャからは数値のリストであることがわかり、目的ステートメントからは、このリストが残りの時間リストに対する賃金のリストを表していることがわかります。

重要なのは、関数がまだ完全に定義されていなくても、このケースで結果を計算する式を定式化するとき、これらの事実を頼りにすることです。

whrs の最初の項目を除くすべての要素に対する賃金のリストはすでに得られているので、関数は whrs 全体に対する期待される出力を生成するために2つの計算を行う必要があります。(first whrs) の週賃金を計算し、さらに whrs のすべての週賃金を表すリストを構築することです。最初の部分には wage を再利用します。2 番目の部分には、2 つの情報を cons で1つのリストにまとめます。

```
(cons (wage (first whrs)) (wage* (rest whrs)))
```

これで定義が完成しました。図 64 を参照してください。

図 64: すべての従業員の賃金を計算する

```
; List-of-numbers -> List-of-numbers
; 与えられたすべての週労働時間から週給を計算する
(define (wage* whrs)
  (cond
    [(empty? whrs) '()]
    [else (cons (wage (first whrs))
                 (wage* (rest whrs)))])])

; Number -> Number
; 労働時間 h に対する賃金を計算する
(define (wage h)
  (* 12 h))
```

練習問題 161. これらの例をテストに変換し、すべて成功することを確認してください。次に、図 64 の関数を変更して、全員が時給 14 ドルを受け取るようにしてください。さらに、プログラム全体を改訂して、全員の時給を変更することがプログラム全体の複数箇所ではなく、単一箇所の変更だけで済むようにしてください。

練習問題 162. 週に 100 時間以上働く従業員はあり得ません。会社を不正から守るために、関数は wage* の入力リストのどの項目も 100 を超えないことを確認する必要があります。もし超えるものがあれば、関数は即座にエラーをシグナルする必要があります。この基本的な現実確認を実行したい場合、図 64 の関数をどのように変更すればよいでしょうか？

練習問題 163. `convertFC` を設計してください。この関数は、華氏の測定値リストを摂氏の測定値リストに変換します。

練習問題 164. US ドルの金額リストをユーロ (€) の金額リストに変換する関数 `convert-euro` を設計してください。ウェブで現在の為替レートを調べてください。 `convert-euro` を関数 `convert-euro*` に一般化してください。この関数は為替レートと US ドルの金額リストを受け取り、後者をユーロの金額リストに変換します。

練習問題 165. おもちゃの説明 (1 単語の文字列) のリストを受け取り、出現するすべての "robot" を "r2d2" に置き換える関数 `subst-robot` を設計してください。それ以外の説明はそのまま残します。 `subst-robot` を `substitute` に一般化してください。後者は `new` と `old` と呼ばれる 2 つの文字列と、文字列のリストを受け取ります。そして、`old` のすべての出現を `new` に置き換えた新しい文字列のリストを生成します。

10.2 リスト内の構造体 (Structures in Lists)

週の労働時間を単なる数値として表すのは、あまり良い選択ではありません。なぜなら、給与小切手の印刷には、週あたりの労働時間以上の情報が必要だからです。また、すべての従業員が同じ時給であるとは限りません。幸いなことに、リストにはアトムックな値以外の項目を含めることができます。実際、リストには必要なあらゆる値、特に構造体を含めることができます。

私たちの進行中の例では、まさにそのようなデータ表現が必要です。数値の代わりに、従業員と彼らの労働時間および賃金率を表す構造体を使用します。

```
(define-struct work [employee rate hours])  
; A Work is a structure:  
;   (make-work String Number Number)  
; 解釈: (make-work n r h) は従業員名 n、時給率 r、労働時間 h を組み合わせる。
```

この表現はまだ単純ですが、構造体を含むリストのデータ定義を定式化する必要があるため、追加の課題として十分です。

```
; Low (list of works の略) は以下のいずれか:  
; -- '()  
; -- (cons Work Low)  
; 解釈: Low のインスタンスは、複数の従業員が働いた時間を表す。
```

以下は `Low` の 3 つの要素の例です。

```
'()  
(cons (make-work "Robby" 11.95 39)  
      '())  
(cons (make-work "Matthew" 12.95 45)  
      (cons (make-work "Robby" 11.95 39)  
            '()))
```


データ定義を使って、これらのデータがなぜ Low に属するのか説明してください。 ストップ! また、データ定義を使って、さらに 2 つの例を作成してください。

実際のプロジェクトで作業するときは、このようなサフィックス (.v2 など) は使用せず、代わりに異なるバージョンのコードを管理するためのツールを使用します。

Low の定義が理にかなっていることが分かったので、単なる数値リストではなく、Low の要素を消費するように関数 wage* を再設計する時が来ました。

```
; Low -> List-of-numbers
; 与えられたレコードから週給を計算する
(define (wage*.v2 an-low)
  '())
```

関数名の最後にあるサフィックス “.v2” は、これが関数の改訂された第 2 バージョンであることを読者に伝えます。この場合、改訂は新しいシグネチャと適応された目的ステートメントから始まります。ヘッダーは上記と同じです。

設計レシピの第 3 ステップは、例を作成することです。上記の 2 番目のリストから始めましょう。これには 1 つの作業レコード、すなわち (make-work "Robby" 11.95 39) が含まれています。その解釈は、“Robby” が 39 時間働き、時給 11.95 ドル支払われるということです。したがって、彼のその週の賃金は (* 11.95 39)、つまり 466.05 ドルです。したがって、wage*.v2 に対する期待される結果は (cons 466.05 '()) です。当然ながら、入力リストに 2 つの作業レコードが含まれている場合、この種の計算を 2 回実行し、結果は 2 つの数値のリストになります。ストップ! 上記の 3 番目のデータ例に対する期待される結果を決定してください。

数値に関する注意 BSL は、他のほとんどのプログラミング言語とは異なり、小数を正確な分数として理解することに注意してください。例えば Java のような言語では、最初の作業レコードの期待される賃金に対して 466.04999999999995 を生成します。小数を扱う演算がこのような奇妙な動作をするタイミングを予測することはできないため、他のプログラミング言語に備えるためにも、以下のように例を記述しておくのが得策です。

```
(check-expect
  (wage*.v2 (cons (make-work "Robby" 11.95 39) '()))
  (cons (* 11.95 39) '()))
```

このように例を書き留めることは、実際に賃金を計算する方法を本当に理解したことを意味します。[注意終わり]

ここからテンプレートの開発に移ります。テンプレートに関する質問を使用すると、すぐに次のようになります。

```
(define (wage*.v2 an-low)
  (cond
    [(empty? an-low) ...]
```

```
[(cons? an-low)
 (... (first an-low) ...
 (... (wage*.v2 (rest an-low)) ...))])
```

データ定義が2つの節で構成されており、最初の節で '()', 2番目の節で cons された構造体を導入しているためです。しかし、入力についてこのテンプレートが表現している以上のことが分かっていることにも気づくでしょう。例えば、(first an-low) は与えられたリストから3つのフィールドを持つ構造体を取り出します。これは、テンプレートにさらに3つの式を追加することを示唆しているようです。

```
(define (wage*.v2 an-low)
  (cond
    [(empty? an-low) ...]
    [(cons? an-low)
     (... (first an-low) ...
          ... (work-employee (first an-low)) ...
          ... (work-rate (first an-low)) ...
          ... (work-hours (first an-low)) ...
          ... (wage*.v2 (rest an-low)) ...)])
```

この詳細なテンプレートは、2つの指針を組み合わせた結果です。入力データに自己参照リストが含まれているため、テンプレートには自己参照関数呼び出しが含まれます。また、リストには構造体が含まれているため、テンプレートには構造体からフィールド値を抽出するためのセレクトタ式が含まれます。

これで最も創造的なステップの準備が整いました。最初の cond 節は、空リスト '()' に対するものであり、答えは '()' です。

2番目の節については、テンプレートに用意されている構成要素が何を計算するかを書き留めます。 - (first an-low) はリスト an-low の最初の Work 構造体です。 - (work-employee (first an-low)) は最初の従業員の名前です。 - (work-rate (first an-low)) は最初の従業員の時給です。 - (work-hours (first an-low)) は最初の従業員の労働時間です。 - (wage*.v2 (rest an-low)) は、リストの残りの作業レコードに対する賃金のリストを計算します。

残りのレコードに対する賃金リストはすでに得られているので、必要なのは最初の作業レコードの賃金を計算し、それを再帰の結果に cons することだけです。最初の作業レコードの賃金は、その時給に労働時間を掛けることで計算できます。

```
(* (work-rate (first an-low)) (work-hours (first an-low)))
```

これら2つの値を cons すると、完全な関数定義が得られます。図65を参照してください。

図 65: 従業員レコードのリストから賃金を計算する

```
(define-struct work [employee rate hours])
; A Work is a structure:
;   (make-work String Number Number)
```

; 解釈: (make-work n r h) は従業員名、時給率 r、労働時間 h を組み合わせる

; Low (list of works の略) は以下のいずれか:
; -- '()
; -- (cons Work Low)
; 解釈: Low のインスタンスは、複数の従業員が働いた時間を表す

; Low -> List-of-numbers
; 与えられたすべての週労働時間から週給を計算する
(define (wage*.v2 an-low)
 (cond
 [(empty? an-low) '()]
 [else (cons (* (work-rate (first an-low))
 (work-hours (first an-low)))
 (wage*.v2 (rest an-low)))]))

この定義は完璧ですが、設計上のアドバイスの最初の部分に少し違反しています。それは、1つの関数に1つのタスクを持たせるという点です。ここでの2番目の cond 節は、作業レコードを処理してその週の賃金を計算することと、その賃金を残りの賃金リストと組み合わせることの2つを行っています。これらのタスクを分離するために、1つの作業レコードを受け取ってその賃金を計算するヘルパー関数を「願望 (wish)」します。

; Work -> Number
; 与えられた作業レコード w に対する賃金を計算する
(define (wage.v2 w)
 (* (work-rate w) (work-hours w)))

このヘルパー関数を使用すると、wage*.v2 の定義は非常に簡潔になり、設計も分かりやすくなります。

(define (wage*.v2 an-low)
 (cond
 [(empty? an-low) '()]
 [else (cons (wage.v2 (first an-low))
 (wage*.v2 (rest an-low)))]))

このようにプログラムを補助関数に分けることで、より複雑なデータ表現が必要になったとき（例えば、残業手当の計算方法の追加など）の拡張が非常に容易になります。

練習問題 166. 時給と従業員名のリストに加えて、小切手を郵送するために各従業員の従業員番号も保持する必要があるとします。Work 構造体の代わりに、work レコードと従業員番号 (Number) を組み合わせた Employee 構造体を使用するプログラムを設計してください。あるいは、Work 構造体の定義を変更して、従業員名の代わりに新しい Employee 構造体 (名前と従業員番号を含む) を使用することもできます。これら2つの選択肢についてデータ定義を作成し、それぞれの利点と欠点について議論してください。第2の選択肢に基づいて、wage*.v3 を設計してください。この関数は、給与情報を表す構造体のリストを受け取り、名前と週給を含む小切手 (payment 構造体のリスト) を生成します。

練習問題 167. ポインツ (Posn 構造体) のリストにあるすべての X 座標の合計を計算する関数 `sum` を設計してください。

練習問題 168. Posn 構造体のリストを受け取り、すべての Posn の Y 座標に 1 を加えた新しい Posn のリストを生成する関数 `translate` を設計してください。

練習問題 169. Posn 構造体のリストを受け取り、X 座標が 0 から 100 の間、かつ Y 座標が 0 から 200 の間にある Posn のみを残した新しいリストを生成する関数 `legal` を設計してください。

練習問題 170. トイ・レコードのリストから電話番号を更新するプログラムを設計してください。おもちゃ屋の電話帳は、次の構造体のリストです。

```
(define-struct phone [name number])  
; A Phone is a structure:  
;   (make-phone String String)  
; 解釈: (make-phone n num) は名前 n と電話番号 num を関連付ける。
```

電話番号は文字列として表されます (例: "281-582-3652")。あなたのアシスタントは "713" 区域コードのすべての局番号 "701" を "712" に変更したいと考えています。つまり、電話番号が "713-701-XXXX" であれば "713-712-XXXX" に変更されます。この変更を自動的に行う関数 `replace` を設計してください。この関数は、電話帳 (Phone 構造体のリスト) を受け取り、すべての局番号 "701" が "712" に置き換えられた新しい電話帳を生成します。

10.3 リストのリスト、およびテキストファイル (Lists in Lists, and Text Files)

「リストのリスト」という考え方は、最初は恐ろしく聞こえるかもしれませんが、心配する必要はありません。設計レシピは、このような複雑な場合でも有効に機能します。

ファイルの内容をリストとして表すことを考えてみましょう。これはリストのリストを使用する典型的な例です。たとえば、ファイルを「行のリスト」として読み込み、各行を「単語のリスト」として表す場合、それは「文字列のリストのリスト」になります。この表現は、ファイルの元の構造 (空行など) を完全に維持できるという利点があります。

図 67 は、ファイルからテキストを読み込むためのいくつかの便利な関数を示しています。

図 67: ファイルの読み込み

```
; String -> String  
; ファイル f の内容を単一の文字列として返す  
(define (read-file f) ...)  
  
; String -> List-of-strings  
; ファイル f の内容を行ごとの文字列 of リストとして返す  
(define (read-lines f) ...)
```

```

; String -> List-of-strings
; ファイル f の内容を単語ごとの文字列 of リストとして返す
(define (read-words f) ...)

; String -> List-of-list-of-strings
; ファイル f の内容を「行のリスト（各行は単語のリスト）」として返す
(define (read-words/line f) ...)

```

これらの関数は、指定されたファイルの名前を文字列引数として受け取ります。指定されたファイルがプログラムと同じフォルダーに存在しない場合、エラーをシグナルします。

これらの関数はすべてのテキストファイルを処理するのに十分ではありませんが、私たちの目的であるシステム開発の原則を学ぶためには十分であり、面白いプログラムを設計する力を与えてくれます。

練習問題 171. List-of-strings のデータ定義がどのようなものか、あなたは知っています。それを書き出してください。また、Piet Hein の詩を、各行が文字列として表されるインスタンス、および各単語が文字列として表されるインスタンスとして表現できることを確認してください。read-lines と read-words を使用して、表現の選択を確認してください。

次に、List-of-list-of-strings のデータ定義を作成してください。同じ詩を、各行が単語（文字列）のリストとして表され、詩全体がそれら行のリストであるインスタンスとして表現してください。read-words/line を使用して確認できます。

オペレーティングシステムには、ファイルを測定するプログラム (Unix の wc コマンドなど) が付属しています。あるプログラムは行数を数え、別のプログラムは各行に現れる単語数を測定します。後者を例にして、設計レシピが複雑な関数の設計にどのように役立つかを見てみましょう。

最初のステップは、必要なすべてのデータ定義を用意することです。上記の練習問題で入力データの定義を作成しました。また、前セクションで出力データを表す List-of-numbers を定義しました。簡潔にするために、LN を「文字列のリストのリスト」のクラスを表す名前として使用し、目的の関数のヘッダーを記述します。

```

; LN -> List-of-numbers
; 各行にある単語の数を決定する
(define (words-on-line lls)
  '())

```

次に必要なのは、データ例の作成です。

```

(define line0 (cons "hello" (cons "world" '())))
(define line1 '())

(define lls0 '())
(define lls1 (cons line0 (cons line1 '())))

```

line0 は2つの単語を含み、line1 は単語を含いません。lls0 は行を含まず、lls1 はこれら2つの行を含んでいます。

これらのデータ例から、関数例を作成します。lls0 に words-on-line を適用すると、行がないため空リスト '()' が返されるはずです。lls1 に適用すると、2つの行があるため2つの数値のリスト (cons 2 (cons 0 '())) が返されるはずです。

これをテストケースに変換します。

```
(check-expect (words-on-line lls0) '())
(check-expect (words-on-line lls1) (cons 2 (cons 0 '())))
```

テンプレートの開発が、このサンプル問題の最も興味深いステップです。図 52 のテンプレートに関する質問に答えることで、おなじみのリスト処理テンプレートがすぐに得られます。

```
(define (words-on-line lls)
  (cond
    [(empty? lls) ...]
    [else (... (first lls) ...
               ... (words-on-line (rest lls)) ...)]))
```

前セクションと同様に、(first lls) は複雑な構造を持つ List-of-strings を返すことが分かっています。この知識を表現するために入れ子のテンプレートを挿入したくなりますが、より良いアプローチは、別個の補助テンプレート（ここでは line-processor）を用意し、最初のテンプレートからそれを参照することです。

補助テンプレートはリストを処理する関数のためのものなので、ほぼ同じに見えます。

```
(define (line-processor ln)
  (cond
    [(empty? ln) ...]
    [else (... (first ln) ...
               ... (line-processor (rest ln)) ...)]))
```

重要な違いは、(first ln) がリストからアトミックな値である文字列を抽出することです。このテンプレートを使用すると、words-on-line の第2のケースの最初の行を以下のように変更できます。

```
(... (line-processor (first lls)) ...
... (words-on-line (rest lls)) ...)
```

これは、第5のステップにおいて、words-on-line の定義が補助関数の設計を要求することを示唆しています。

さて、プログラミングの時間です。空リスト（行がない）に対する最初のケースは簡単で、答えは '()' です。2番目のケースにはいくつかの式があり、それぞれが何を計算するかを思い出します。- (first lls) は最初の行を取り出します。- (line-processor (first lls)) は、この行を処理するための補助関数を設計する可能性を示唆します。- (rest

l1s) は残りの行のリストです。 - (words-on-line (rest l1s)) は、残りの行に対する単語数のリストを計算します。

1つの行を受け取ってその中の単語数をカウントする補助関数 (words# と呼びましょう) を設計できると仮定すれば、2 番目のケースの本体は簡単に完成します。

```
(cons (words# (first l1s)) (words-on-line (rest l1s)))
```

この式は、最初の行の単語数を、残りの行の単語数を表すリストの先頭に cons します。

残るは words# 関数の設計です。そのテンプレートは line-processor であり、その目的はリスト内の文字列の数をカウントすることです。

```
; List-of-strings -> Number
; los に含まれる単語の数をカウントする
(define (words# los)
  0)
```

しかし、あなたは以前にこれとまったく同じ機能を持つ how-many 関数を設計したことを思い出すでしょう。既存の関数を再利用するのは良いことなので、以下のように定義できます。

```
(define (words# los)
  (how-many los))
```

実際には、BSL にはリストの要素数をカウントする標準の length 関数がすでに存在します。したがって、それを使用するのが最適です。完全な設計を図 68 にまとめます。

図 68: 各行の単語数をカウントする

```
; LN (list of lines の略) は以下のいずれか:
; -- '()
; -- (cons List-of-strings LN)
; 解釈: 行のリスト。各行は文字列 (単語) のリスト。

(define line0 (cons "hello" (cons "world" '())))
(define line1 '())
(define ln0 '())
(define ln1 (cons line0 (cons line1 '())))

; LN -> List-of-numbers
; 各行にある単語の数を決定する
(check-expect (words-on-line ln0) '())
(check-expect (words-on-line ln1) (cons 2 (cons 0 '())))

(define (words-on-line ln)
  (cond
    [(empty? ln) '()]
    [else (cons (length (first ln))
                  (words-on-line (rest ln)))]))
```


これで、最初のファイルユーティリティを設計する準備が整いました。

```
; String -> List-of-numbers
; 与えられたファイル内の各行の単語数をカウントする
(define (file-statistic file-name)
  (words-on-line (read-words/line file-name)))
```

これは、ティーチパックの関数と私たちが設計した `words-on-line` を組み合わせただけです。

練習問題 172. 行のリストを単一の文字列に変換する関数 `collapse` を設計してください。各単語はスペース (" ") で区切り、各行は改行文字 ("\n") で区切る必要があります。[挑戦] 完成したら、プログラムを以下のように使用して、Piet Hein の詩の元のファイルと同一のファイルが生成されることをテストしてください。

```
(write-file "ttt.dat"
  (collapse (read-words/line "ttt.txt")))
```

練習問題 173. テキストファイルからすべての冠詞 ("a"、"an"、"the") を削除するプログラムを設計してください。プログラムはファイル名 `n` を受け取り、ファイルを読み込み、冠詞を削除して、結果を "no-articles-" に `n` を連結した名前のファイルに書き出します。元のテキストの行と単語の構造を維持するために、`read-words/line` を使用してください。設計が完了したら、Piet Hein の詩で実行してください。

練習問題 174. テキストファイルを数値的にエンコードするプログラムを設計してください。単語内の各文字は、0 から 256 の間の値を持つ 3 桁の数値文字列としてエンコードされる必要があります。図 69 は、1 文字に対するエンコード関数を示しています。開始する前に、これらの関数を説明してください。[ヒント] (1) 行と単語の構造を維持するために `read-words/line` を使用してください。(2) `explode` 関数の説明を再読してください。

図 69: 文字を数値コードにエンコードする

```
; lString -> String
; lString を3桁の数値文字列に変換する
(check-expect (code1 "a") "097")
(check-expect (code1 "z") "122")

(define (code1 c)
  (number->string-three (string->int c)))

; Number -> String
; 数値 n を3桁の文字列に変換する
(check-expect (number->string-three 7) "007")
(check-expect (number->string-three 12) "012")
(check-expect (number->string-three 120) "120")

(define (number->string-three n)
```

```
(cond
  [(and (<= 0 n) (< n 10))
   (string-append "00" (number->string n))]
  [(and (<= 10 n) (< n 100))
   (string-append "0" (number->string n))]
  [else (number->string n)]))
```

練習問題 175. Unix コマンドの `wc` をシミュレートする BSL プログラムを設計してください。このコマンドの目的は、与えられたファイル内の `lString` の数、単語数、および行数をカウントすることです。すなわち、コマンドはファイル名を受け取り、3つの数値からなる値を生成します（構造体を作成して結果を返すとよいでしょう）。（コンピュータの `wc` コマンドがあなたのプログラムとわずかに異なる結果を返すのはなぜでしょうか？）

次に、行列の計算を考えてみましょう。数学の授業で行列の計算を導入されたことがあるかもしれません。原則として、行列とは単に数値の長方形（グリッド）のことです。以下は行列に対する一つのデータ表現です。

```
; A Matrix is one of:
; -- (cons Row '())
; -- (cons Row Matrix)
; 制約： 行列内のすべての行は同じ長さである。

; A Row is one of:
; -- '()
; -- (cons Number Row)
```

行列に対する制約に注意してください。このデータ定義を調べて、11, 12, 21, 22 という数からなる 2×2 の行列をこの表現に翻訳してください。ストップ！データ例を作成するまで先を読まないでください。

このパズルに対する回答は以下の通りです。

```
(define row1 (cons 11 (cons 12 '())))
(define row2 (cons 21 (cons 22 '())))
(define mat1 (cons row1 (cons row2 '())))
```

作成できなかった場合は、このコードをよく調べてください。

図 70 の関数は、行列の要素を対角線（左上から右下への線）に沿って反転させる「転置 (transpose)」という重要な数学的操作を実装しています。

図 70: 行列の転置

```
; Matrix -> Matrix
; 与えられた行列を対角線に沿って転置する

(define wor1 (cons 11 (cons 21 '())))
(define wor2 (cons 12 (cons 22 '())))
(define tam1 (cons wor1 (cons wor2 '())))
```

```
(check-expect (transpose mat1) tam1)

(define (transpose lln)
  (cond
    [(empty? (first lln)) '()]
    [else (cons (first* lln)
                  (transpose (rest* lln)))]))
```

ストップ！ mat1 を手動で転置し、それから図 70 を読んでください。なぜ transpose は (empty? (first lln)) を確認しているのでしょうか？

この定義は、以下の 2 つの補助関数を仮定しています。 - first*: 行列を受け取り、最初の列を数値のリストとして返します。 - rest*: 行列を受け取り、最初の列を削除します。結果は行列になります。

これらの関数の定義がなくても、transpose がどのように機能するかを理解できるはずです。また、これまでに見た設計レシピではこの関数を設計できないことも理解してください（なぜでしょうか？）。

これら 2 つの「願望」関数を設計してください。その後、いくつかのテストケースを使って transpose の設計を完了させてください。

10.4 グラフィカルエディタの再訪 (A Graphical Editor, Revisited)

第 5 章で、私たちはインタラクティブなグラフィカルな 1 行エディタを設計しました。そこではエディタの状態を表現するために、2 つの文字列を保持する構造体か、文字列とカーソル位置のインデックスを保持する構造体のいずれかを使用しました。

3 番目の選択肢は、1String の 2 つのリストを組み合わせた構造体を使用することです。

```
(define-struct editor [pre post])
; An Editor is a structure:
;   (make-editor Lo1S Lo1S)

; An Lo1S (list of 1Strings の略) は以下のいずれか:
; -- '()
; -- (cons 1String Lo1S)
```

なぜこのような表現にするのかを考える前に、2 つのデータ例を作成してみましょう。

```
(define good
  (cons "g" (cons "o" (cons "o" (cons "d" '())))))
(define all
  (cons "a" (cons "l" (cons "l" '()))))
(define lla
  (cons "l" (cons "l" (cons "a" '()))))

; データ例1:
```

```
(make-editor all good)
```

```
; データ例2:
```

```
(make-editor lla good)
```

これら2つの例は、解釈 (interpretation) を書き留めることがいかに重要であることを示しています。エディタの2つのフィールドがカーソルの左側と右側の文字を表すことは明らかですが、この構造体タイプを解釈する方法には少なくとも2つのアプローチがあります。

1. (make-editor pre post) は、pre の文字がカーソルの前にあり、post の文字がカーソルの後ろにあることを意味し、組み合わされたテキストは次のようになる。

```
(string-append (implode pre) (implode post))
```

implode 関数は lString のリストを単一の String に変換することを思い出してください。

2. (make-editor pre post) は、pre の文字がカーソルの前に逆順 (reverse) で並んでいることを意味する。この場合、エディタに表示されるテキストは次のようになります。

```
(string-append (implode (reverse pre)) (implode post))
```

どちらの解釈も素晴らしい選択肢ですが、2番目の解釈を使用するとプログラムの設計が大幅に簡素化されることが分かっています。このセクションの残りの部分では、リストを構造体の中に組み込む使用法を示しながら、この点について説明します。

まず、データ定義を理解するために明らかに必要となる rev 関数 (リストを逆順にする関数) から始めましょう。

```
; lols -> lols
```

```
; 与えられたリストを逆順にしたリストを生成する
```

```
(check-expect (rev (cons "a" (cons "b" (cons "c" '()))))  
              (cons "c" (cons "b" (cons "a" '()))))
```

```
(define (rev l)  
  l)
```

rev のテンプレートは、通常のリストテンプレートです。

```
(define (rev l)  
  (cond  
    [(empty? l) ...]  
    [else (... (first l) ...  
               ... (rev (rest l)) ...)]))
```

空リストの逆順は空リストなので、最初の節を埋めるのは簡単です。2番目の節については、設計の質問を使います。 - (first l) はリストの最初の lString です。 - (rest l) はリストの残りの部分です。 - (rev (rest l)) はリストの残りの部分の逆順です。

ヒントとして、図 71 の表を作成します。

図 71: `rev` のための表

<code>l</code>	<code>(first l)</code>	<code>(rest l)</code>	<code>(rev (rest l))</code>	<code>(rev l)</code>
<code>(cons "a" '())</code>	<code>"a"</code>	<code>'()</code>	<code>'()</code>	<code>(cons "a" '())</code>
<code>(cons "a" (cons "a" '()))</code>	<code>"a"</code>	<code>(cons "b" '())</code>	<code>(cons "c" '())</code>	<code>(cons "c" (cons "a" '()))</code>
<code>(cons "b" (cons "a" '()))</code>	<code>"b"</code>	<code>(cons "c" (cons "b" '()))</code>	<code>(cons "b" (cons "c" '()))</code>	<code>(cons "b" (cons "c" (cons "a" '())))</code>
<code>(cons "c" (cons "b" (cons "a" '())))</code>	<code>"c"</code>	<code>'()</code>	<code>'()</code>	<code>(cons "a" (cons "b" (cons "c" '())))</code>

2 番目の例を見ると、`(rev (rest l))` が `(cons "c" (cons "b" '()))` を生成して、ほとんどの仕事を成し遂げていることが分かります。期待される結果は `(cons "c" (cons "b" (cons "a" '())))` なので、`rev` は再帰の結果の末尾に `"a"` (つまり `(first l)`) を追加する必要があります。

末尾に要素を追加する関数 `add-at-end` を「願望」し、それを使用して関数定義を完了させます。

```
(define (rev l)
  (cond
    [(empty? l) '()]
    [else (add-at-end (rev (rest l)) (first l))]))
```

願望リストの `add-at-end` の設計：

```
; lols lString -> lols
; l の末尾に s を追加した新しいリストを生成する
```

```
(check-expect (add-at-end (cons "c" (cons "b" '())) "a")
               (cons "c" (cons "b" (cons "a" '()))))
```

```
(define (add-at-end l s)
  l)
```

`add-at-end` もリスト処理関数なので、そのテンプレートは次のようになります。

```
(define (add-at-end l s)
  (cond
    [(empty? l) ...]
    [else (... (first l) ...
                ... (add-at-end (rest l) s) ...)]))
```

最初のケース (空リスト) に対する結果は、要素 `s` のみを含むリスト、すなわち `(cons s '())` です。2 番目のケース (複雑なケース) について、構成要素が何を計算するかを考えます。 - `(first l)` はリストの最初の `lString`。 - `(add-at-end (rest l) s)` は「`s` を `l` の残りの部分の末尾に追加した新しいリスト」。ここから、再帰の結果の先頭に `(first l)` を追加し直すだけでよいことが分かります。

```
(define (add-at-end l s)
  (cond
    [(empty? l) (cons s '())]
    [else (cons (first l) (add-at-end (rest l) s))]))
```

テストを実行して、この関数が機能し、それによって `rev` も機能することを確認してください。もちろん、BSL には標準で任意のリストを反転させる `reverse` 関数がすでに用意されているため、実際にはそれを使用できます。

練習問題 177. 関数 `create-editor` を設計してください。この関数は 2 つの文字列を受け取り、`Editor` を生成します。最初の文字列はカーソルの左側のテキスト、2 番目の文字列はカーソルの右側のテキストです。

これでエディタのデータ表現を完全に理解できました。インタラクティブなプログラムの設計戦略に従って、定数を定義します。

```
(define HEIGHT 20) ; エディタの高さ
(define WIDTH 200) ; エディタの幅
(define FONT-SIZE 16) ; フォントサイズ
(define FONT-COLOR "black") ; フォントカラー
```

```
(define MT (empty-scene WIDTH HEIGHT))
(define CURSOR (rectangle 1 HEIGHT "solid" "red"))
```

次に、イベントハンドラと描画関数のための願望リストを作成します。

```
; Editor -> Image
; カーソルで区切られた2つのテキストの画像としてエディタを描画する
(define (editor-render e)
  MT)

; Editor KeyEvent -> Editor
; キーイベントを処理する
(define (editor-kh ed k)
  ed)
```

また、プログラムを開始する `main` 関数を定義します。

```
; String -> Editor
; 初期文字列を受け取りエディタを起動する
(define (main s)
  (big-bang (create-editor s "")
    [on-key editor-kh]
    [to-draw editor-render]))
```

`editor-kh` を設計するために、2 つの例を作成します。

```
(check-expect (editor-kh (create-editor "" "") "e")
  (create-editor "e" ""))
```

```
(check-expect (editor-kh (create-editor "cd" "fgh") "e")
               (create-editor "cde" "fgh"))
```

これらはいずれも、キーボードの“e”を押したときに何が起こるかを示しています。関数はカーソル位置に文字を挿入します。

バックスペース (“\b”) キーを押して文字を削除したとき、“left” および “right” 矢印キーを押してカーソルを移動したときなどの例も作成してください。エディタが空のとき、カーソルが左端や右端にあるときなどの極端なケースについても検討してください。

editor-kh のテンプレートは、どの KeyEvent を受信したかをチェックする大きな cond 式になります。

```
(define (editor-kh ed k)
  (cond
    [(key=? k "left") ...]
    [(key=? k "right") ...]
    [(key=? k "\b") ...]
    [(key=? k "\t") ...]
    [(key=? k "\r") ...]
    [(= (string-length k) 1) ...]
    [else ...]))
```

練習問題 178. editor-kh のテンプレートが、長さ 1 の文字列をチェックする前に “\t” と “\r” を処理する理由を説明してください。

各節の実装には、タスクごとに 1 つの関数を「願望」します。

```
(define (editor-kh ed k)
  (cond
    [(key=? k "left") (editor-lft ed)]
    [(key=? k "right") (editor-rgt ed)]
    [(key=? k "\b") (editor-del ed)]
    [(key=? k "\t") ed]
    [(key=? k "\r") ed]
    [(= (string-length k) 1) (editor-ins ed k)]
    [else ed]))
```

ここで、editor-ins を設計します。

```
; Editor lString -> Editor
; lString k を pre と post の間に挿入する
(define (editor-ins ed k)
  ed)
```

この関数は、エディタの pre フィールドの先頭に k を追加した新しいエディタを作成するだけです。

```
(define (editor-ins ed k)
  (make-editor (cons k (editor-pre ed))
               (editor-post ed)))
```


pre が逆順で保持されているため、pre の先頭に cons することは、カーソル位置の直前に文字を挿入することに対応します。補助関数を使用せずに、単なる cons だけで挿入を実装できるのが、この表現形式の大きなメリットです。

練習問題 179. 以下の関数を設計してください。 - editor-lft: カーソルを 1 文字分左に移動する (可能であれば)。 - editor-rgt: カーソルを 1 文字分右に移動する (可能であれば)。 - editor-del: カーソルの左側の文字を削除する (可能であれば)。豊富な例を作成して、実装を検証してください。

最後に、エディタを描画する editor-render 関数を設計します。editor-render はエディタの状態を受け取り、カーソルを挟んだ 2 つのテキスト画像を合成して表示します。

たとえば、(create-editor "pre" "post") の結果を描画するには、次のような式を使用します。

```
(place-image/align
  (beside (text "pre" FONT-SIZE FONT-COLOR)
    CURSOR
    (text "post" FONT-SIZE FONT-COLOR))
  1 1 "left" "top"
  MT)
```

この処理を行うためのテンプレートを記述します。

```
(define (editor-render e)
  (... (editor-pre e) ... (editor-post e) ...))
```

editor-pre と editor-post は lString のリストなので、リストをテキスト画像に変換するヘルパー関数 editor-text を願望します。

```
; Editor -> Image
(define (editor-render e)
  (place-image/align
    (beside (editor-text (editor-pre e))
      CURSOR
      (editor-text (editor-post e)))
    1 1 "left" "top"
    MT))
```

editor-text は lString のリストを受け取り、それをテキスト画像に変換します。

```
; Lols -> Image
; lString のリストをテキスト画像として描画する
(define (editor-text s)
  (text (implode s) FONT-SIZE FONT-COLOR))
```

しかし、このままでテストを走らせると、editor-render のテストは失敗します。なぜなら、pre フィールドの文字リストは逆順で保持されているため、そのまま描画すると左側のテキストが逆順に表示されてしまうからです。

したがって、editor-pre のリストを reverse で元に戻してから editor-text に渡す必要があります。

```
(define (editor-render ed)
  (place-image/align
    (beside (editor-text (reverse (editor-pre ed)))
            CURSOR
            (editor-text (editor-post ed)))
    1 1 "left" "top"
    MT))
```

これでテストが成功します。

練習問題 180. implode を使用せずに editor-text を設計してください。

注意 現代のアプリケーションでは、ユーザーがマウスでカーソル位置を指定できます。このような機能を追加することは可能ですが、それは第6部「アキュムレータを用いたエディタ」まで保留します。

11 合成による設計 (Design by Composition)

これまでに、プログラムは複雑な成果物であり、その作成には協調して動作する多くの関数の設計が必要であることを学びました。設計者が「いつ複数の関数を設計すべきか」、そして「それらの関数をどのように単一のプログラムに合成するか」を理解していれば、この協調関係はうまく機能します。

あなた自身、相互に関連する関数を設計する必要性をこれまでに何度も経験してきました。ある場合には、問題文が複数の異なるタスクを示唆し、各タスクが1つの関数で表現されるのが最適であることもありました。また別の場合には、あるデータ定義が別のデータ定義を参照し、その結果、前者のデータを処理する関数が後者のデータを処理する関数に依存することもありました。

この章では、多くの関数を合成するプログラムの設計が必要となるいくつかのシナリオを紹介します。このような設計をサポートするために、この章では関数の分割とそれらの合成に関するいくつかの非公式な指針を提示します。ただし、これらの例には複雑な形式のリストが必要となるため、まずは簡潔なリストの表記法を説明するセクションから始めます。

11.1 list 関数 (The list Function)

これまでに、リストを作成するためだけにこれほど多くの cons を書くこと、特に多くの値を含むリストにおいて cons を書き連ねることにうんざりしているかもしれません。幸いなことに、プログラマーのこのような負担を軽減するためのメカニズムを提供する教育用言語が用意されています。BSLにリストの略記法を追加した言語 (BSL+) がそれです (Racket の「Beginning Student with List Abbreviations」を選択してください)。

重要な技術革新は、任意個の値を受け取ってリストを生成する `list` 関数です。`list` を理解する最も簡単な方法は、それを単なる略記法（アブレヴィエーション）と考えることです。具体的には、次の形式の式：

```
(list exp-1 ... exp-n)
```

は、`n` 個の `cons` 式が入れ子になった一連の式の略記です。

```
(cons exp-1 (... (cons exp-n '())))
```

ここで `'()` はリストの要素ではなく、リストの終端（残りの部分）であることに注意してください。以下に3つの例を示します。

略記表現 (short-hand)	通常表現 (long-hand)
<code>(list "ABC")</code>	<code>(cons "ABC" '())</code>
<code>(list #false #true)</code>	<code>(cons #false (cons #true '()))</code>
<code>(list 1 2 3)</code>	<code>(cons 1 (cons 2 (cons 3 '())))</code>

これらはそれぞれ、1つ、2つ、および3つの要素を含むリストを生成します。

当然ながら、`list` には値だけでなく、任意の式を引数として渡すことができます。

```
> (list (+ 0 1) (+ 1 1))
(list 1 2)
> (list (/ 1 0) (+ 1 1))
/ : division by zero
```

リストが構築される前に、まず引数の式が評価されます。式の評価中にエラーが発生した場合、リストは構築されません。要するに、`list` は任意個の引数を受け取る他のプリミティブ操作と同様に動作し、その結果がたまたま `cons` で構築されたリストになるだけです。

`list` を使用すると、多数の項目を含むリストや、リストや構造体をネストした複雑なリストの記述が大幅に簡素化されます。

```
(list 0 1 2 3 4 5 6 7 8 9)
```

この10個の要素を含むリストを `cons` だけで記述すると、10回の `cons` と1回の `'()` が必要になります。同様に、次のリスト：

```
(list (list "bob" 0 "a")
      (list "carl" 1 "a")
      (list "dana" 2 "b")
      (list "erik" 3 "c")
      (list "frank" 4 "a")
      (list "grant" 5 "b")
      (list "hank" 6 "c")
      (list "ian" 7 "a"))
```

```
(list "john" 8 "d")
(list "karel" 9 "e"))
```

は 11 回の `list` の使用で済みますが、これを `cons` で書くと 40 回の `cons` と 11 回の `'()` が必要になります。

練習問題 181. `list` を使用して、以下のリストと等価なリストを構築してください。 1. `(cons "a" (cons "b" (cons "c" (cons "d" '()))))` 2. `(cons (cons 1 (cons 2 '())) '())` 3. `(cons "a" (cons (cons 1 '()) (cons #false '())))` 4. `(cons (cons "a" (cons 2 '())) (cons "hello" '()))` また、次のものにも挑戦してください。 5. `(cons (cons 1 (cons 2 '())) (cons (cons 2 '()) '()))` まず、それぞれのリストおよび入れ子になったリストに要素がいくつ含まれているかを判断してください。解答を `check-expect` を使って表現し、あなたの略記法が通常表現と本当に等価であることを確認してください。

練習問題 182. `cons` と `'()` のみを使用して、以下のリストと等価な表現を作成してください。 1. `(list 0 1 2 3 4 5)` 2. `(list (list "he" 0) (list "it" 1) (list "lui" 14))` 3. `(list 1 (list 1 2) (list 1 2 3))` `check-expect` を使って解答を表現してください。

練習問題 183. 場合によっては、リストが `cons` と `list` を組み合わせて記述されることがあります。 1. `(cons "a" (list 0 #false))` 2. `(list (cons 1 (cons 13 '())))` 3. `(cons (list 1 (list 13 '())) '())` 4. `(list '() '() (cons 1 '()))` 5. `(cons "a" (cons (list 1) (list #false '())))` これらの式を、`cons` のみ、または `list` のみを使用して書き直してください。 `check-expect` を使用して答えを確認してください。

練習問題 184. 次の式が評価されたときの値を決定してください。 1. `(list (string=? "a" "b") #false)` 2. `(list (+ 10 20) (* 10 20) (/ 10 20))` 3. `(list "dana" "jane" "mary" "laura")` `check-expect` を使って解答を表現してください。

練習問題 185. BSL の `first` と `rest` については知っていますが、BSL+ にはさらに多くのセレクタが存在します。次の式の値を決定してください。 1. `(first (list 1 2 3))` 2. `(rest (list 1 2 3))` 3. `(second (list 1 2 3))` ドキュメントを参照して、`third` や `fourth` が存在するかどうかを確認してください。

11.2 関数の合成 (Composing Functions)

『プログラム設計の手法』では、プログラムが定義（構造体定義、データ定義、定数定義、関数定義、およびテスト）の集まりであることを説明しています。関数間の役割分担を管理するために、大まかな指針を提示しました。

1つのタスクにつき1つの関数を設計せよ。問題の中の数量間の依存関係ごとに補助関数を定義せよ。

また、補助関数に関する別の指針も紹介しました。

1つのデータ定義につき1つのテンプレートを設計せよ。あるデータ定義が別のデータ定義を指し示している場合、補助関数を定義せよ。

このセクションでは、設計プロセスの特定の段階でさらなる補助関数の設計が必要になるケース、すなわち「テンプレートから完全な関数定義を作成するステップ（ステップ5）」に焦点を当てます。テンプレートを完全な関数定義にするということは、テンプレートのサブ式の値を組み合わせて最終的な回答にすることです。その際、次のような状況において補助関数が必要になることがあります。

1. ドメイン知識が必要な場合：値の組み合わせに特定の応用分野（コンピューターグラフィックス、会計、音楽、科学など）の専門知識が必要な場合、その処理を補助関数として設計します。
2. 複雑な条件分岐がある場合：値の組み合わせが、利用可能な値のケース分析（例えば、数値が正、ゼロ、または負であるかなど）に依存する場合、cond 式を使用します。もし cond 式が複雑になりそうな場合は、テンプレートの各式を引数として受け取り、その内部で cond による分岐を行う補助関数を設計します。
3. 自己参照データを処理する場合：値の組み合わせにおいて、自己参照データ定義（リストや自然数など）から得られる要素を処理する必要がある場合、補助関数を設計します。
4. 問題の一般化が必要な場合：どうしても解決できない場合、より「一般的な」問題を解く関数を設計し、メイン関数をその汎用関数の特定の用途として定義することがあります。これは直感に反するよう思えるかもしれませんが、驚くほど多くのケースで必要とされます。

最後の2つの基準はまだ詳しく議論していませんが、これまでに例がいくつか登場しています。次の2つのセクションで、これらの原則を具体例とともに説明します。

しかし、続ける前に、プログラムの設計を管理する鍵は、しばしば言及される「願望リスト (Wish List)」を維持することであることを思い出してください。

願望リストの維持 プログラムを完了するために設計する必要がある関数のヘッダー（シグネチャ、目的ステートメント、および関数ヘッダー）のリストを維持します。完全な関数ヘッダーを書き留めておけば、未完成の関数をスタブ（ダミー）で補うことで、完成した部分のプログラムをテストできます。テストの多くは最初は失敗しますが、これは非常に有用です。当然ながら、願望リストが空になれば、すべてのテストがパスし、すべての関数がテストによってカバーされるはずです。

関数を願望リストに追加する前に、使用している言語のティーチパックに同様の関数がすでに存在しないか、あるいはすでに願望リストに似たようなものが入っていないかを確認する必要があります。BSL や BSL+、およびすべてのプログラミング言語は、多くの

組み込み操作やティーチパック関数を提供しています。時間があるときや必要が生じたときに、使用している言語で何が提供されているか探究してください。

11.3 再帰する補助関数 (Auxiliary Functions that Recur)

人は常に物事をソート（整列）する必要がありますし、プログラムも同様です。投資アドバイザーは保有資産を生み出す利益順にソートし、ゲームプログラムはプレイヤーをスコア順にソートし、メールプログラムはメッセージを日付や送信者などの基準でソートします。

一般に、データの各ペアを比較して順序付けできるなら、任意のデータをソートできます。すべてのデータ型に比較プリミティブが備わっているわけではありませんが、私たちは誰もがよく知っているデータ型、すなわち「数値」を対象にします。したがって、このセクションでは単純ですが非常に代表的なサンプル問題に取り組みます。

サンプル問題 実数のリストをソートする関数を設計してください。

以下の練習問題では、この関数を他のデータ型にどのように適応させるかを説明します。

問題文には他のタスクについての言及がなく、ソート自体が他のタスクを示唆しているようには見えないため、設計レシピをそのまま適用します。ソートとは、いくつかの数値を並べ替えることを意味します。この言い換えは、関数の入力と出力に対する自然なデータ定義（したがってシグネチャ）を示唆しています。List-of-numbers の定義があるため、最初のステップは簡単です。

```
; List-of-numbers -> List-of-numbers
; alon を降順にソートしたリストを生成する
(define (sort> alon)
  alon)
```

入力の alon をそのまま返すことで、関数のシグネチャに関しては適切な結果になりますが、一般には与えられたリストはソートされていないため、この実装は間違っています。

例を作成し始めると、問題文が非常に曖昧であることに気づきます。いつものように、例の開発を整理するために List-of-numbers のデータ定義を使用します。データ定義には 2 つの節があるため、2 つの例が必要です。明らかに、sort> を '() に適用した場合の結果は '() です。問題は、次のリストに対する結果がどうあるべきかです。

```
(cons 12 (cons 20 (cons -5 '())))
```

このリストはソートされていませんが、ソートには 2 つの方向があります。- (cons 20 (cons 12 (cons -5 '()))) (降順) - (cons -5 (cons 12 (cons 20 '()))) (昇順)

ここでは降順（大きい順）にソートすることにし、関数名を sort> とします。

したがって、テストケースは次のようになります。

```
(check-expect (sort> '()) '())
(check-expect (sort> (list 20 12 5 10 2)) (list 20 12 10 5 2))
(check-expect (sort> (list 12 20 -5)) (list 20 12 -5))
```

略記表現 (list) を使用すると、例の記述が簡単になり、何が期待されているかを見やすくなります。

第4ステップは、通常のテンプレートのコピーです。

```
(define (sort> alon)
  (cond
    [(empty? alon) ...]
    [else (... (first alon) ...
               ... (sort> (rest alon)) ...)]))
```

これで、最も興味強く創造的なステップ、すなわちテンプレートを定義に変換する段階に達しました。最初のケースは簡単で、空リストに対する結果は '()' です。

2 番目のケースについて、構成要素が何を計算するかを確認します。- (first alon) はリストの最初の数値です。- (rest alon) はリストの残りの部分です。- (sort> (rest alon)) は、リストの残りの部分を降順にソートしたリストを返します。目的ステートメントとシグネチャがこの結果を保証しています。

たとえば、alon が (list 12 20 -5) の場合、これらの式は次の値を生成します。- (first alon) -> 12 - (sort> (rest alon)) -> (list 20 -5) (すでにソートされている)

私たちの仕事は、12 と (list 20 -5) を組み合わせて、全体をソートしたリスト (list 20 12 -5) を生成することです。

単純に cons で 12 を (list 20 -5) の先頭に追加すると、(list 12 20 -5) が生成され、これはソートされていません。したがって、単なる cons 以上のものがが必要です。

必要なのは、数値を「すでにソートされた数値のリスト」の正しい位置に挿入する関数です。このタスクはそれ自体が複雑であるため、補助関数として設計するよう促す最初の基準に当てはまります。この関数を insert と呼び、願望リストに追加します。

```
; Number List-of-numbers -> List-of-numbers
; すでにソートされているリスト alon の正しい位置に n を挿入する
(define (insert n alon)
  alon)
```

この願望関数があれば、sort> は次のように定義できます。

```
(define (sort> alon)
  (cond
    [(empty? alon) '()]
    [else (insert (first alon) (sort> (rest alon)))]))
```

この定義は、テンプレートから直接派生しており、insert を使って最初の数値を残りのソート済みリストに挿入しています。

次に、insert 関数を設計する必要があります。最初のステップとして、入力と出力のデータ定義を確認します。最初の引数は単なる数値、2 番目の引数はすでにソートされている数値のリストです。結果もソートされた数値のリストです。

2 番目のステップは、シグネチャと目的ステートメント（上記で定義済み）です。

第3ステップは、例の作成です。List-of-numbers の定義に基づいて、2 番目の引数が空の場合とそうでない場合の例を作成します。 - (insert 5 '()) -> (list 5) - (insert 5 (list 6)) -> (list 6 5) - (insert 5 (list 4)) -> (list 5 4) - (insert 5 (list 6 4)) -> (list 6 5 4) - (insert 5 (list 6 3)) -> (list 6 5 3)

これをテストケースに書き換えます。

```
(check-expect (insert 5 '()) (list 5))
(check-expect (insert 5 (list 6)) (list 6 5))
(check-expect (insert 5 (list 4)) (list 5 4))
(check-expect (insert 12 (list 20 -5)) (list 20 12 -5))
```

第4ステップは、リストを処理する関数のテンプレートの作成です。

```
(define (insert n alon)
  (cond
    [(empty? alon) ...]
    [else (... n ...
              ... (first alon) ...
              ... (insert n (rest alon)) ...)]))
```

テンプレートには、関数のすべてのパラメータ (n と alon) が含まれ、alon が自己参照データであるため、再帰呼び出し (insert n (rest alon)) が含まれます。

第5のステップに進み、各節の本体を埋めていきます。最初の節 (alon が空) は簡単で、結果は (list n)、すなわち (cons n '()) です。

2 番目の節 (alon が空ではない) について、構成要素が何を計算するかを整理します。 - n は挿入したい数値。 - (first alon) はリストの最初の（そして最大の）数値。 - (insert n (rest alon)) は、n を残りのソート済みリストに正しく挿入したリスト。

n を alon に挿入するとき、n が最初の要素 (first alon) よりも大きい（あるいは等しい）場合、n をリストの先頭に追加するだけで完了です。なぜなら、alon はすでに降順にソートされており、(first alon) がその中で最大だからです。

```
(cons n alon)
```

そうでない場合 (n が (first alon) より小さい場合)、最初の要素 (first alon) は全体の最大値であり続けるため、結果の先頭に残らなければなりません。そして、n はリストの残りの部分 (rest alon) の中に挿入する必要があります。

```
(cons (first alon) (insert n (rest alon)))
```

この論理から、2 番目の節では `n` と `(first alon)` の比較による条件分岐が必要であることが分かります。

```
(define (insert n alon)
  (cond
    [(empty? alon) (cons n '())]
    [else (cond
              [(>= n (first alon)) (cons n alon)]
              [else (cons (first alon) (insert n (rest alon)))]))]))
```

このように、`cond` の中に別の `cond` 式を入れ子にすることができます。また、このコードを整理して、単一の `cond` に平坦化することも可能です。図 72 に完成したプログラムを示します。

図 72: リストをソートする

```
; List-of-numbers -> List-of-numbers
; alon を降順にソートしたリストを生成する
(check-expect (sort> '()) '())
(check-expect (sort> (list 20 12 5 10 2)) (list 20 12 10 5 2))
(check-expect (sort> (list 12 20 -5)) (list 20 12 -5))
```

```
(define (sort> alon)
  (cond
    [(empty? alon) '()]
    [else (insert (first alon) (sort> (rest alon)))]))
```

```
; Number List-of-numbers -> List-of-numbers
; すでにソートされているリスト alon の正しい位置に n を挿入する
(check-expect (insert 5 '()) (list 5))
(check-expect (insert 5 (list 6)) (list 6 5))
(check-expect (insert 5 (list 4)) (list 5 4))
(check-expect (insert 12 (list 20 -5)) (list 20 12 -5))
```

```
(define (insert n alon)
  (cond
    [(empty? alon) (cons n '())]
    [else (cond
              [(>= n (first alon)) (cons n alon)]
              [else (cons (first alon) (insert n (rest alon)))]))]))
```

練習問題 186. 図 72 のプログラムを DrRacket でテストしてください。次に、`sort>` を `sort<` (リストを昇順にソートする関数) に変更してください。これを行うには、プログラムの何箇所を変更する必要があるでしょうか？

練習問題 187. ゲームプログラムがプレイヤーのスコアリストをソートするとします。プレイヤーを表す構造体を次のように定義します。

```
(define-struct gp [name score])
; A GamePlayer is a structure:
```

```
; (make-gp String Number)
; 解釈: (make-gp n s) はプレイヤー名 n とそのスコア s を表す。
```

GamePlayer のリストをスコアの降順にソートする関数 `sort-players` を設計してください。

練習問題 188. 郵便物やメールソフトは、メールリストをさまざまな基準でソートします。電子メールを表す構造体を次のように定義します。

```
(define-struct email [from date message])
; An Email is a structure:
; (make-email String Number String)
; 解釈: (make-email f d m) は送信者 f、送信日（1970年からの経過日数）d、メッセージ内容 m を表す。
```

1. Email のリストを日付順にソートする関数 `sort-by-date` を設計してください。
2. Email のリストを送信者名でアルファベット順にソートする関数 `sort-by-from` を設計してください。

練習問題 189. すでにソートされた数値リスト `l` と数値 `n` に対する `search` 関数を設計してください。この関数は、`n` が `l` に含まれているかどうかを判定します。リストの残りに対して再帰する前に、検索を停止できるのはどのような場合でしょうか？

練習問題 190. 文字列のリストを受け取り、その中のすべてのプレフィックス（接頭辞）をリストとして返す関数 `prefixes` を設計してください。たとえば、`(list "a" "b" "c")` に対しては、次のリストのリストを返します。

```
(list (list "a" "b" "c")
      (list "a" "b")
      (list "a")
      '())
```

同様に、すべてのサフィックス（接尾辞）を返す関数 `suffixes` を設計してください。

11.4 別の視点からの多角形描画 (Another Look at Polygon Rendering)

第1部「2.4 グラフィックスの描画」では、画面上に幾何学的な形状を描画する方法を見ました。そこでは多角形を描画する関数を設計しようとしたしましたが、最終的には三角形しか描画できませんでした。今回はリストを使用できるようになったため、任意の数の頂点を持つ多角形を描画する関数を設計できます。

多角形は、頂点を表すポイント（`Posn` 構造体）のリストとして定義されます。

```
; A Polygon is one of:
; -- (list Posn Posn Posn)
; -- (cons Posn Polygon)
```

このデータ定義は、多角形が少なくとも3つの頂点を持つことを保証しています。

多角形を描画する関数 `render-poly` を設計します。

```
; Image Polygon -> Image
; 与えられた画像 img に多角形 p の境界を描画する
(define (render-poly img p)
  img)
```

この関数をテストするために、2つの多角形の例を作成します。

```
(define triangle-p
  (list (make-posn 20 10)
        (make-posn 20 20)
        (make-posn 30 20)))
```

```
(define square-p
  (list (make-posn 10 10)
        (make-posn 20 10)
        (make-posn 20 20)
        (make-posn 10 20)))
```

これらを描画したときの期待されるイメージを作成し、テストケースを定式化します。

Polygon のデータ定義に基づいて、テンプレートを作成します。

```
(define (render-poly img p)
  (cond
    [(empty? (rest (rest (rest p))))] ...)
    [else (... (first p) ...
                ... (render-poly img (rest p)) ...)]))
```

最初の節は、リストの残りの残りの残りが空である場合、すなわちリストがちょうど3つの要素（三角形）で構成されている場合に対応します。この場合、3つの頂点は first、second、および third セレクタで抽出されます。2番目の節は、4つ以上の頂点を持つ多角形に対応し、first で最初の頂点、rest で残りの多角形を抽出します。

最初の節（三角形）の描画には、3つの点を結ぶ補助関数 render-line を「願望」します。

```
; Image Posn Posn -> Image
; img に p から q への赤色の線を描画する
(define (render-line img p q)
  img)
```

この関数を使用すると、最初の節は次のようになります。

```
(render-line
  (render-line
    (render-line MT (first p) (second p))
    (second p) (third p))
  (third p) (first p))
```

2番目のケースでは、構成要素は次のようになります。 - (first p) : 最初の頂点。 - (rest p) : 残りの多角形。 - (render-poly img (rest p)) : 残りの多角形を描画した画像。

最初の頂点と、残りの多角形の最初の頂点を接続し、その結果を残りの多角形を描画した画像の上に重ねることで、全体の多角形を描画できます。

```
(render-line (render-poly img (rest p))
              (first p)
              (second p))
```

完成した関数定義と `render-line` を合わせたプログラムを走らせてテストすると、テストは失敗します。描画された画像を見ると、最後の点が最初の点に接続されておらず、多角形が「閉じていない」ことが分かります（あるいは、常に最後の三角形の部分だけが閉じていて、他は開いた状態になります）。

これは、関数が「多角形を閉じる（最後の点と最初の点を結ぶ）」というタスクを、再帰の各ステップで正しく処理できていないためです。

この問題を解決するために、より「一般的な」問題を定義します。＞ 一般的な問題 与えられた点のリストを順に接続する（ただし最後の点と最初の点は接続しない）関数 `connect-dots` を設計してください。

この関数の入力データ定義として、少なくとも1つの点を含むリスト `NELoP` (Non-Empty List of Posns) を定義します。

```
; An NELoP (Non-empty list of Posns) is one of:
; -- (cons Posn '())
; -- (cons Posn NELoP)
```

`connect-dots` を設計します。

```
; Image NELoP -> Image
; img に p 内の点を順に結ぶ線を描画する
(define (connect-dots img p)
  (cond
    [(empty? (rest p)) img]
    [else (render-line (connect-dots img (rest p))
                        (first p)
                        (second p))]))
```

この `connect-dots` 関数を使用すると、元の多角形描画関数 `render-poly` は、非常にシンプルかつ正しく再定義できます。多角形を描画するということは、「すべての点を順に接続し」、さらに「最後の点と最初の点を接続する」ことです。

```
(define (render-poly img p)
  (render-line (connect-dots img p)
                (first p)
                (last p)))
```

ここで、`last` はリストの最後の要素を抽出する補助関数です。

練習問題 191. `connect-dots` に対するテストケースを作成してください。

練習問題 192. `last` 関数を設計してください。この関数は、非空リストの最後の要素を返します。

練習問題 193. `render-poly` の 2 つの定義（三角形を明示的に処理するバージョンと、`connect-dots` を使用するバージョン）について、2 番目の定義の方が優れている理由を説明してください。

練習問題 194. `connect-dots` の定義を変更して、頂点リストの最後に達したときに最初の頂点と接続するようにすることで、`render-poly` を `connect-dots` の単なる呼び出しとして定義できるようにしてください。これを行うには、`connect-dots` が多角形の最初の頂点を「記憶」している必要があります。どのように変更すればよいでしょうか？

12 プロジェクト：リスト (Projects: Lists)

この章では、いくつかの拡張的な練習問題（プロジェクト）を紹介します。これらはすべて、バッチプログラムや対話的プログラムの設計、合成による設計、願望リスト（ウィッシュリスト）の設計、および関数の設計レシピといった、設計の構成要素に対する理解をより確固たるものにすることを目的としています。

最初のセクションでは、現実世界のデータを取り扱います。具体的には、英語の辞書データと iTunes のライブラリ情報です。また、ワードゲーム（単語パズル）に関する課題では、2 つのセクションを割いて、関数の合成や問題の本質的な解決に焦点を当てます。残りのセクションでは、ゲームプログラミング（ワームゲーム、テトリス、スペースウォー）や、有限状態機械（FSM）の構築を行います。

12.1 現実世界のデータ：辞書 (Real-World Data: Dictionaries)

現実世界の情報は大量に存在する傾向があり、だからこそプログラムを使ってそれを処理することには大きな意味があります。たとえば、辞書には数個の単語だけでなく、何十万もの単語が含まれています。このような大量の情報を処理するプログラムを設計するときは、小さな例を使用して慎重に進める必要があります。プログラムが正しく動作することを確認できたら、現実世界のデータに対して実行して実際の結果を得ます。もしプログラムの処理速度があまりにも遅い場合は、各関数の動作を振り返り、無駄な計算を排除できないか検討してください。

図 74 は、英語の辞書全体を読み込むために必要な、わずか 1 行のコードを示しています。

図 74: 辞書の読み込み

```
; OS X の場合:
(define LOCATION "/usr/share/dict/words")
; LINUX の場合: /usr/share/dict/words または /var/lib/dict/words
; WINDOWS の場合: Linuxを使っている友人からファイルをもらうか、Webから取得してください

; A Dictionary is a List-of-strings.
(define AS-LIST (read-lines LOCATION))
```


図 74 で示されているコードをあなたのコンピューターに合わせて調整し、length 関数を使用して、あなたの辞書にいくつの単語が含まれているかを確認してください（著者の環境では 2017 年 7 月 25 日時点で 235,886 単語ありました）。

以下の練習問題では、文字（Letter）が重要な役割を果たします。図 74 のコードに加えて、プログラムの先頭に以下の定義を追加するとよいでしょう。

```
; A Letter is one of the following 1Strings:  
; -- "a"  
; -- ...  
; -- "z"  
; または、等価的に次のリストの要素:  
(define LETTERS (explode "abcdefghijklmnopqrstuvwxyz"))
```

ヒント：練習問題の例やテストを作成する際には list を活用してください。

練習問題 195. 文字 Letter と辞書 Dictionary を受け取り、与えられた辞書の中にその文字で始まる単語がいくつあるかをカウントする関数 starts-with# を設計してください。関数が正しく動作することを確認したら、あなたのコンピューターの辞書で “e” で始まる単語と “z” で始まる単語がそれぞれいくつあるかを調べてください。

練習問題 196. 関数 count-by-letter を設計してください。この関数は辞書 Dictionary を受け取り、その辞書内の各文字が単語の先頭文字として何回使われているかをカウントします。結果は Letter-Count のリストになります。Letter-Count は、文字とカウント数を組み合わせるデータ型です。まず、このデータ構造（構造体定義など）を設計してから、関数を設計してください。設計が完了したら、あなたのコンピューターの辞書にあるすべての文字について、それぞれの出現回数を調べてください。

練習問題 197. 関数 most-frequent を設計してください。この関数は辞書 Dictionary を受け取り、その辞書の中で最も頻繁に単語の先頭文字として出現する文字に対応する Letter-Count を返します。あなたのコンピューターの辞書で最も頻繁に使われている先頭文字は何で、それは何回出現しますか？

設計の選択肢に関する注意：この練習問題では、前の演習の解決策と、Letter-Count のリストから適切なペアを選択する関数を合成することが求められます。後者の関数を設計するには、次の 2 つのアプローチがあります。 - カウントが最大のペアを選択する関数を設計する。 - ソートされたペアのリストから最初の要素を選択する関数を設計する。両方の設計を検討してください。どちらが好みですか？その理由は何ですか？

練習問題 198. 関数 words-by-first-letter を設計してください。この関数は辞書 Dictionary を受け取り、文字ごとに分類された辞書のリスト（すなわち、辞書のリスト）を返します。この新しい関数を使用して、練習問題 197 の most-frequent を再設計してください（これを most-frequent.v2 と呼びます）。設計が完了したら、2 つの関数があるあなたのコンピューターの辞書に対して同じ結果を返すことを確認してください。

```
(check-expect (most-frequent AS-LIST) (most-frequent.v2 AS-LIST))
```


設計の選択肢に関する注意: words-by-first-letter については、与えられた辞書にある特定の文字で始まる単語が1つも存在しない場合の処理に選択肢があります。 - 1つの代替案は、結果のリストから空の辞書を除外することです。これにより、関数のテストや most-frequent.v2 の設計が簡単になりますが、追加の補助関数を設計する必要があります。 - もう1つの代替案は、該当する単語がない場合でも結果として空リスト '()' を含むことです。この方法は、除外するための補助関数は不要になりますが、most-frequent.v2 の設計が少し複雑になります。

12.2 現実世界のデータ: iTunes (Real-World Data: iTunes)

Apple の iTunes ソフトウェアは、音楽やビデオなどのコレクション管理に広く使われてきました。あなたの iTunes アプリケーションが収集した情報を解析したいと思うかもしれません。データベースを抽出するのは実は簡単で、メニューから「ライブラリを書き出す」を選択すると、iTunes の情報の XML 表現をエクスポートできます。XML の処理は後のプロジェクトで詳しく扱いますが、ここでは 2http/itunes ティーチパックを使用してデータを取得します。

iTunes ライブラリは、各音楽トラックについて以下のような情報を管理しています。 - トラック ID (Track ID): ライブラリ内のユニークな識別子。例: 442 - 名前 (Name): トラックのタイトル。例: "Wild Child" - アーティスト (Artist): 制作アーティスト。例: "Enya" - album (Album): 所属するアルバムのタイトル。例: "A Day Without Rain" - ジャンル (Genre): 割り当てられた音楽ジャンル。例: "New Age" - 種類 (Kind): ファイルのエンコーディング。例: "MPEG audio file" - サイズ (Size): ファイルサイズ (バイト)。例: 4562044 - トータルタイム (Total Time): 再生時間 (ミリ秒)。例: 227996 - トラック番号 (Track Number): アルバム内での位置。例: 2 - トラック数 (Track Count): アルバム内の総トラック数。例: 11 - 年 (Year): リリース年。例: 2000 - 追加日 (Date Added): トラックが追加された日時。例: "2002-7-17 3:55:14" - 再生回数 (Play Count): これまでに再生された回数。例: 20 - 再生日 (Play Date): 最後に再生された日時 (Unix タイムスタンプまたは UTC)。

私たちは音楽トラックについて、2つの表現 (構造体ベースの表現と、リストベースの表現) を使用します。

図 75 および図 76 は、2http/itunes ティーチパックで実装されている構造体ベースのトラック表現を示しています。

図 75: 構造体によるトラック表現 (構造体定義)

; ティーチパックのドキュメント (パート 1):

```
(define-struct track [name artist album time track# added play# played])
; A Track is a structure:
;   (make-track String String String N N Date N Date)
; 解釈: トラックのタイトル、アーティスト、アルバム、再生時間 (ミリ秒)、
```

```
(define-struct date [year month day hour minute second])
; A Date is a structure:
;   (make-date N N N N N N)
; 解釈: 年、月 (1~12)、日 (1~31)、時間 (0~23)、分 (0~59)、秒 (0~59) を記録する。
```

```
; create-track : Any Any Any Any Any Any Any -> Track or #false  
; 引数が正当な値であれば Track 構造体のインスタンスを生成し、そうでなければ #false を返す。  
  
; create-date : Any Any Any Any Any Any -> Date or #false  
; 引数が正当な値であれば Date 構造体のインスタンスを生成し、そうでなければ #false を返す。  
  
; read-itunes-as-tracks : String -> LTracks  
; XMLファイルから読み込み、トラック構造体のリスト LTracks を生成する。
```

練習問題 199. Date、Track、および LTracks の例を作成してください。これらは以降の練習問題でのテスト入力として役立ちます。

練習問題 201. 関数 `select-all-album-titles` を設計してください。この関数は `LTracks` を受け取り、すべてのアルバムタイトルのリストを文字列のリストとして返します。また、文字列のリストを受け取り、重複を排除したユニークな値のリスト(集合)を返す関数 `create-set` を設計してください。最後に、これらを組み合わせて `select-album-titles/unique` を設計し、あなたのライブラリに含まれるユニークなアルバムのリストと、その総数を求めてください。

練習問題 203. アルバム名、日付 `Date`、および `LTracks` を受け取り、そのアルバムに属し、かつ指定された日付より後に再生されたトラックのリストを返す関数 `select-album-date` を設計してください。

図 77 は、もう 1 つの表現であるリストベースの表現を示しています。

; ティーチパックのドキュメント (パート 2) :

```

; An LLists is one of:
; -- '()
; -- (cons LAssoc LLists)

; An LAssoc is one of:
; -- '()
; -- (cons Association LAssoc)

; An Association is a list of two items:
;   (cons String (cons BSDN '()))

; A BSDN is one of:
; -- Boolean
; -- Number
; -- String
; -- Date

; read-itunes-as-lists : String -> LLists
; XMLファイルから読み込み、リスト形式で表されたトラックのリスト LLists を生成する。

```

この表現では、各トラックが連想リスト LAssoc (キーと値のペアのリスト) として表され、ライブラリ全体がそのリスト LLists として表されます。

練習問題 205. LAssoc と LLists の例を作成してください。

練習問題 206. キー (文字列)、連想リスト LAssoc、およびデフォルト値を受け取り、キーに一致する最初の Association を返す関数 find-association を設計してください。見つからない場合はデフォルト値を返します。

練習問題 207. LLists を受け取り、総再生時間を計算する関数 total-time/list を設計してください。

練習問題 208. LLists を受け取り、ユニークなアルバムタイトルのセットを返す関数 select-album-titles/unique/list を設計してください。

12.3 ワードゲーム：合成 (Word Games: Composition)

多くの言葉遊び (アナグラムやスペリングゲームなど) では、特定の文字の組み合わせから生成できる単語を見つける必要があります。このセクションでは、与えられた単語の文字を並べ替えて、辞書に存在する意味のある単語 (アナグラム) をすべて見つけるプログラムを設計します。

この問題は、以下の2つの主要なタスクに分解できます。1. 与えられた単語の文字のすべての順列 (並べ替え) を生成する。2. 生成された文字列のリストのうち、辞書に存在する単語だけをフィルタリングする。

練習問題 215. 単語（文字列）と辞書 Dictionary を受け取り、その単語の文字を並べ替えて作ることができる、辞書に存在するすべての単語のリストを返す関数 `alternative-words` を設計してください。

練習問題 216. 文字列を文字（`lString`）のリストに変換する `explode` や、文字のリストを文字列に結合する `implode` を使って、単語ゲームプログラムのテストを整理してください。

12.4 ワードゲーム：問題の核心 (Word Games: The Heart of the Problem)

文字のすべての順列を生成することは、このプログラムの最も難しい部分であり、合成による設計の優れた練習になります。

文字のリストを受け取り、そのすべての順列（リストのリスト）を返す関数 `arrangements` を設計します。

```
; List-of-lStrings -> List-of-List-of-lStrings
; 与えられた文字リストのすべての順列を生成する
(define (arrangements w)
  '())
```

この再帰関数を設計するために、次のように考えます。リストの最初の文字 `char` を取り出し、残りの文字リストのすべての順列を再帰的に生成します。そして、その再帰結果の各順列の「すべての可能な位置」に `char` を挿入します。

練習問題 217. 文字 `lString` と、順列のリスト `List-of-List-of-lStrings` を受け取り、その文字を各順列のすべての可能な挿入位置（先頭、中間、末尾など）に挿入して得られる新しい順列のリストを返す関数 `insert-everywhere/in-all-words` を設計してください。

12.5 ワードゲーム再訪とワームゲーム (Worm Game)

これまでに設計したワーム（スネーク）ゲームを拡張します。リストを使用してワームの体を表現します。ワームの体は、頭から尾に向かう `Posn` のリストとして表されます。

```
; A Worm is a List-of-posns.
; 解釈: リストの最初の要素は頭の位置で、後続の要素は体の各セグメントの位置。
```

練習問題 218. ワームゲームの設計レシピに従って、ワームが壁や自分自身の体に衝突したときにゲームオーバーになるように、ゲームの終了条件を再設計してください。

練習問題 219. ワームがエサを食べたときに体が 1 セグメント分成長し、新しいエサがランダムな位置に再配置されるようにゲームを拡張してください。エサがワームの体と重ならないように配置するロジックを実装してください。

12.6 シンプル・テトリス (Simple Tetris)

テトリスは有名な落ち物パズルゲームです。このゲームの状態をリストと構造体を使って設計します。

```
(define-struct tetris [block landscape])  
; A Tetris is a structure:  
;   (make-tetris Block Landscape)  
; A Landscape is a List-of-blocks.
```

ここで、Block は現在落下中のブロックの座標を表し、Landscape はこれまでに着地して積み上がったブロックのリストです。

練習問題 220. 与えられた Tetris 状態をキャンバスに描画する関数 `tetris-render` を設計してください。

練習問題 221. ブロックがまっすぐ落下し、床または積み上がった他のブロックに着地すると、次の新しいブロックが生成されて落下を開始する、基本的なテトリスゲーム `tetris-main` を設計してください。

練習問題 222. プレイヤーが左右の矢印キーを使用して、落下中のブロックを左右に移動できるようにゲームを拡張してください。ブロックが壁を突き抜けたり、すでに積み上がっているブロックと重なったりしないように制御してください。

練習問題 223. ブロックがキャンバスの最上部に達した（積み上がりすぎた）ときにゲームが終了する機能を追加してください。

12.7 フル・スペースウォー (Full Space War)

第1部で設計したスペースインベーダーゲームを拡張し、プレイヤーが任意の数のミサイルを同時に発射できるようにします。ミサイルのリストを処理するためにリストを使用します。

練習問題 224. ミサイルのリストを管理するデータ定義を作成し、インベーダー (UF0) を撃墜できる完全なスペースウォーゲームを設計してください。

練習問題 225. 同様の設計手法を使用して、森林火災を消火する空中消火ゲーム（飛行機から水を投下して地上でランダムに発生する火災を消火するゲーム）を設計してください。

12.8 有限状態機械 (Finite State Machines)

有限状態機械 (FSM) を解釈・シミュレートする一般的なプログラムを設計します。

図 82 は、FSM のデータ表現を示しています。

図 82: 有限状態機械の表現

```

; An FSM is one of:
; -- '()
; -- (cons Transition FSM)

(define-struct transition [current next])
; A Transition is a structure:
;   (make-transition FSM-State FSM-State)

; A FSM-State is a Color.

```

FSMは遷移のリストとして表され、状態は色の名前（文字列）です。任意のキー入力によって状態が遷移します。

練習問題 226. 2つの状態が等しいかどうかを判定する述語 `state=?` を設計してください。

練習問題 227. 黒と白が交互に入れ替わる「BW マシン」の FSM データ表現を作成してください。

練習問題 228. 現在の FSM 状態とキー入力 `KeyEvent` を受け取り、次の状態を遷移テーブルから検索して返す関数 `find-next-state` を設計してください。

13 まとめ (Summary)

本書のこの第2部は、任意サイズのデータを扱うプログラムの設計についてでした。容易に想像できるように、事前にサイズの上限が決められていない情報に対して使用されるとき、ソフトウェアは特にその本領を発揮します。つまり、「任意サイズのデータ」を扱うことは、本物のプログラマーになるための道のりにおける極めて重要なステップです。この精神に基づき、以下の3つの教訓を持ち帰ることを提案します。

1. データ定義における自己参照と相互参照を扱うための、設計レシピの洗練：データ定義に自己参照が現れる場合は「再帰関数」の設計が必要になり、相互参照（他のデータ定義への参照）が現れる場合は「補助関数」の設計が必要になります。
2. 複雑な問題の、個別の問題への分解：問題を分解するときには、2つの要素が必要です。それは、個別の問題を解決する関数と、それらの個別的な解決策を1つの解決策へと組み立てるデータ定義です。個々のプログラムに時間を費やした後に、それらを組み合わせた合成プログラムが正しく機能することを保証するためには、必要なデータ定義とともに「願望リスト（ウィッシュリスト）」を定式化しておく必要があります。この「分解と合成による設計」は、問題文が暗黙的または明示的に補助的なタスクを示唆している場合や、ある関数を記述するステップで別の大規模データを走査する必要がある場合、そして（おそらく驚くべきことに）具体的な問題よりも一般的な問題を解決する方が簡単な場合に、特に役立ちます。
3. 実用的な事柄（プラグマティクス）の重要性：big-bang プログラムを設計したいのであれば、その様々な句（句ごとの役割）と、それらが何を達成するのかを理解する必要があります。あるいは、数学的な問題を解決するプログラムを設計するのであれ

ば、選択した言語とそのライブラリがどのような数学的オペレーションを提供しているかを確認しておく必要があります。

この部では、任意サイズのデータの好例として主に「リスト」に焦点を当てました。それは、Haskell、Lisp、ML、Racket、Scheme などの言語においてリストが実用的に非常に有用だからですが、ここで得たアイデアは、ファイル、フォルダ、データベースなど、あらゆる種類の任意サイズデータに適用できます。

「第4部 絡み合ったデータ (Intertwined Data)」では、より大規模で複雑な構造を持つデータの探求を続け、最も複雑なデータに対して設計レシピをどのようにスケールさせるかを示します。その前に、次の部 (第3部) では、プログラマーの仕事が「何度も何度も同じようなプログラムを作り続けること」ではないかという、現時点で抱くかもしれない重要な懸念に対処します。

II 任意サイズのデータ (Arbitrarily Large Data)

Intermezzo 2: Quote, Unquote

(今後の翻訳作業のためのプレースホルダー)

III 抽象化 (Abstraction)

導入：抽象化の必要性

多くのデータ定義と関数定義は似ています。例えば、String のリストの定義は Number のリストの定義と、データクラスの名前と「String」「Number」という単語の2箇所しか異なりませ。同様に、String のリストから特定の文字列を探す関数は、Number of lists から特定の数を探す関数とほとんど区別が付きません。

経験から、このような類似性は問題を引き起こすことがわかっています。類似性は、プログラマーがコードを (物理的または精神的に) コピーすることから生じます。プログラマーが別の問題に直面したとき、既存の解決策をコピーして新しい問題に合わせて修正します。この行動は「本物の」プログラミングだけでなく、スプレッドシートや数学的モデリングの世界でも見られます。

しかし、コードのコピーは以下のような問題を引き起こします： - ミスのコピー - 同じ修正を多くのコピーに適用する必要性 - 基礎データ定義が変更されたときにすべてのコピーを探して修正するコスト

このプロセスは高価でエラーが発生しやすく、プログラミングチームに不必要なコストを課します。

良いプログラマーは、プログラミング言語が許す限り類似性を排除しようとします。

「プログラムはエッセイのようなものです。最初のバージョンは下書きであり、下書きには編集が必要です。」

「排除する」とは、プログラマがプログラムの最初のドラフトを書き留め、類似性（および他の問題）を見つけ、それらを取り除くことを意味します。最後のステップでは、抽象化を行うか、既存の（抽象化された）関数を使用します。このプロセスを何度か繰り返すことで、プログラムを満足のいく形にすることがよくあります。

このパートの前半では、関数とデータ定義の類似性を抽象化する方法を示します。プログラマはこのプロセスの結果も「抽象化」と呼びます。後半は、既存の抽象化の使用と、このプロセスを容易にする新しい言語要素についてです。このパートの例はリストの領域から取られていますが、考え方は普遍的に適用可能です。

14 類似性はどこにでもある (Similarities Everywhere)

Arbitrarily Large Data の演習（の一部）を解いた人は、多くの解決策が似ていることを知っているでしょう。実際、類似性は、ある問題の解決策をコピーして次の問題の解決策を作成する誘惑に駆られるかもしれません。

しかし、thou shall not steal code（コードを盗んてはならない）、たとえ自分のコードであってもです。代わりに、類似したコード片を抽象化しなければなりません。この章では、その抽象化の方法を教えます。

私たちの手段は「Intermediate Student Language」（ISL）に特有のものです。

DrRacket で、「Language」メニューの「How to Design Programs」サブメニューから「Intermediate Student」を選択してください。

ほとんどすべての他のプログラミング言語も同様の手段を提供しています。オブジェクト指向言語では、追加の抽象化メカニズムが見つかるかもしれません。いずれにせよ、これらのメカニズムは本章で述べる基本的な特性を共有しており、ここで説明する設計の考え方は他の文脈でも適用されます。

14.1 関数における類似性 (Similarities in Functions)

設計レシピは、関数の基本的な組織を決定します。なぜなら、テンプレートは関数の目的に関係なくデータ定義から作成されるからです。したがって、同じ種類のデータを消費する関数は似ているのは当然です。

以下は 2 つの非常に似た関数です（図 86）：

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (cond
    [(empty? l) #false]
    [else
```

```

      (or
        (string=? (first l) "dog")
        (contains-dog? (rest l))))))
; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (cond
    [(empty? l) #false]
    [else
     (or
      (string=? (first l) "cat")
      (contains-cat? (rest l))))]))

```

図 86: 2つの類似した関数

これらの関数はほぼ区別が付きません。各々は文字列のリストを消費し、本体は2つの節を持つ cond 式で構成されます。入力为空リストなら #false を返し、or を使って最初の要素が目的の文字列かどうかを調べ、そうでなければ再帰的に残りを調べます。唯一の違いは比較する文字列です（図 86 の “dog” と “cat”）。

良いプログラマは、このような密接に関連する関数を複数定義するのを避けます。代わりに、探す文字列を追加の引数として受け取る 1 つの汎用関数を定義します：

```

; String Los -> Boolean
; determines whether l contains the string s
(define (contains? s l)
  (cond
    [(empty? l) #false]
    [else
     (or (string=? (first l) s)
         (contains? s (rest l))))]))

```

これで contains-dog? と contains-cat? は 1 行で定義できます：

```

; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (contains? "dog" l))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (contains? "cat" l))

```

図 87: 2つの類似した関数、再考

これが関数的抽象化（Functional Abstraction）です。異なるバージョンの関数を抽象化することは、プログラムから類似性を排除する一つの方法であり、類似性を排除することで、長期にわたってプログラムを健全に維持することが容易になります。

Exercise 235. `contains?` を使って “atom”, “basic”, “zoo” を探す関数を定義せよ。

Exercise 236. 次の2つの関数のテストスイートを作成せよ：

```
; Lon -> Lon
; adds 1 to each item on l
(define (add1* l)
  (cond
    [(empty? l) '()]
    [else
     (cons (add1 (first l)) (add1* (rest l)))]))

; Lon -> Lon
; adds 5 to each item on l
(define (plus5 l)
  (cond
    [(empty? l) '()]
    [else
     (cons (+ (first l) 5) (plus5 (rest l)))]))
```

これらを抽象化せよ。抽象化した関数を使って上記2つを1行で再定義し、テストで確認せよ。最後に、各数から2を引く関数を設計せよ。

14.2 異なる類似性 (Different Similarities)

`contains-dog?` と `contains-cat?` のケースでは抽象化は簡単に見えました。2つの関数定義を比較し、リテラル文字列を関数パラメータに置き換え、元の関数が抽象関数を使って簡単に定義できることを確認するだけでした。この種の抽象化は非常に自然なため、本書の前半2つのパートでも特に断ることなく登場していました。

本節では、これと同じ原理が、より強力な抽象化の形態をどのように生み出すかを示します。図 88 を見てください。どちらの関数も数値のリストと閾値を消費します。左側は閾値未満のすべての数値のリストを生成し、右側は閾値を超えるすべての数値のリストを生成します。

```
; Lon Number -> Lon
; select those numbers on l
; that are below t
(define (small l t)
  (cond
    [(empty? l) '()]
    [else
     (cond
       [(< (first l) t) (cons (first l) (small (rest l) t))]
       [else (small (rest l) t)])]))

; Lon Number -> Lon
; select those numbers on l
; that are above t
(define (large l t)
```

```
(cond
  [(empty? l) '()]
  [else
   (cond
    [(> (first l) t) (cons (first l) (large (rest l) t))]
    [else (large (rest l) t))])])
```

図 88: さらに 2 つの類似した関数

この 2 つの関数は、与えられたリストの数値が結果の一部になるべきかどうかを決定する「比較演算子」の 1 箇所だけが異なります。左側の関数は `<` を使用し、右側は `>` を使用しています。それ以外は、関数名を除いて 2 つの関数は同一です。

最初の例に従って、追加のパラメータを使ってこれら 2 つの関数を抽象化しましょう。今回は、追加のパラメータは文字列ではなく、比較演算子（関数）を表します：

```
(define (extract R l t)
  (cond
    [(empty? l) '()]
    [else
     (cond
      [(R (first l) t)
       (cons (first l) (extract R (rest l) t))]
      [else
       (extract R (rest l) t))])])
```

この新しい関数を適用するには、3 つの引数を指定する必要があります：2 つの数値を比較する関数 `R`、数値のリスト `l`、そして閾値 `t` です。この関数は、`(R i t)` が `#true` に評価される `l` 内のすべての要素 `i` を抽出します。

[!NOTE] ここで、この定義が意味をなすのか疑問に思うかもしれません。私たちは何の苦労もなく、「関数を消費する関数」を作成しました。微分演算や不定積分を学んだことがあるなら、これらが関数を消費して関数を生成する関数であることを知っているでしょう。しかし、本コースではその知識を仮定しません。私たちの単純な学習言語 ISL はこのような関数をサポートしており、このような関数を定義することは優れたプログラマの最も強力なツールの 1 つです。

テストを実行すると、`(extract < l t)` が `(small l t)` と同じ結果を計算することがわかります：

```
(check-expect (extract < '() 5) (small '() 5))
(check-expect (extract < '(3) 5) (small '(3) 5))
(check-expect (extract < '(1 6 4) 5) (small '(1 6 4) 5))
```

同様に、`(extract > l t)` は `(large l t)` と同じ結果を生成します。つまり、元の 2 つの関数を次のように再定義できます：

```
; Lon Number -> Lon
(define (small-1 l t) (extract < l t))
```

```
; Lon Number -> Lon
(define (large-1 l t) (extract > l t))
```

重要な洞察は、small-1 と large-1 が1行で定義できることだけではありません。extract のような抽象関数があれば、他の場所でも有効に活用できます： * (extract = l t)： l 内の t と等しい数値をすべて抽出します。 * (extract <= l t)： l 内の t 以下の数値をすべて抽出します。 * (extract >= l t)： l 内の t 以上の数値をすべて抽出します。

実際、extract の第1引数は ISL の定義済み演算である必要はありません。2つの引数を消費して真偽値を生成する任意の関数を使用できます。次の例を考えてみましょう：

```
; Number Number -> Boolean
; is the area of a square with side x larger than c
(define (squared>? x c)
  (> (* x x) c))
```

すなわち、squared>? は平方 $x^2 > c$ が成り立つかどうかを検査し、extract で使用できます：

```
(extract squared>? (list 3 4 5) 10)
```

この適用により、(list 3 4 5) の中で平方した値が 10 より大きい数値（この場合は 4 と 5）が抽出されます。

Exercise 237. DrRacket で (squared>? 3 10) と (squared>? 4 10) を評価せよ。(squared>? 5 10) はどうなるか。

抽象化された関数定義は、元の関数よりも便利になり得ることがわかりました。たとえば、contains? は contains-dog? や contains-cat? よりも便利であり、extract は small や large よりも便利です。抽象化のこれらの利点は、文書作成、スプレッドシート、小さなアプリ、大規模な産業プロジェクトなど、あらゆるレベルのプログラミングで得られます。

また、抽象化のもう1つの重要な側面は、これらの関数に対して単一の制御点 (Single Point of Control) を持てることです。抽象関数に間違いがあることが判明した場合、その定義を修正するだけで、他のすべての定義も修正されます。同様に、抽象関数の計算を高速化する方法を見つければ、この関数を使用して定義されたすべての関数が追加の労力なしに改善されます。

```
; NeLon -> Number
; determines the smallest number on l
(define (inf l)
  (cond
    [(empty? (rest l)) (first l)]
    [else
```

```

      (if (< (first l) (inf (rest l)))
          (first l)
          (inf (rest l))))))
; Nelon -> Number
; determines the largest number on l
(define (sup l)
  (cond
    [(empty? (rest l)) (first l)]
    [else
     (if (> (first l) (sup (rest l)))
         (first l)
         (sup (rest l))))]))

```

図 89: リスト内の最小値 (inf) と最大値 (sup) の探索

Exercise 238. 図 89 の 2 つの関数を 1 つの関数に抽象化せよ。どちらも空でない数値のリスト (Nelon) を消費し、1 つの数値を生成します。左側はリスト内の最小値を生成し、右側は最大値を生成します。

抽象関数を用いて inf-1 と sup-1 を定義せよ。以下の 2 つのリストでテストせよ：

```

(list 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25)

```

なぜこれらの関数は長いリストの一部で遅くなるのか。元の関数を min (2 つの数値から小さい方を選択) および max (大きい方を選択) を使って修正せよ。その後、再度抽象化し、inf-2 と sup-2 を定義して、同じ入力でテストせよ。なぜこれらのバージョンははるかに高速になるのか。

14.3 データ定義における類似性 (Similarities in Data Definitions)

ここで、次の 2 つのデータ定義を詳しく見てみましょう：

```

; An Lon (List-of-numbers) is one of:
; - '()
; - (cons Number Lon)
; An Los (List-of-strings) is one of:
; - '()
; - (cons String Los)

```

左側は数値のリストを導入し、右側は文字列のリストを記述しています。そして、これら 2 つのデータ定義は非常に似ています。類似した関数と同様に、2 つのデータ定義は異なる名前を使用していますが、名前は任意であるためこれは重要ではありません。唯一の本当の違いは、2 番目の節の cons の最初の位置にあり、リストがどのような種類の要素を含むかを指定している点です。

この 1 つの違いを抽象化するために、データ定義を関数であるかのように扱います。パラメータを導入し、異なる型 (要素) を参照していた場所にこのパラメータを使用します：

```
; A [List-of ITEM] is one of:
; - '()
; - (cons ITEM [List-of ITEM])
```

このような抽象的なデータ定義をパラメータ化されたデータ定義 (Parametric Data Definitions) と呼びます。大雑把に言えば、パラメータ化されたデータ定義は、関数が特定の値から抽象化するのと同様に、特定のデータコレクションへの参照から抽象化します。

問題は、もちろん、これらのパラメータがどのような範囲をとるかです。関数のパラメータが「未知の値」を表すのに対し、データ定義のパラメータは「データのクラス全体 (型)」を表します。パラメータ化されたデータ定義に具体的なデータコレクションの名前を割り当てるプロセスをインスタンス化 (Instantiation) と呼びます。以下は List-of 抽象化のサンプルインスタンス化です： * [List-of Number] と書くとき、ITEM が Number を表すため、これは List-of-numbers の別名になります。 * 同様に、[List-of String] は List-of-strings と同じデータクラスを定義します。 * 以下のような在庫レコード IR のクラスを定義した場合： racket (define-struct IR [name price]) ; An IR is a structure: (make-IR String Number) [List-of IR] は在庫レコードのリストの名前になります。

慣例として、データ定義のパラメータには大文字の名前 (例：ITEM, x, y) を使用します。

この抽象データ定義が正当であることを確認するには、パラメータ ITEM に実際のデータ定義名 (たとえば Number) を代入し、自己参照 (再帰) を展開してみます。得られる定義は従来の数値リストの定義と完全に一致します。

2 番目の例として、次の構造体定義から始めましょう：

```
(define-struct point [hori veri])
```

この構造体タイプを使用する 2 つの異なるデータ定義を示します：

```
; A Pair-boolean-string is a structure:
; (make-point Boolean String)
```

```
; A Pair-number-image is a structure:
; (make-point Number Image)
```

この場合、データ定義は 2 箇所で異なります。hori フィールドに対応する違いと、veri フィールドに対応する違いです。したがって、抽象データ定義を作成するには 2 つのパラメータを導入する必要があります：

```
; A [CP H V] is a structure:
; (make-point H V)
```

ここで、H は hori フィールドのデータコレクション用のパラメータであり、V は veri フィールドに表示されるデータコレクション用のパラメータです。

2つのパラメータを持つデータ定義をインスタンス化するには、データコレクションの2つの名前が必要です。パラメータに `Number` と `Image` を使用すると、数値と画像を組み合わせたポイント構造体のコレクションを記述する `[CP Number Image]` が得られます。対照的に、`[CP Boolean String]` は真偽値と文字列をポイント構造体で組み合わせます。

Exercise 239. 2つの要素のリストは、ISL プログラミングで頻繁に使用されるもう1つのデータ形式です。以下は、2つのパラメータを持つデータ定義です：

```
; A [List X Y] is a list:  
;   (cons X (cons Y '()))
```

この定義をインスタンス化して、以下のデータクラスを記述せよ：1. 数値のペア (`Number` と `Number`) 2. 数値と `lString` のペア 3. 文字列と真偽値のペア

また、これら3つのデータ定義のそれぞれについて、具体的な例を1つずつ作成せよ。

パラメータ化されたデータ定義が得られたら、それらを組み合わせて大きな効果を得ることができます。次の定義を考えてみましょう：

```
; [List-of [CP Boolean Image]]
```

最も外側の表記は `[List-of ...]` であり、これはリストを扱っていることを意味します。このリストがどのようなデータを含むかという問いに答えるには、内部の式を調べる必要があります：`[CP Boolean Image]`。これは真偽値と画像をポイント内で組み合わせたものです。含意として、上記は真偽値と画像を組み合わせたポイントのリストです。同様に、

```
; [CP Number [List-of Image]]
```

は、1つの数値と画像のリストを組み合わせた `CP` のインスタンス化です。

Exercise 240. 奇妙ですが類似した次の2つのデータ定義があります：

```
; An LStr is one of:  
; - String  
; - (make-layer LStr)  
; An LNum is one of:  
; - Number  
; - (make-layer LNum)
```

どちらのデータ定義も、次の構造体定義に依存しています：

```
(define-struct layer [stuff])
```

両者について具体例を作成せよ。2つを抽象化したデータ定義を作成せよ。そして、抽象定義をインスタンス化して元の定義を復元せよ。

Exercise 241. `NEList-of-temperatures` と `NEList-of-Booleans` の定義を比較せよ。そして、空でないリストを表す抽象データ定義 `NEList-of` を策定せよ。

Exercise 242. 以下は、もう1つのパラメータ化されたデータ定義です：

```
; A [Maybe X] is one of:
; - #false
; - X
```

これらのデータ定義を解釈せよ：[Maybe String], [Maybe [List-of String]], [List-of [Maybe String]]。

次の関数署名は何を意味するか：

```
; String [List-of String] -> [Maybe [List-of String]]
; returns the remainder of los starting with s
; #false otherwise
(check-expect (occurs "a" (list "b" "a" "d" "e")) (list "d" "e"))
(check-expect (occurs "a" (list "b" "c" "d")) #false)
(define (occurs s los) los)
```

設計レシピの残りのステップを実行せよ。

14.4 関数は値である (Functions Are Values)

本パートの関数は、プログラムの評価に関する私たちの理解を拡張します。関数が数値だけでなく、文字列や画像を消費することは理解しやすいでしょう。構造体やリストは少し拡張が必要ですが、結局のところ有限の「もの」です。しかし、関数を消費する関数は奇妙に思えます。実際、このアイデア自体が最初のインターメッツォ (BSL) の規則に2つの方法で違反しています：1. プリミティブや関数の名前が、適用式の引数として使用されている。2. パラメータが、適用式の「関数位置」で使用されている。

問題を詳しく説明すると、ISLの文法がBSLとどのように異なるかがわかります。第1に、式言語は定義内の関数名とプリミティブ演算の名前を含めるべきです。第2に、適用式の最初の位置は、関数名やプリミティブ演算以外のもの (変数や関数のパラメータなど) を許容しなければなりません。

文法への変更は評価規則への変更を要求するように見えますが、変更されるのは「値の集合」だけです。具体的には、関数を関数の引数として受け入れるために、最も単純な変更は「関数やプリミティブ演算もまた値である」とすることです。

Exercise 243. DrRacket の定義領域に以下が含まれていると仮定します：

```
(define (f x) x)
```

以下の式のうち、値 (値の定義を満たすもの) を特定せよ：1. (cons f '()) 2. (f f) 3. (cons f (cons 10 (cons (f 10) '())))

なぜそれらが値である (または値でない) のかを説明せよ。

Exercise 244. 次の記述がISLにおいて合法 (文法的に正しい) である理由を論じよ：1. (define (f x) (x 10)) 2. (define (f x) (x f)) 3. (define (f x y) (x 'a y 'b))

あなたの推論を説明せよ。

Exercise 245. `function=at-1.2-3-and-5.775?` 関数を設計せよ。数値から数値への2つの関数が与えられたとき、この関数は、1.2、3、および -5.775 に対して2つの関数が同じ結果を生成するかどうかを判定します。

数学者は、同じ入力を与えられたときに同じ結果を計算する場合、2つの関数は等しいと言います。数値から数値への2つの関数が等しいかどうかを判定する一般関数 `function=?` を定義することは可能でしょうか。もし可能なら定義せよ。不可能なら、その理由を説明し、これが「簡単に定義できるアイデアであるにもかかわらず、その関数を定義できない最初の例」に遭遇したことを意味するインプリケーションを考慮せよ。

14.5 関数を使った計算 (Computing with Functions)

BSL+ から ISL への移行により、関数を引数として使用し、適用式の最初の位置にパラメータ名を使用できるようになりました。DrRacket はこれらの位置にある名前を他の場所と同様に処理しますが、当然ながら結果として関数を期待します。驚くべきことに、代数の法則を単純に適用するだけで、ISL プログラムを評価することができます。

`extract` の評価がどのように機能するかを見てみましょう。明らかに、

```
(extract < '() 5) == '()
```

が成り立ちます。「Intermezzo 1」の代入モデルを使用して、関数の本体で計算を続けることができます。パラメータ `R`, `l`, `t` が、引数 `<`, `'()`, `5` にそれぞれ置き換えられます。ここからは、条件式の評価から始まるプレーンな計算です：

```
==
(cond
  [(empty? '()) '()]
  [else
   (cond
     [(< (first '()) 5)
      (cons (first '()) (extract < (rest '()) 5))]
     [else (extract < (rest '()) 5)]]])
==
(cond
  [#true '()]
  [else
   (cond
     [(< (first '()) 5)
      (cons (first '()) (extract < (rest '()) 5))]
     [else (extract < (rest '()) 5)]]])
== '()
```

次に、要素が1つのリストを考えます：

```
(extract < (cons 4 '()) 5)
```

リストの唯一の要素は 4 であり、`(< 4 5)` は真であるため、結果は `(cons 4 '())` になるはずです。評価の最初のステップは次のようになります：

```

(extract < (cons 4 '()) 5)
==
(cond
  [(empty? (cons 4 '())) '()]
  [else
   (cond
     [(< (first (cons 4 '())) 5)
      (cons (first (cons 4 '()))
            (extract < (rest (cons 4 '())) 5))]
     [else (extract < (rest (cons 4 '())) 5)]]])

```

やはり、R は <, l は (cons 4 '()), t は 5 に置き換えられます。残りは簡単です：

```

(cond
  [(empty? (cons 4 '())) '()]
  [else
   (cond
     [(< (first (cons 4 '())) 5)
      (cons (first (cons 4 '()))
            (extract < (rest (cons 4 '())) 5))]
     [else (extract < (rest (cons 4 '())) 5)]]])
==
(cond
  [#false '()]
  [else
   (cond
     [(< (first (cons 4 '())) 5)
      (cons (first (cons 4 '()))
            (extract < (rest (cons 4 '())) 5))]
     [else (extract < (rest (cons 4 '())) 5)]]])
==
(cond
  [(< (first (cons 4 '())) 5)
   (cons (first (cons 4 '()))
         (extract < (rest (cons 4 '())) 5))]
  [else (extract < (rest (cons 4 '())) 5)])
==
(cond
  [(< 4 5)
   (cons (first (cons 4 '()))
         (extract < (rest (cons 4 '())) 5))]
  [else (extract < (rest (cons 4 '())) 5)])

```

ここが重要なステップであり、代入された位置で < が実際に使用されています。そして、計算が続きます：

```

==
(cond
  [#true
   (cons (first (cons 4 '()))

```

```

      (extract < (rest (cons 4 '())) 5))
    [else (extract < (rest (cons 4 '())) 5)]]
  ==
  (cons 4 (extract < (rest (cons 4 '())) 5))
  ==
  (cons 4 (extract < '() 5))
  ==
  (cons 4 '())

```

最後の例は、2つの要素のリストへの適用です：

```

(extract < (cons 6 (cons 4 '())) 5)
== (extract < (cons 4 '()) 5)
== (cons 4 (extract < '() 5))
== (cons 4 '())

```

ステップ 1 は、閾値未満でない場合に `extract` がリストの最初の要素を除外するケースを扱っています。

Exercise 246. DrRacket の Stepper (ステップ実行) を使用して、最後の計算のステップ 1 を確認せよ。

Exercise 247. DrRacket の Stepper を使用して `(extract < (cons 8 (cons 4 '())) 5)` を評価せよ。

Exercise 248. DrRacket の Stepper で `(squared>? 3 10)` と `(squared>? 4 10)` を評価せよ。

次の対話を考えてみます：

```

> (extract squared>? (list 3 4 5) 10)
(list 4 5)

```

Stepper が示すステップの一部を以下に示します：

```

(extract squared>? (list 3 4 5) 10)
; (1) ==
(cond
  [(empty? (list 3 4 5)) '()]
  [else
   (cond
     [(squared>? (first (list 3 4 5)) 10)
      (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
     [else (extract squared>? (rest (list 3 4 5)) 10)]]])

; (2) == ... ==
(cond
  [(squared>? 3 10)
   (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
  [else (extract squared>? (rest (list 3 4 5)) 10)])

```

Stepper を使用して、行 (1) から (2) へのステップを確認せよ。さらにステップを進めて、ステップ (2) から最終結果までの隙間を埋めよ。各ステップを代入モデルや関数の評価ルールを用いて説明せよ。

Exercise 249. 関数は「値」であり、引数、結果、リストの要素になり得ます。以下の定義と式を DrRacket の定義ウィンドウに入力し、Stepper を使ってこのプログラムがどのように動作するかを調べよ：

```
(define (f x) x)
(cons f '())
(f f)
(cons f (cons 10 (cons (f 10) '()))))
```

15 抽象化の設計 (Designing Abstractions)

本質的に、抽象化とは「具体的なものをパラメータに変換すること」です。前章でもこれを行いました。類似した関数定義を抽象化するには、定義内の具体的な値を置き換えるパラメータを追加します。類似したデータ定義を抽象化するには、パラメータ化されたデータ定義を作成します。他のプログラミング言語に触れると、その抽象化メカニズムもパラメータの導入を必要とすることを目にするでしょう（それが関数の引数であるとは限りませんが）。

15.1 例からの抽象化 (Abstractions from Examples)

あなたが足し算を初めて学んだとき、具体的な例を使って作業しました。親はおそらく指を使って小さな2つの数を足すことを教えたでしょう。後になって、任意の2つの数を足す方法を学びました。そこで初めてある種の抽象化に触れました。さらに後で、摂氏から華氏への温度変換や、与えられた速度と時間で車が移動する距離を計算する式を定式化することを学びました。要するに、非常に具体的な例から抽象的な関係へと移行したのです。

```
; List-of-numbers -> List-of-numbers
; converts a list of Celsius temperatures to Fahrenheit
(define (cf* l)
  (cond
    [(empty? l) '()]
    [else
     (cons (C2F (first l)) (cf* (rest l)))]))

; Inventory -> List-of-strings
; extracts the names of toys from an inventory
(define (names i)
  (cond
    [(empty? i) '()]
    [else
     (cons (IR-name (first i)) (names (rest i)))]))
```

```

; Number -> Number
; converts one Celsius temperature to Fahrenheit
(define (C2F c)
  (+ (* 9/5 c) 32))

(define-struct IR [name price])
; An IR is a structure: (make-IR String Number)
; An Inventory is one of:
; - '()
; - (cons IR Inventory)

```

図 90: 似た 2 つの関数

本節では、例から抽象化を作成するための設計レシピを紹介します。抽象化を作成すること自体は簡単ですが、既存の抽象化を見つけて利用するという「難しい部分」は次の節に譲ります。

「類似性はどこにでもある」のエッセンスを思い出してください。2つの具体的な定義から始め、それらを比較し、違いをマークし、そして抽象化します。抽象化を作成する手順は、ほとんどこれだけです： 1. 類似点と相違点の比較 (Compare) ほぼ同じで、名前と特定の位置にある具体的な値だけが異なる2つの関数定義を見つけたら、それらを比較して相違点をマークします。もし違いが複数箇所にある場合は、対応する違い同士を結びつけます。図 90 は、類似した関数定義のペアを示しています。2つの関数はリストを処理し、各要素に関数を適用しています。違いは「どの関数を要素に適用するか」という点だけです。 2. パラメータ化 (Parameterize) 相違点としてマークした部分を新しいパラメータ名 (引数) に置き換え、パラメータリストに追加します。図 90 の例では、相違点を `g` という名前に置き換えます。これにより本質的な違いが排除されます。次に、非本質的な違い (関数名やリスト引数の名前など) を統一します。関数名を `map1`、リストパラメータを `k` と呼ぶことにすると、図 91 のように、完全に同一の関数定義が得られます。 racket

```

(define (map1 k g) (cond [(empty? k) '()] [else (cons (g (first k)) (map1 (rest k) g))]))

```

図 91: 抽象化された同一の関数定義

違いが複数ある場合は、対応する違いのラインごとに1つの追加パラメータを導入します。そして、すべての再帰呼び出し部分にも追加パラメータを忘れずに引き渡すように変更します。

3. 検証と再定義 (Validate) 新しい抽象関数が、元の関数の正しい抽象化であることを検証します。これは、元の関数を抽象関数を使って定義し直し、既存のテストスイートが正常に通るか確認することを意味します。元の関数が `f-original` で引数を1つ消費し、抽象関数が `abstract` であるとします。 `f-original` が他方と `val` という値の使用において異なる場合、次の定義を行います：

```

(define (f-from-abstract x)
  (abstract x val))

```

この `f-from-abstract` が `f-original` と同等であることをテストします。先ほどの例に戻ると：


```
(define (cf*-from-map1 l) (map1 l C2F))
(define (names-from-map1 i) (map1 i IR-name))
```

元のテストケースが以下のように定義されていたとします：

```
(check-expect (cf* (list 100 0 -40)) (list 212 32 -40))
(check-expect (names (list (make-IR "doll" 21.0) (make-IR "bear" 13.0)))
  (list "doll" "bear"))
```

これらを次のように書き換えて実行し、成功を確認します：

```
(check-expect (cf*-from-map1 (list 100 0 -40)) (list 212 32 -40))
(check-expect (names-from-map1 (list (make-IR "doll" 21.0) (make-IR "bear"
13.0)))
  (list "doll" "bear"))
```

- 署名の定式化 (Signature) 新しい抽象関数に署名 (Signature) を与えます。これについては 15.2 で詳しく扱います。

新しい抽象化を行ったら、他に使える用途がないか確認します。用途が多ければ、その抽象化は本当に有用です。たとえば、map1 を使って数値リストの各要素に 1 を加える関数は以下のように定義できます：

```
; List-of-numbers -> List-of-numbers
(define (add1-to-each l)
  (map1 l add1))
```

このような汎用的な関数がすでに言語の一部として提供されていることもよくあります (Chapter 16 を参照)。

```
; Number -> [List-of Number]
; tabulates sin between n and 0 (incl.) in a list
(define (tab-sin n)
  (cond
    [(= n 0) (list (sin 0))]
    [else (cons (sin n) (tab-sin (sub1 n)))]))

; Number -> [List-of Number]
; tabulates sqrt between n and 0 (incl.) in a list
(define (tab-sqrt n)
  (cond
    [(= n 0) (list (sqrt 0))]
    [else (cons (sqrt n) (tab-sqrt (sub1 n)))]))
```

図 92: 演習 250 のための類似した関数

```
; [List-of Number] -> Number
; computes the sum of the numbers on l
(define (sum l)
  (cond
    [(empty? l) 0]
    [else (+ (first l) (sum (rest l)))]))
```

```
; [List-of Number] -> Number
; computes the product of the numbers on l
(define (product l)
  (cond
    [(empty? l) 1]
    [else (* (first l) (product (rest l)))]))
```

図 93: 演習 251 のための類似した関数

Exercise 250. 図 92 の 2 つの関数の抽象化である `tabulate` を設計せよ。設計が完了したら、それを使って `sqr` (平方) と `tan` の表を作成する関数を定義せよ。

Exercise 251. 図 93 の 2 つの関数の抽象化である `fold1` を設計せよ。

Exercise 252. 図 94 の 2 つの関数の抽象化である `fold2` を設計せよ。この演習と演習 251 を比較せよ。どちらも `product` 関数を含んでいますが、この演習は 2 番目の関数 `image*` が `Posn` のリストを消費して `Image` を生成するため、難易度が高くなっています。得られた設計と前演習の設計を比較し、署名の抽象化についての洞察を得よ。

```
; [List-of Number] -> Number
(define (product l)
  (cond
    [(empty? l) 1]
    [else (* (first l) (product (rest l)))]))

; [List-of Posn] -> Image
(define (image* l)
  (cond
    [(empty? l) emt]
    [else (place-dot (first l) (image* (rest l)))]))

; Posn Image -> Image
(define (place-dot p img)
  (place-image dot (posn-x p) (posn-y p) img))

; 描画用定数
(define emt (empty-scene 100 100))
(define dot (circle 3 "solid" "red"))
```

図 94: 演習 252 のための類似した関数

15.2 署名における類似性 (Similarities in Signatures)

関数の署名 (Signature) は、その再利用の鍵となります。したがって、抽象化された関数を表現するのに十分な「最も一般的な署名」を記述する方法を学ぶ必要があります。

基本的に、署名はデータ定義と同じです。どちらもデータのクラスを指定しますが、データ定義はクラスに名前を付けるのに対し、署名はそうしません。それでも、以下のコード:

```
; Number Boolean -> String
(define (f n b) "hello world")
```

の最初の行は、Number と Boolean を消費して String を生成するすべての関数のクラスを表しています。

したがって、抽象化の設計レシピは署名にも適用できます。類似した署名を比較し、相違点を強調し、それらをパラメータで置き換えます。

cf* と names の元の署名を比較してみましょう： * cf* : [List-of Number] -> [List-of Number] * names : [List-of IR] -> [List-of String]

比較すると、矢印の左側で Number と IR が異なり、右側で Number と String が異なります。これら2つの相違点をパラメータ x と y に置き換えると、次のようになります：

```
; [List-of X] -> [List-of Y]
```

このシグネチャに現れる変数 x と y は、データ定義のパラメータ（Chapter 14.3）と同じようにデータのクラス全体を表します。これらをインスタンス化（代入）すると、元の署名が得られます：* x と y に Number を代入すると、cf* の署名になります。* x に IR、y に String を代入すると、names の署名になります。

次に、追加の関数引数を受け取る map1 の署名を考えます。map1 の署名は次のようになります：

```
; [X Y] [List-of X] [X -> Y] -> [List-of Y]
```

具体的には、map1 は x という型に属する要素のリストを消費します。また、x の要素を消費して y という（また別の）型の要素を生成する関数を消費します。そして、結果として y のリストを返します。

もう1つの例として、演習 252 の product と image* の署名を抽象化します：* product の署名： [List-of Number] Number [Number Number -> Number] -> Number * image* の署名： [List-of Posn] Image [Posn Image -> Image] -> Image

これらを比較すると、すべての Number が一様に対応しているわけではなく： 1. ある Number は Posn に対応している（リストの要素）。 2. 別の Number は Image に対応している（ベースケースと結果の型）。

したがって、2つのパラメータ x と y を導入し、fold2 の署名は次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
```

x と y の両方に Number を代入すると product の署名が得られ、x に Posn、y に Image を代入すると image* の署名が得られることを確認してください。

署名を抽象化するためのステップを以下にまとめます： 1. シグネチャの比較：2つの類似した関数の具体的な署名を比較し、相違点をマークします。 2. パラメータの抽出：抽象化によって追加されたパラメータが何を表しているかを特定し、対応するプレースホルダーを割り当てます。 3. 変数の置き換え：相違点を表す型パラメータ（x, y など）を

導入し、統一された汎用的な署名を作成します。 4. 検証: 型パラメータに具体的な型を代入し、元の個々の署名が正しく復元されることを確認します。

Exercise 253. 以下のそれぞれの署名は関数のクラスを表しています: 1. `[Number -> Boolean]` 2. `[Boolean String -> Boolean]` 3. `[Number Number Number -> Number]` 4. `[Number -> [List-of Number]]` 5. `[[List-of Number] -> Boolean]`

それぞれのクラスに属する具体的な関数の例を ISL で記述せよ。

Exercise 254. 以下の関数の署名を策定せよ: * `sort-n`: 数値のリストと、2つの数値を比較して真偽値を返す関数を消費し、ソートされた数値のリストを生成する。* `sort-s`: 文字列のリストと、2つの文字列を比較して真偽値を返す関数を消費し、ソートされた文字列のリストを生成する。

その後、2つの署名を抽象化せよ。さらに、一般化された署名が、在庫レコード (IR) のリストをソートする関数の署名としてインスタンス化できることを示せ。

Exercise 255. 以下の関数の署名を策定せよ: * `map-n`: 数値のリストと、数値から数値への関数を消費し、数値のリストを生成する。* `map-s`: 文字列のリストと、文字列から文字列への関数を消費し、文字列のリストを生成する。

その後、2つの署名を抽象化せよ。また、一般化された署名が上記 `map1` の署名としてインスタンス化できることを示せ。

15.3 単一の制御点 (Single Point of Control)

プログラムは文章の下書き (ドラフト) のようなものです。誤字の修正、文法エラーの修正、文章の一貫性の確保、そして重複の排除などの編集作業は、文章を良くするために不可欠です。誰も同じことを何度も繰り返す文章を読みたくはありませんし、そのようなプログラムも読みたくはありません。

重複を排除して抽象化を作成することには、多くの利点があります。定義がシンプルになり、既存の関数の潜在的な不具合 (境界の不一致など) が明らかになります。しかし、最も重要な利点は、共通の機能に対して単一の制御点 (Single Point of Control) を作ることです。

機能の定義を1箇所にとめることで、プログラムのメンテナンスが非常に容易になります: * バグ (間違い) を発見したとき、1箇所だけを修正すればよくなります。* 新しいデータ形式に対応する必要が生じたとき、1箇所にコードを追加するだけで済みます。* アルゴリズムを改善したり高速化したりしたとき、1箇所の変更ですべての利用箇所が恩恵を受けます。

もしコードをコピー&ペーストしていた場合、すべてのコピー箇所を探し出して修正しなければならず、ミスの修正漏れが発生する危険性が非常に高くなります。

したがって、次のガイドラインを強く推奨します:

[!IMPORTANT] コードをコピーして修正する代わりに、抽象化を作成せよ。

抽象化の構築はプログラミングスキルを高めるための最も強力な練習の1つです。優れたプログラマは、重複を見つけると積極的にプログラムをリファクタリング（編集）し、新しい抽象化を構築して制御点を1箇所に集中させます。

15.4 テンプレートからの抽象化 (Abstractions from Templates)

リストを処理する多くの関数は、データ定義から自動的に導かれる共通のテンプレートに従っています。したがって、関数が互いに似通った形になるのは必然です。

実は、私たちは個々の関数からだけでなく、テンプレートそのものから直接抽象化を作成することができます。リストを走査する関数の基本テンプレートを思い出してください：

```
(define (fun-for-l l)
  (cond
    [(empty? l) ...]
    [else (... (first l) ... (fun-for-l (rest l)) ...)]))
```

このテンプレートには、各節に1つずつ、合計2つの「空欄 (gap)」があります。リスト処理関数を定義するとき、最初の節の空欄をベース値で埋め、2番目の節の空欄を「組み合わせる（結合する）関数」で埋めます。この結合関数は、リストの最初の要素と、残りのリストに対する再帰呼び出しの結果を組み合わせます。

この仕組みをパラメータ化して抽象関数を定義すると、次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
(define (reduce l base combine)
  (cond
    [(empty? l) base]
    [else
     (combine (first l)
              (reduce (rest l) base combine))]))
```

この `reduce` を使うと、リスト処理関数の多くを1行で定義できるようになります。たとえば、数値のリストの総和 `sum` と総乗 `product` は次のように再定義できます：

```
; [List-of Number] -> Number
(define (sum lon) (reduce lon 0 +))

; [List-of Number] -> Number
(define (product lon) (reduce lon 1 *))
```

`sum` では、空リストの場合のベース値は `0` であり、組み合わせる関数は `+` です。`product` では、ベース値は `1` であり、組み合わせる関数は `*` です。このように、多くのリスト処理関数が `reduce` を使って簡潔に再定義できます。

17 無名関数 (Nameless Functions)

17.1 lambda から生まれる関数 (Functions from lambda)

lambda の構文は次のとおりです：

```
(lambda (variable-1 ... variable-N) expression)
```

特徴はキーワード lambda です。続いて括弧で囲まれた変数列、そして結果を計算する式があります。

例 (1 引数)：

```
(lambda (x) (expt 10 x))  
(lambda (x) (if (> x 0) x (- 0 x)))  
(lambda (x) (string-append "hello " x))
```

これらは無名関数です。インタラクショナルエリアで直接使用できます。

17.2 lambda を使った計算 (Computing with lambda)

lambda 式は関数値として評価されます。

17.3 lambda を使った抽象化 (Abstracting with lambda)

lambda を使ってその場で関数を作成し、map や filter と組み合わせます。

```
(map (lambda (x) (* x x)) (list 1 2 3))
```

17.4 lambda を使った仕様 (Specifying with lambda)

テストケースや目的文で lambda を使って動作を指定できます。

17.5 lambda を使った表現 (Representing with lambda)

lambda は関数を値として表現するための強力な手段です。

Intermezzo 3: Scope and Abstraction

(今後の翻訳作業のためのプレースホルダー)

IV 絡み合ったデータ (Intertwined Data)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 4: The Nature of Numbers

(今後の翻訳作業のためのプレースホルダー)

V 生成的再帰 (Generative Recursion)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 5: The Cost of Computation

(今後の翻訳作業のためのプレースホルダー)

VI 蓄積子 (Accumulators)

(今後の翻訳作業のためのプレースホルダー)

エピローグ: 先へ進む (Epilogue: Moving On)

(今後の翻訳作業のためのプレースホルダー)